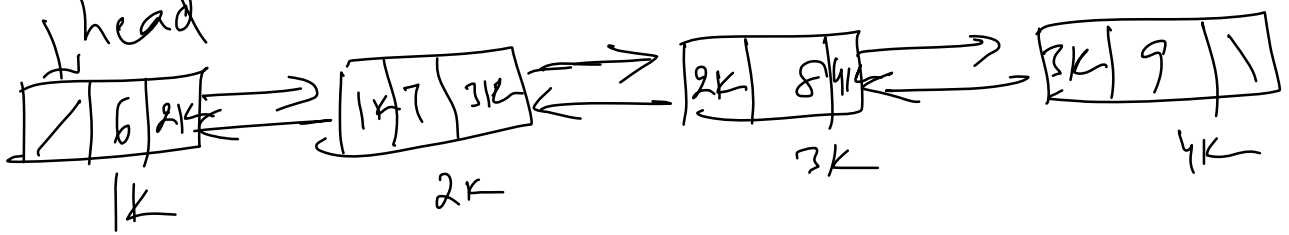
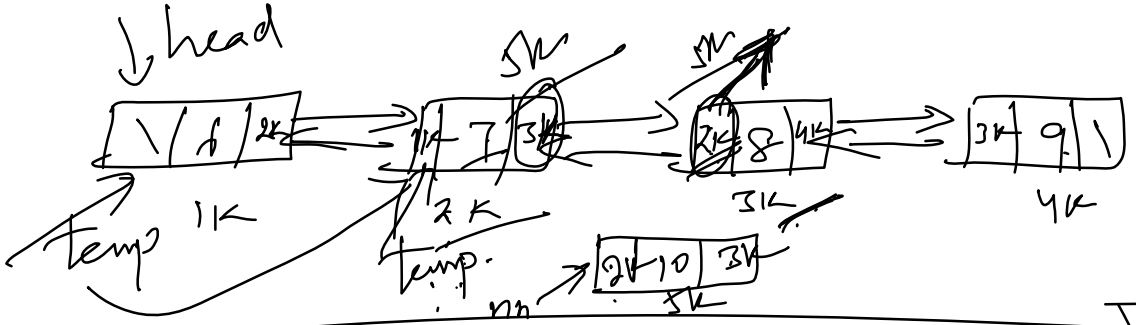
Add at a specified position



if $p^{os} == 1 \Rightarrow$ add at beg

if pos is not present → add as last node

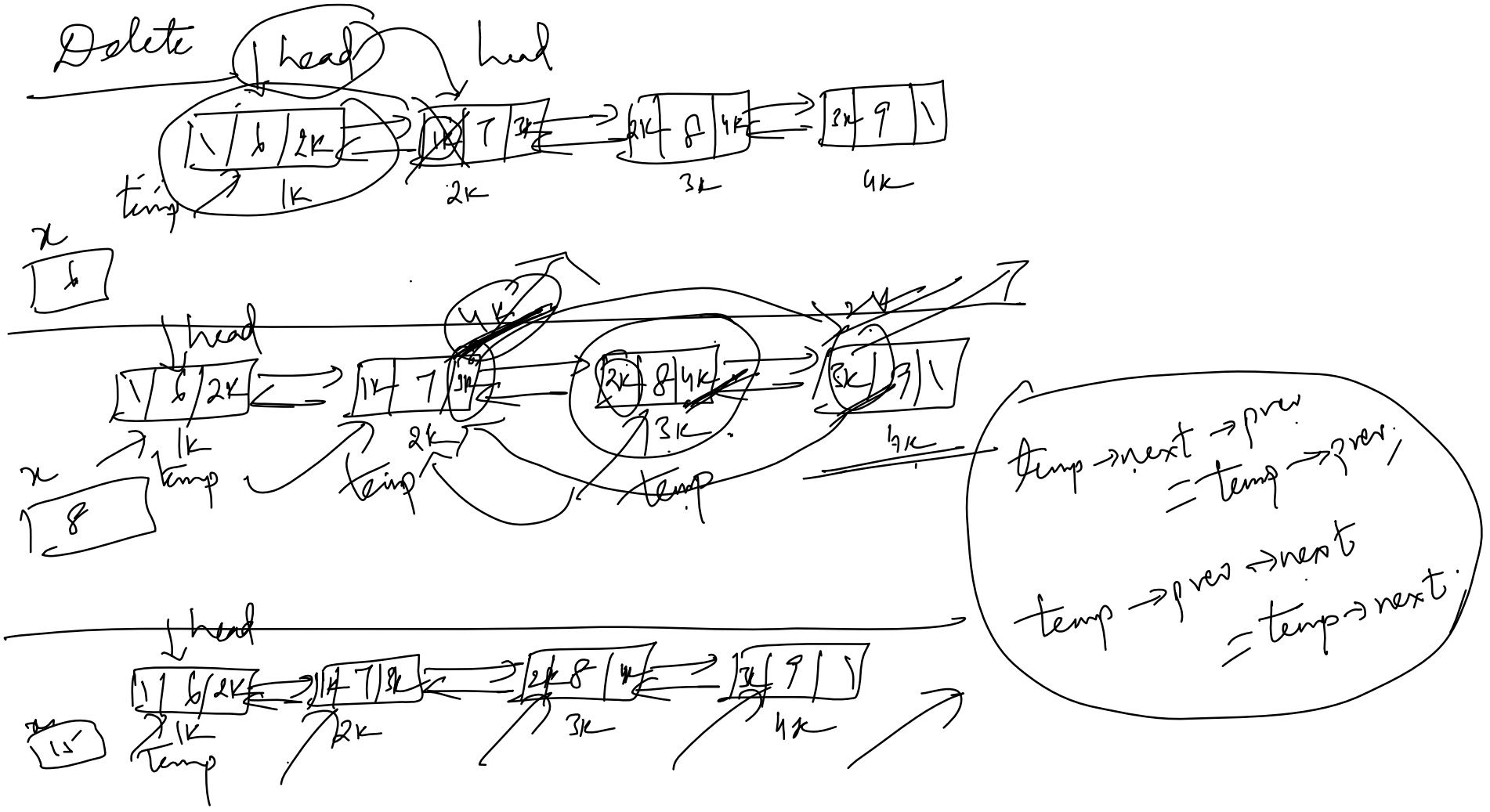
else move to specified position and adjust the nodes.




```
if (p<=1)
{
  addatbeg(n);
  return;
}
create the node
nn->data=k;
```

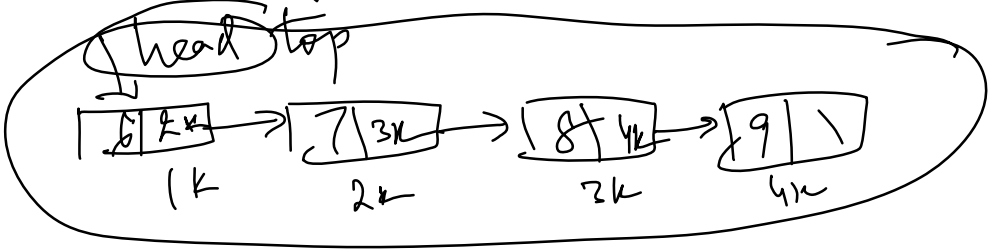
```
for(i=1; i<p-1; i++)
{
  if(temp->next==NULL)
    break;
  temp=temp->next;
}
nn->next=temp->next;
nn->prev=temp
```

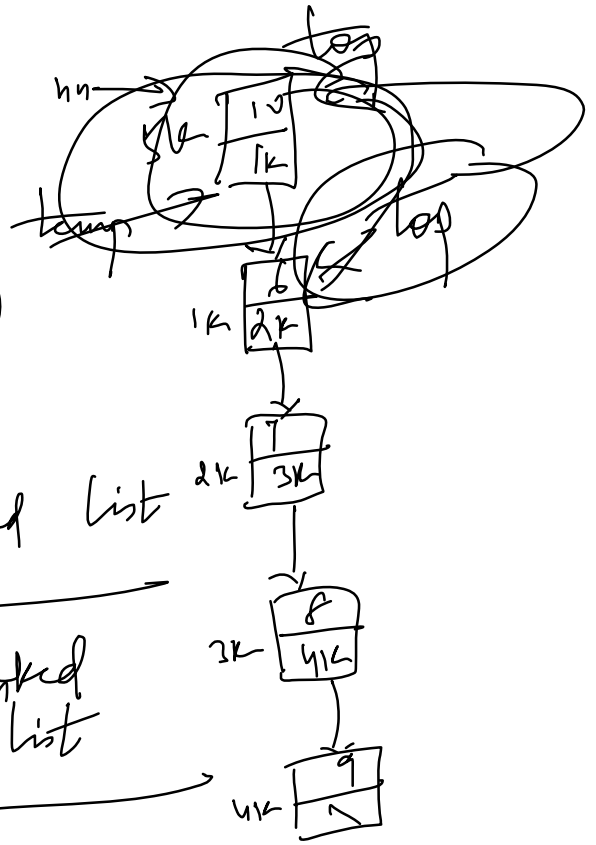
```
if (temp->next!=NULL)
temp->next->prev=nn;
temp->next=nn;
```


Delete
head
head
temp
1K
2K
3K
4K
x
8
head
temp
temp
temp
temp->next->prev
=temp->prev;
temp->prev->next
=temp->next;
head
temp

```
while (temp != NULL)
{
    if (temp->data == x)
    {
        if (temp == head)
        {
            head = temp->next;
            head->prev = NULL;
        }
        else
        {
            temp->next->prev = temp->prev;
            temp->prev->next = temp->next;
        }
        printf("Node deleted");
        free(temp);
        return;
    }
    temp = temp->next;
}
printf("Element not found");
```

Stack using a linked list





push → add at beg of single linked list

pop → removing head node of the top linked list

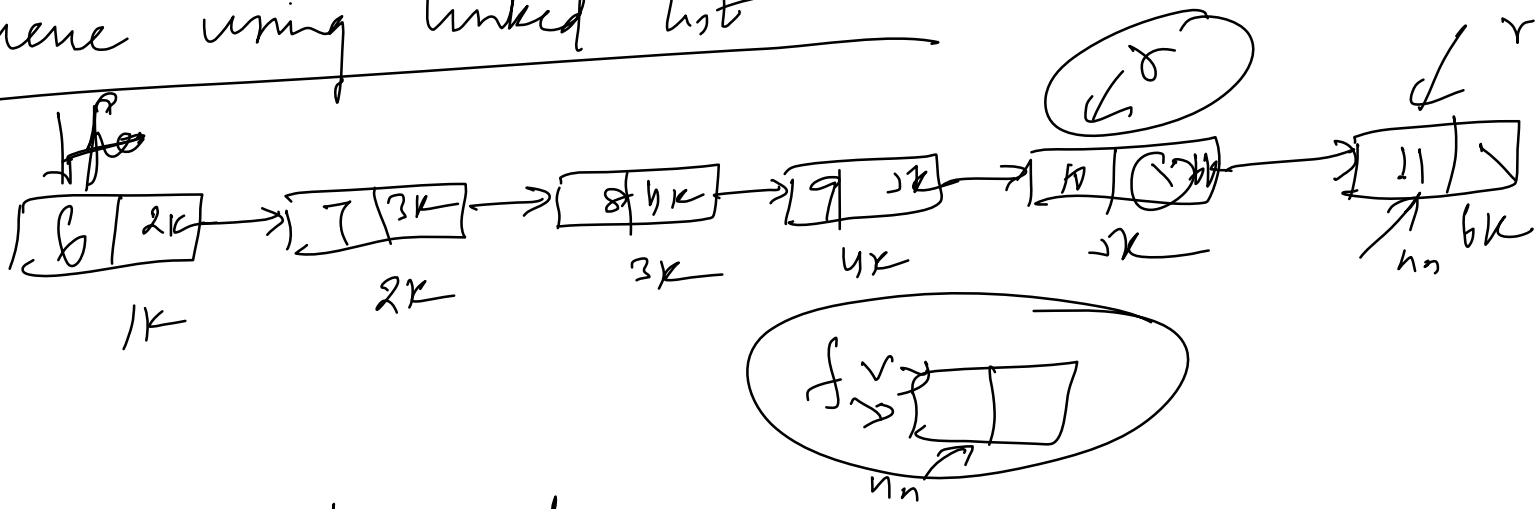
display → same as single linked list

```
struct node
{
  int data
  struct node *next;
} *top = NULL;
```

```
void push(int x)
{
  struct node *nn;
nn = (struct node *) malloc (sizeof(struct node));
  if(nn==NULL)
  {
    printf("stack overflow");
      return;
  }
  nn->data = x;
  nn->next = top;
} top = nn;
```

```
int pop()
{
    int x;
    struct node *temp = top;
    if (top == NULL)
    {
        printf("Stack underflow");
        return;
    }
    x = top -> data;
    top = temp -> next;
    free(temp);
    printf("value deleted");
    return x;
}
```

Queue using linked list

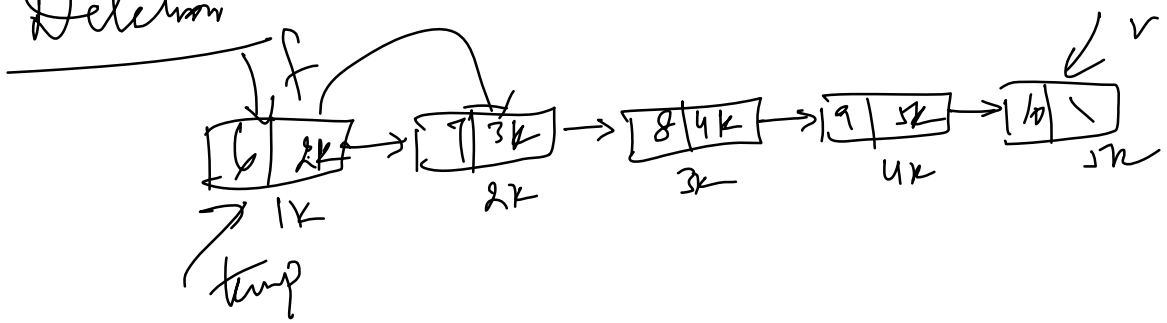


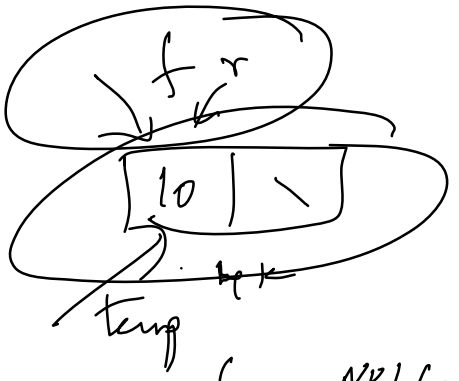
Insert

create a node
if it is for the first time make f & r point to it.
if queue is already present current rear's next must point to newly created node
r should point to newly created node.

```
nn= (struct node*) malloc (sizeof (struct node));
if (nn== NULL)
{ printf("Queue overflow");
  return;
}
nn->data = x;
nn->next = NULL;
if (f== NULL && r==NULL)
  { f= r=nn;
    return;
  }
r->next = nn;
r=nn;
```

Deletion





f = r = NULL

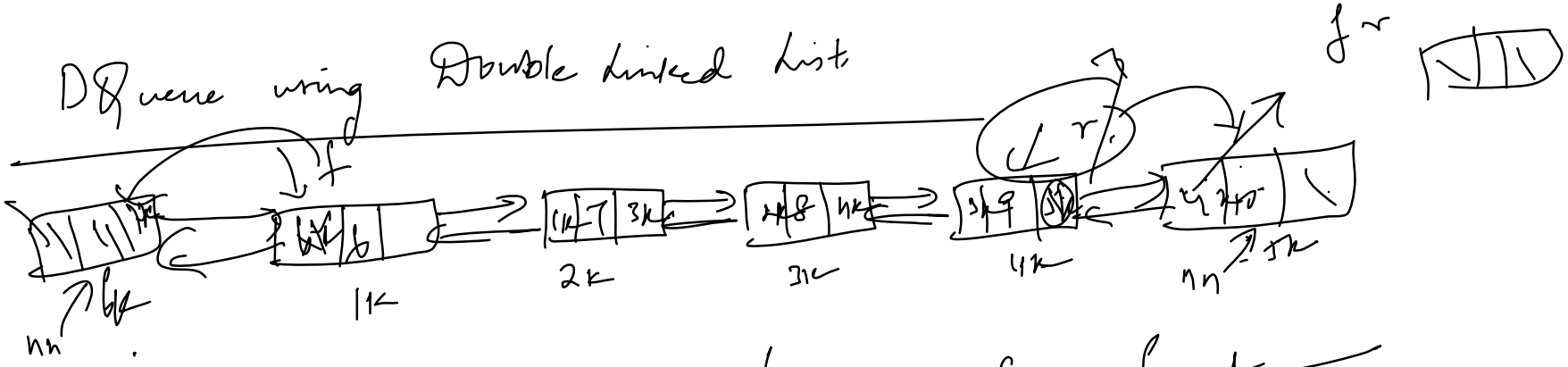
```
if (f == NULL)
{
    printf("Queue underflow");
    return;
}
x = f -> data;
```

```
if (f == r)
    f = r = NULL;
else
    f = temp -> next;
return x;
```

Display

same as single linked list display.

DQueue using Double linked List



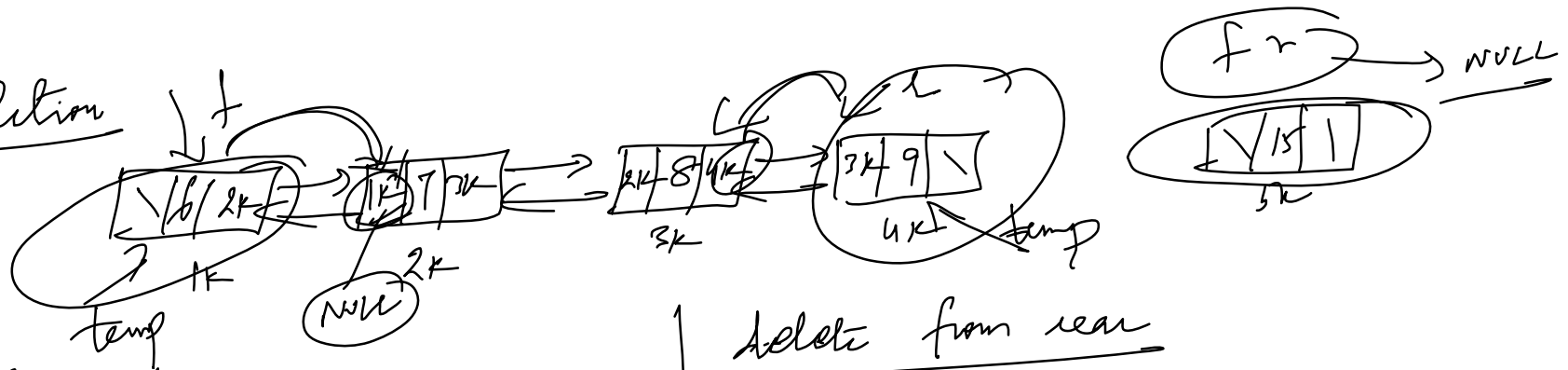
Insert from rear

```
nn=(struct node*) malloc(  )
if (nn== NULL)
{
nn->data=x;
nn->next=NULL;
if (f==NULL&&r==NULL)
f=r=nn;
return;
}
r->next=nn;
nn->prev=r;
r=nn;
```

Insert from front

```
nn=(struct node*) malloc(  )
if (nn==NULL)
{
nn->data=x;
nn->prev=NULL
if (f==NULL&&r==NULL)
nn->next=NULL;
f=r=nn;
return;
}
nn->next=f;
f->prev=nn;
f=nn;
```

Deletion



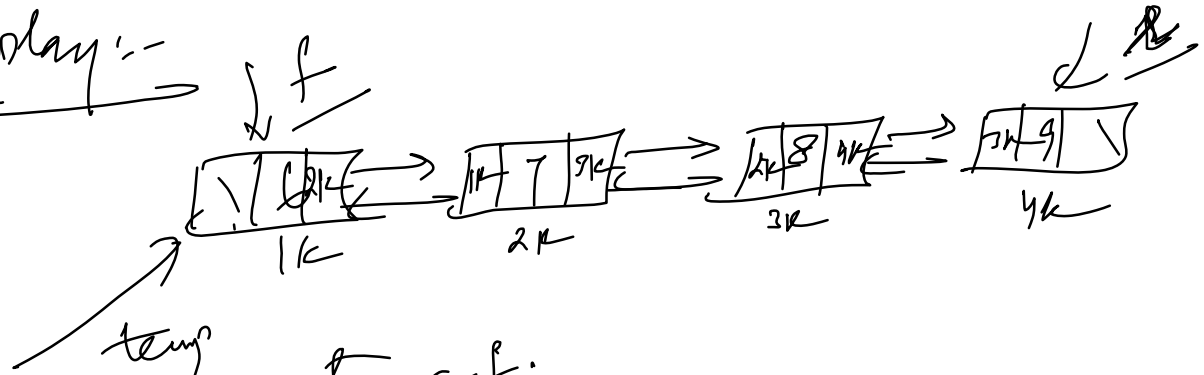
delete from front

```
if (f==NULL)
{
    underflow
    return -1;
}
x = f->data
if (f==r)
f=r=NULL
else
f = temp->next;
f->prev = NULL;
free(temp);
return x;
```

delete from rear

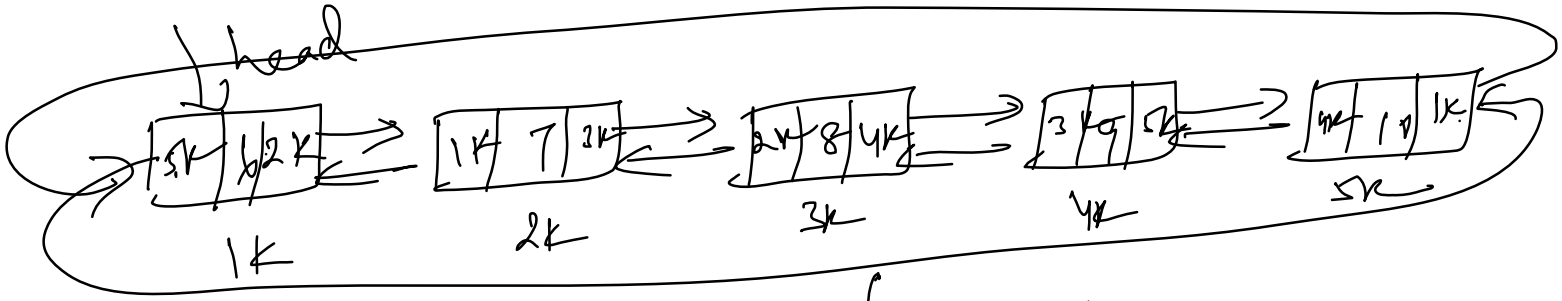
```
if (r==NULL)
{
    underflow
    return -1;
}
x = r->data
if (f==r)
    f=r=NULL;
else
r = temp->prev;
r->next = NULL;
free(temp);
return x;
```

Display :-




```
temp = f;
while (temp != NULL)
{
    printf ("%d ", temp -> data);
    temp = temp -> next;
}
```

Circularly Double linked list :- A double linked list where first node's previous stores the address of last node and the last node's next stores the address of first node.



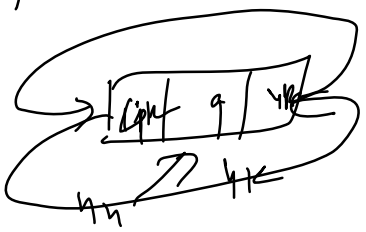

```
struct node
{ int data;
  struct node *next;
  struct node *prev;
}*head=NULL;
```

Create
add at beg
add at any
delete
display.

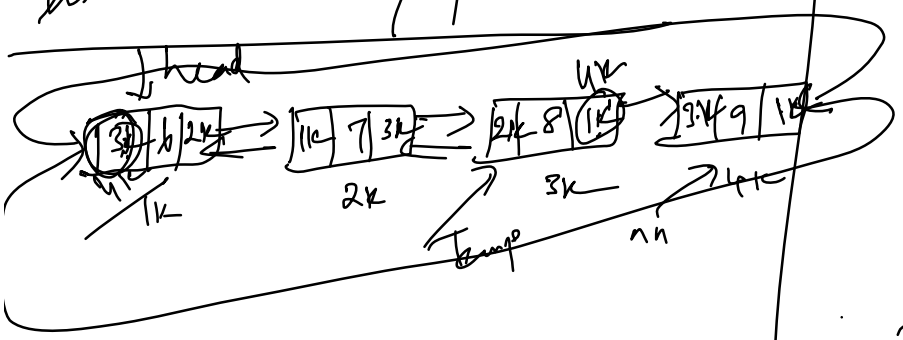
Create:-

list is not present

head = NULL

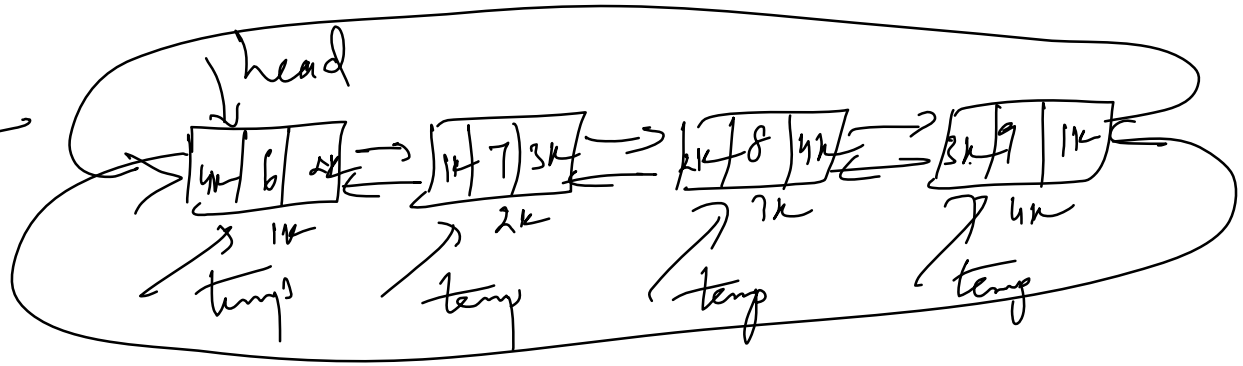


list is already present



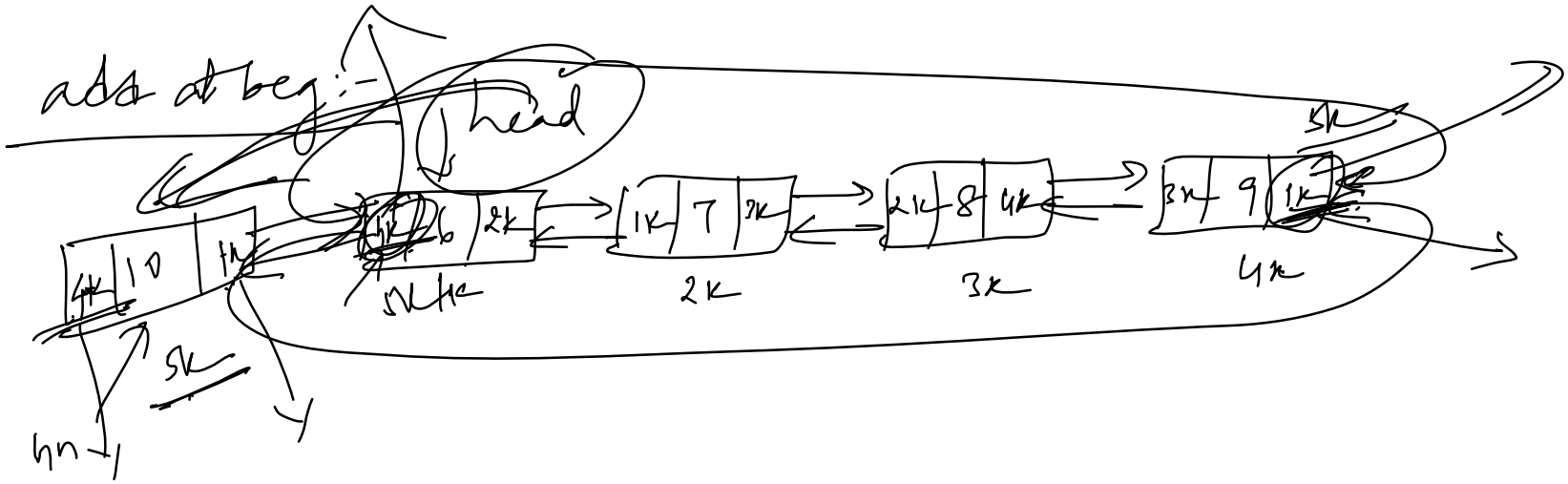

```
struct node *nn, *temp;
nn=(struct node*) malloc( sizeof(struct node));
nn->data=x;
if (head==NULL)
{
    head = nn;
    head->next=nn;
    head->prev=nn;
    return;
}
temp=head->prev;
temp->next=nn;
nn->prev=temp;
nn->next=head;
head->prev=nn;
}
```

Display :-




```
do
{
printf("%d", temp->data);
  temp = temp->next;
} while(temp != head);
```

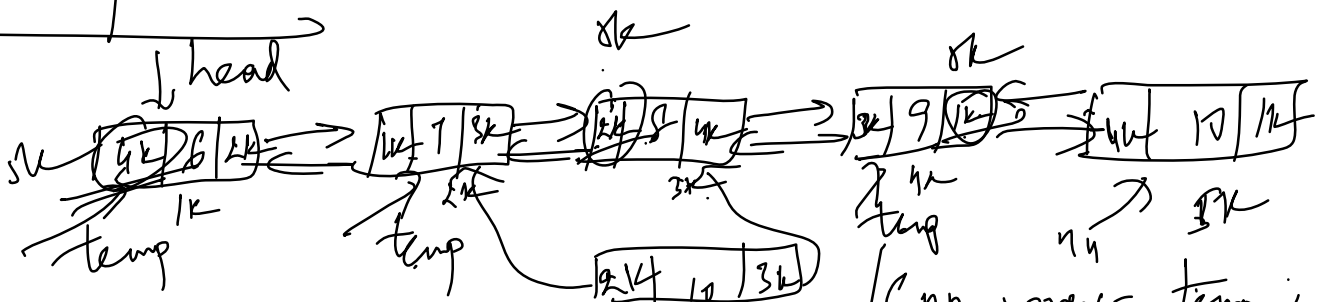
```
if (head == NULL)
{
printf("List is empty\n");
return;
}
```




```
nn=(  )
nn->data=x;
if(head==NULL)
{ head=nn;
  head->prev=nn;
  head->next=nn;
  return;
}
```

```
nn->prev=head->prev;
head->prev->next=nn;
nn->next=head;
head->prev=nn;
head=nn;
```

add at a position

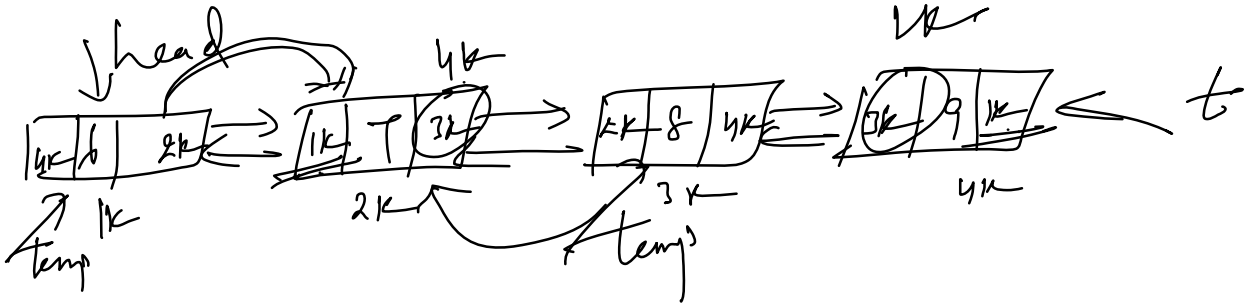



```
if (p<=1)
{
addatbeg(x);
return;
}
```

```
for(i=1;i<p-1;i++)
{
if(temp->next==head)
break;
temp=temp->next;
}
nn=
nn->data=x;
```

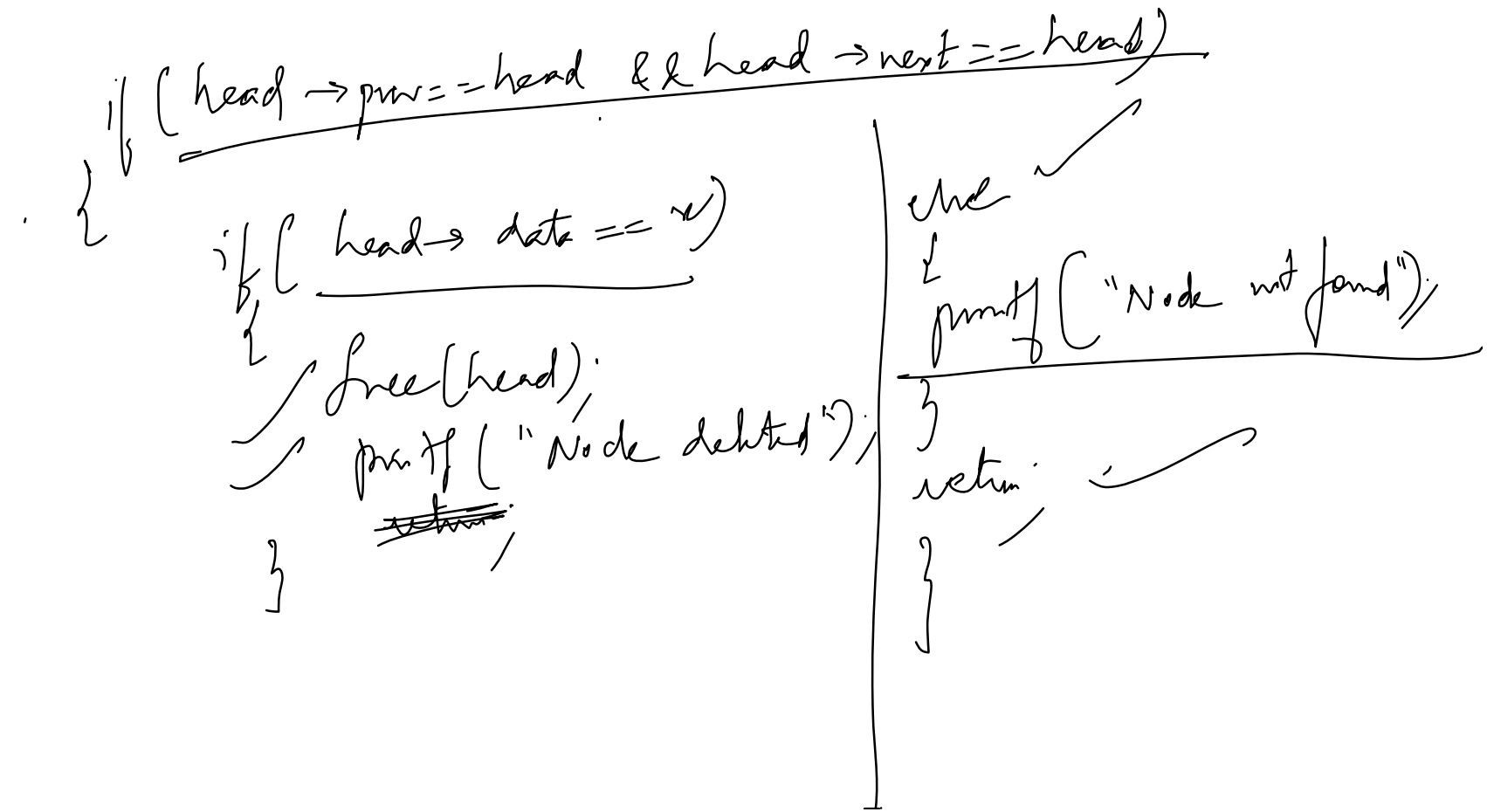
```
nn->prev=temp;
nn->next=temp->next;
temp->next->prev=nn;
temp->next=nn;
```

deletion



x=6

```
do
{
if(temp->data==x)
{
if(temp==head)
{
t=head->prev;
head=temp->next;
head->prev=t;
t->next=head;
return;
}
else
{
temp->prev->next=temp->next;
temp->next->prev=temp->prev;
}
free(temp);
return;
}
temp=temp->next;
}while(temp!=head);
printf("Element not found");
```


if (head → prev == head && head → next == head)
if (head → data == x)
free(head);
printf ("Node deleted");
return;
else
printf ("Node not found");
return;

add 2 polynomials using linked list

$2x^2 + 3x + 4$

$4x^3 + 3x + 7$

$4x^3 + 2x^2 + 8x + 11$

```
node
{
    int coeff, exp;
    struct node *next;
};
```

Searching & Sorting

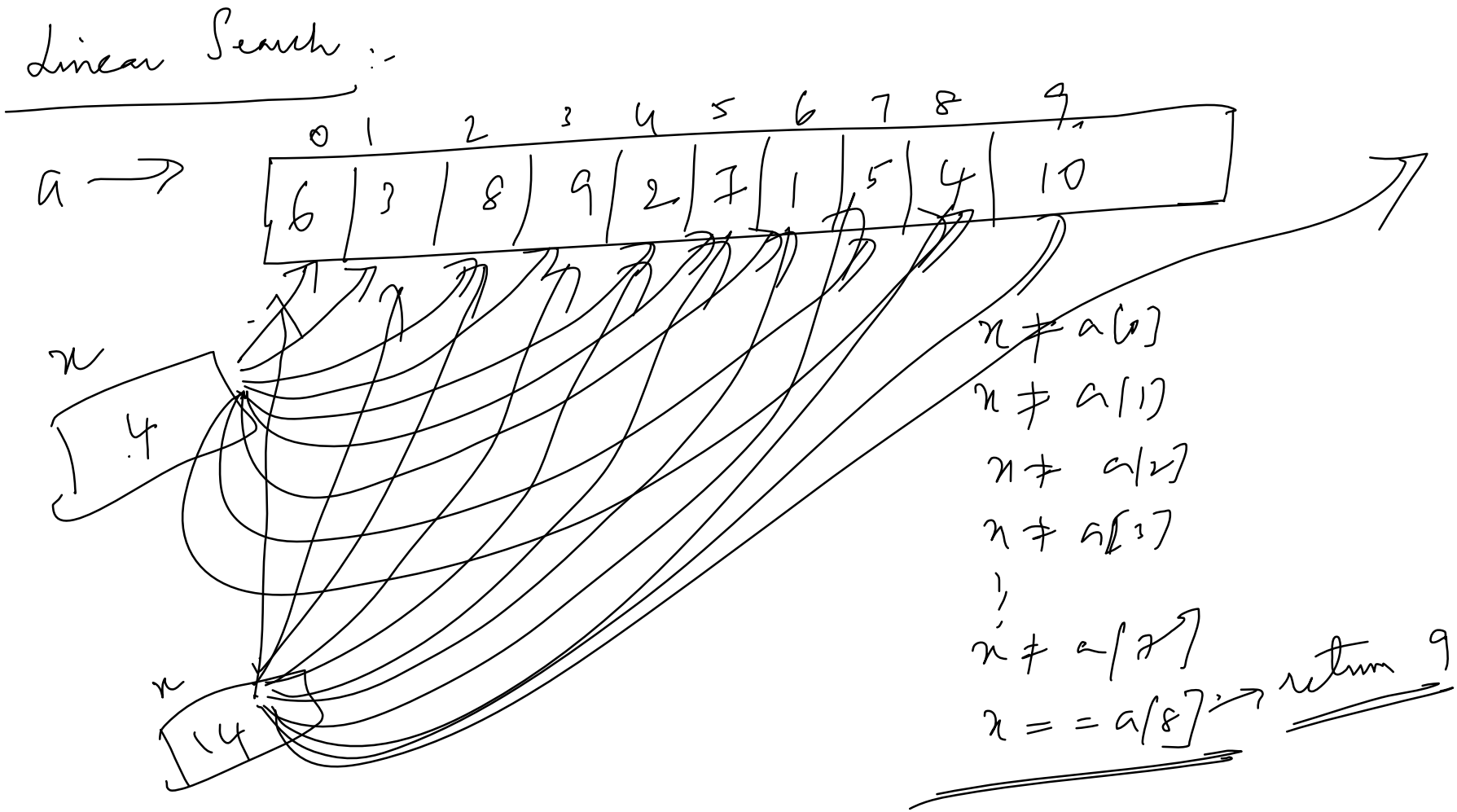
Searching :- It is the process of checking whether an element is present in an list of elements or not array

1) Linear Search
2) Binary Search

Sorting :- Arranging the elements of an array in ascending order is known as sorting

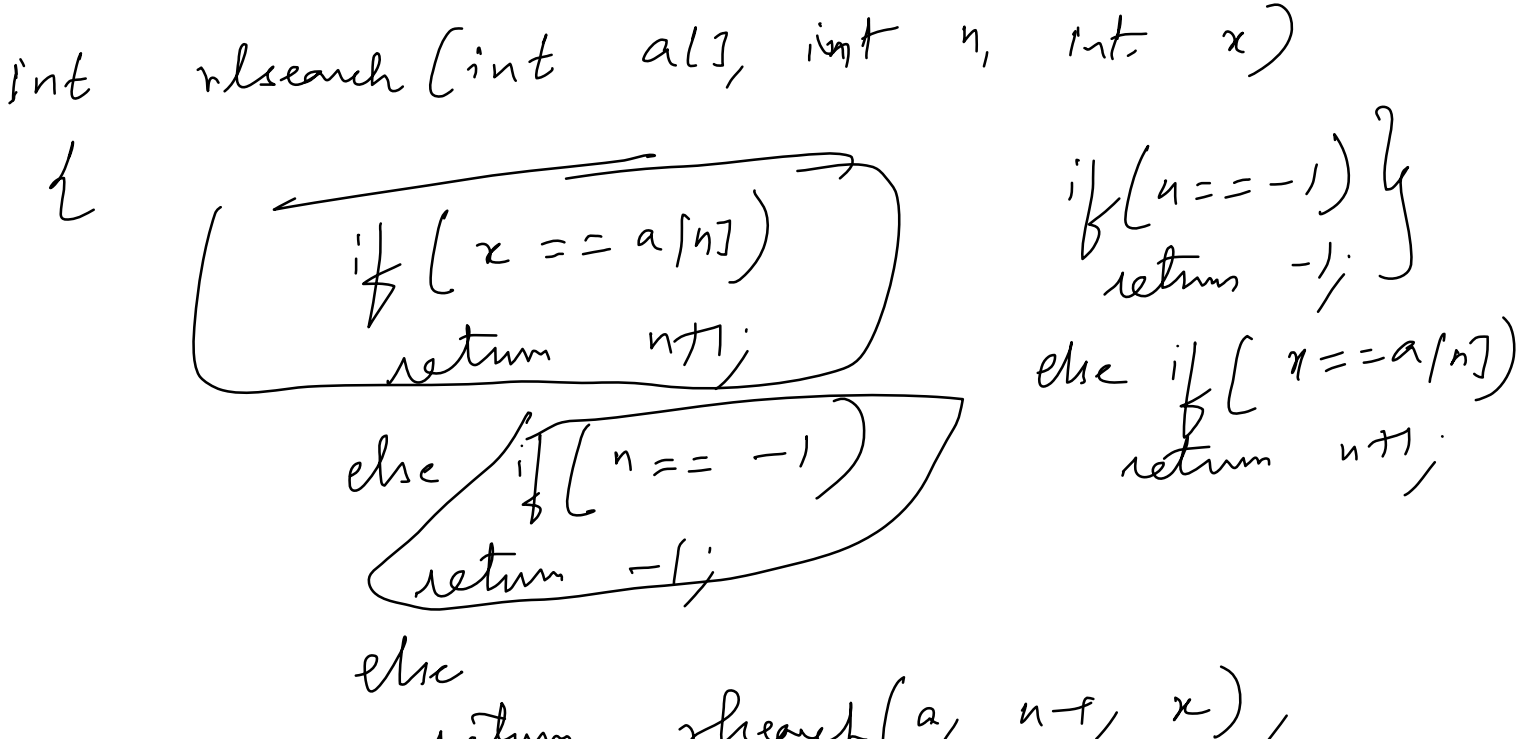
1) Selection sort
2) Bubble sort
3) Insertion Sort
4) Quick sort
5) Merge sort
6) Counting sort
7) Heap Sort
8) radix sort / bucket sort

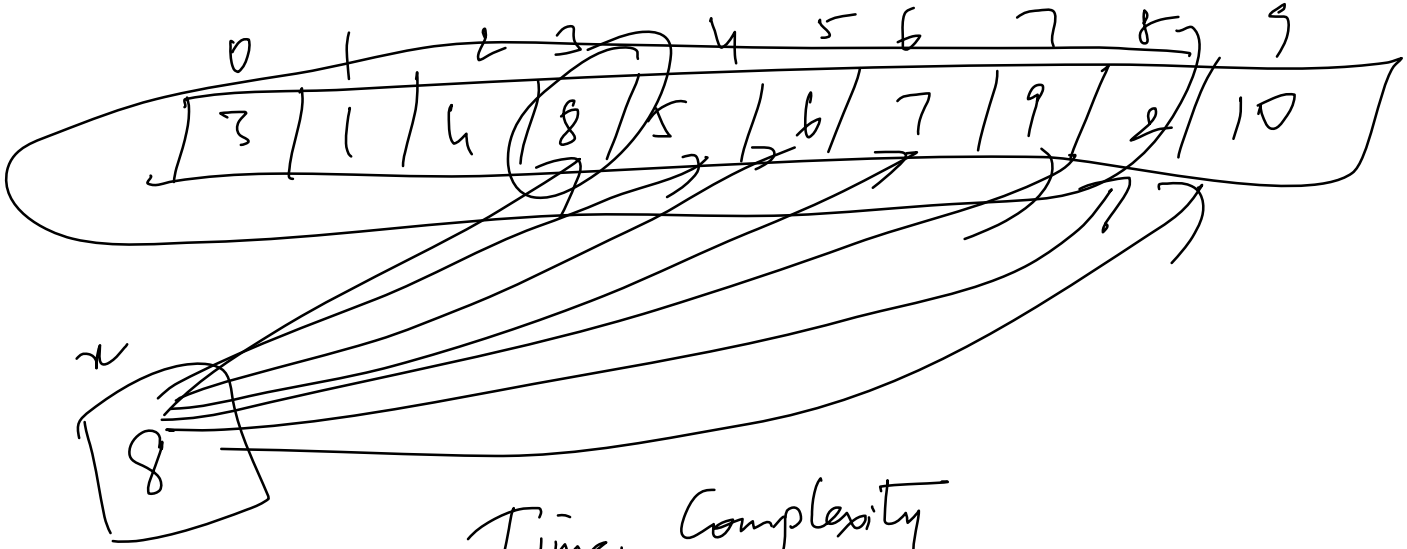
Linear Search :-



```
int lsearch(int a[], int n, int x)
{
    int i;

    for(i=0; i<n; i++)
    {
        if(x==a[i])
            return i;
    }
    return -1;  // element not found
}
```


int rlsearch(int a[], int n, int x)
{
if(x==a[n])
return n+1;
else if(n==-1)
return -1;
if(n==-1) }
return -1;
else if(x==a[n])
return n+1;
else
return rlsearch(a, n-1, x);



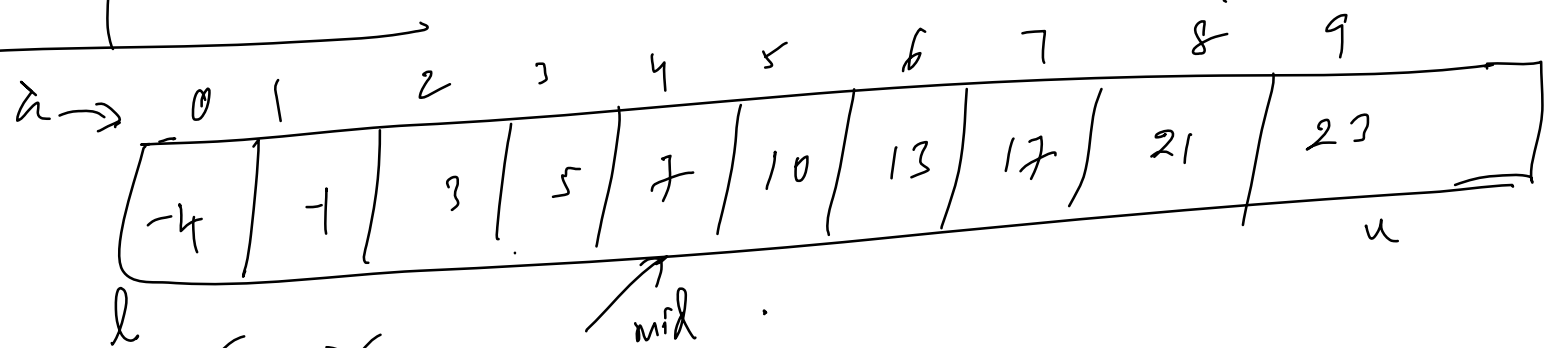
Time Complexity

Best Case → O(1) if element is exactly in 0th position

Avg Case → O(n)

Worst Case → O(n)

Binary Search :- list must be already in ascending order

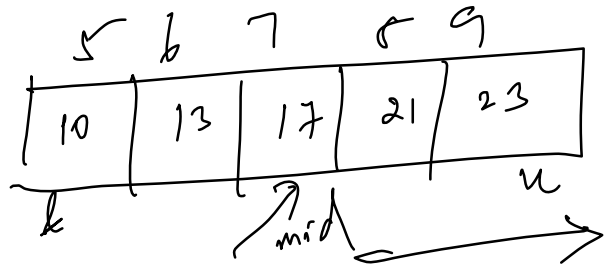


x
13

→ mid = (l+u)/2 = (0+9)/2 = 4

→ x > a[mid]

l = mid+1 → l = 5



x

13

$mid = (5+9)/2 = 7$

$x < a[mid]$

$u = mid - 1 \Rightarrow u = 6$



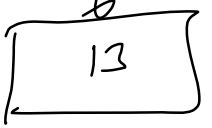
$mid = (5+6)/2 = 5$

x

13

$x > a[mid]$

$\Rightarrow l = mid + 1 \Rightarrow l = 6$



x

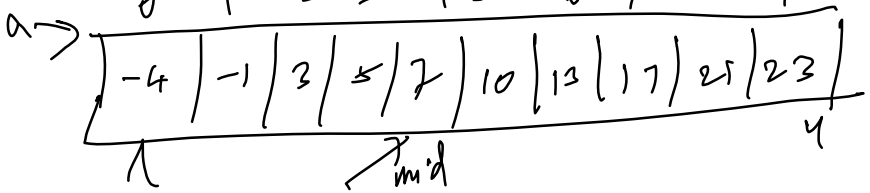
13

$mid = (l+u)/2 = (6+7)/2 = 6$

$x == a[mid]$

$\Rightarrow$ return mid;

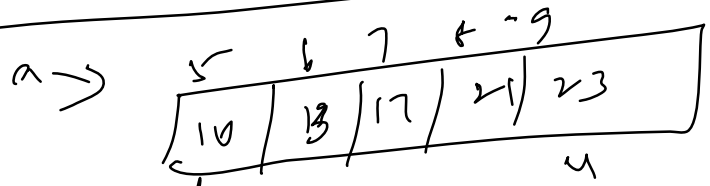
Eg:-



x = 14

mid = (0+9)/2 = 4

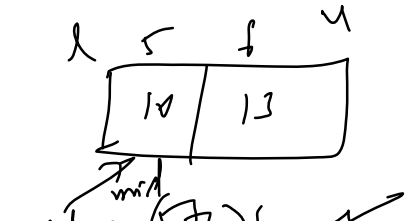
x > a[mid] ⇒ l = mid+1 ⇒ l = 5



x = 14

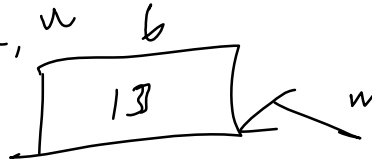
mid = (5+9)/2 = 7

x < a[mid] ⇒ u = mid-1 ⇒ u = 6



mid = (5+6)/2 = 5

x > a[mid] ⇒ l = mid+1 ⇒ l = 6



mid = (6+6)/2 = 6

x > a[mid] ⇒ l = mid+1 ⇒ l = 7

l = 7, u = 6 ⇒ l > u

element is not found

Iterative

```
l = 0, u = n-1

while(l <= u)
{
    mid = (l+u)/2;
    if( x > a[mid])
        l = mid+1;
    else if( x < a[mid])
        u = mid-1;
    else
        return mid+1;
}
    return -1;
```

Recursive

```
if(l > u)
return -1;

else if(l==u)
{
    if(x==a[l])
    return l+1;
    else
    return -1;
}

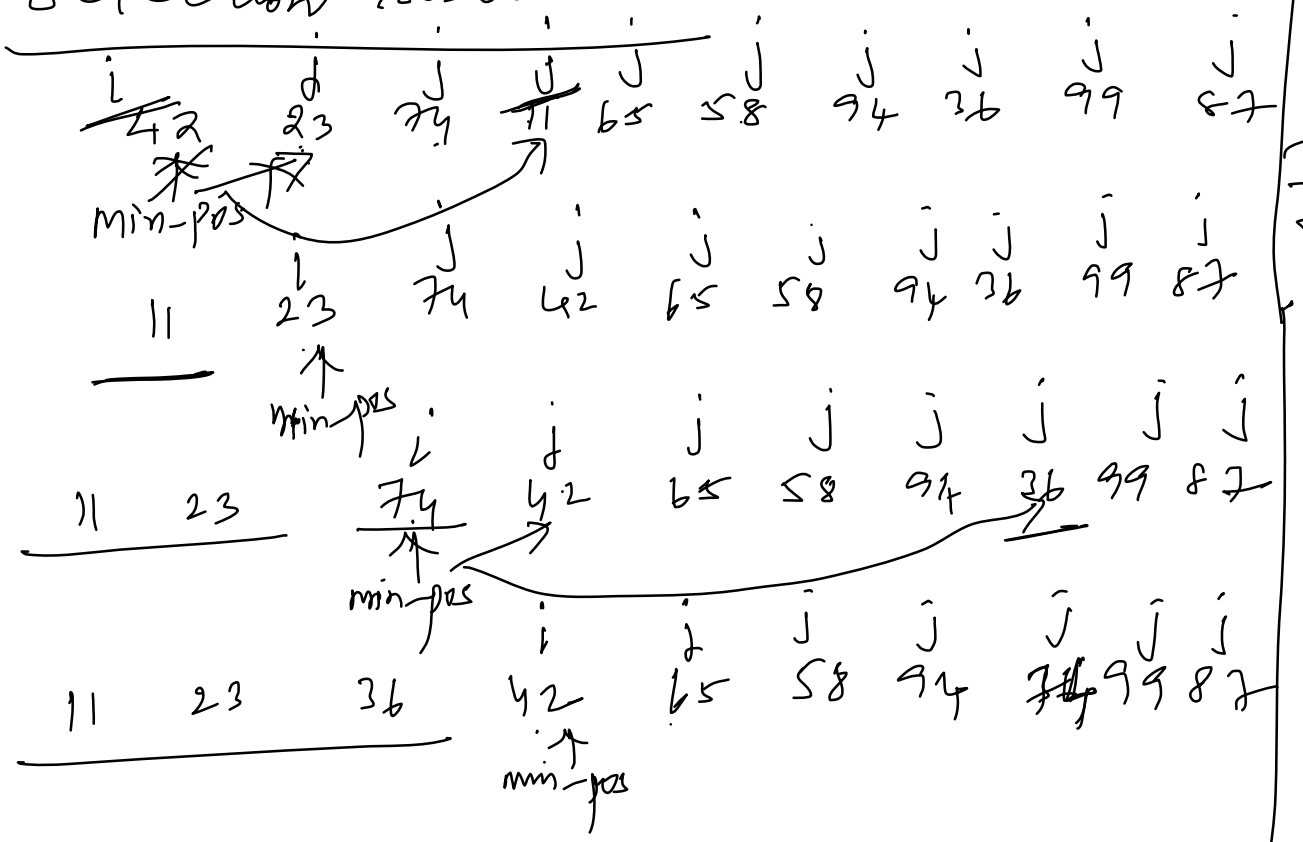
else
{
    mid = (l+u)/2;
    if(x > a[mid])
    return rbsearch(a, mid+1, u, x);
    else if(x < a[mid])
    return rbsearch(a, l, mid+1, u);
    else
    return mid+1;
}
```

Time Complexity

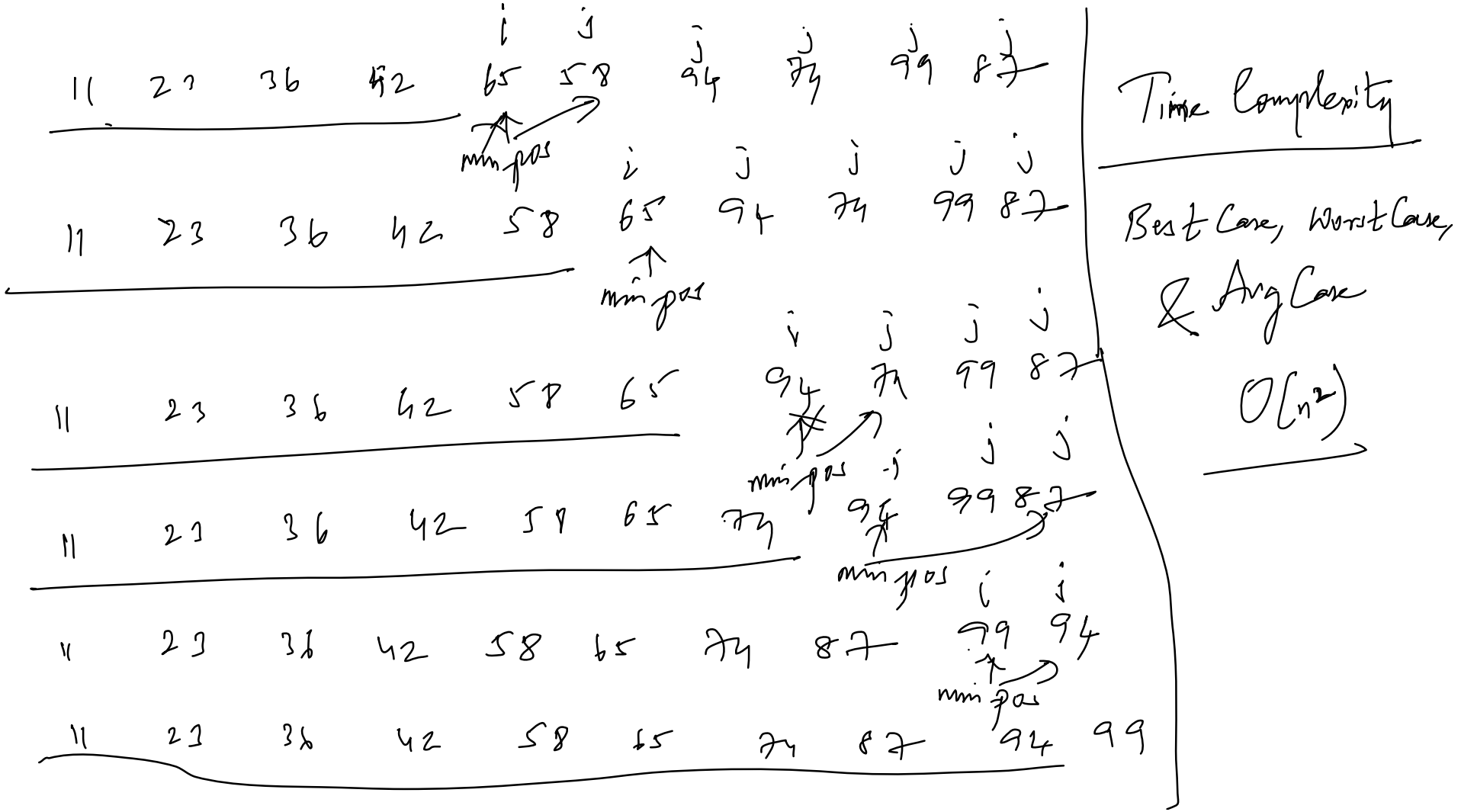
Best Case → $O(1)$ — if element is exactly present in middle position

Avg Case
Worst Case
} $O(\log_2 n)$

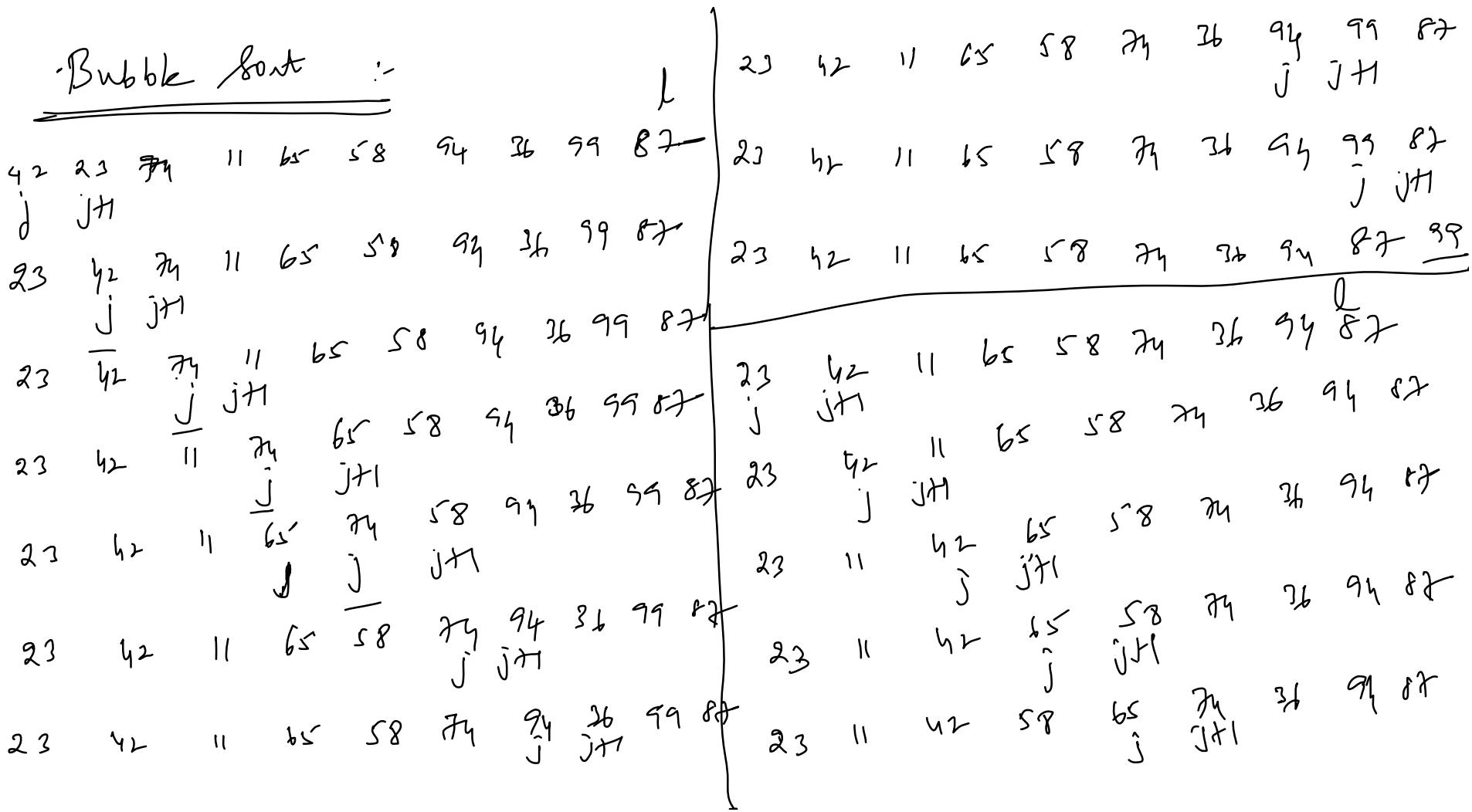
Selection Sort




```
for(i=0; i<n-1; i++)
{
    min_pos = i;
    for(j=i+1; j<n; j++)
    {
        if(a[min_pos] > a[j])
            min_pos = j;
    }
    if(min_pos != i)
    {
        temp = a[min_pos];
        a[min_pos] = a[i];
        a[i] = temp;
    }
}
```


11 23 36 42 65 58 94 74 99 87
i j
min pos
11 23 36 42 58 65 94 74 99 87
i j j j j
min pos
11 23 36 42 58 65 94 74 99 87
i j j j
min pos
11 23 36 42 58 65 74 94 99 87
i j j
min pos
11 23 36 42 58 65 74 87 99 94
i j
min pos
11 23 36 42 58 65 74 87 94 99
Time Complexity
Best Case, Worst Case,
& Avg Case
O(n²)

Bubble Sort :-



23 11 42 58 65 74 36 94 87 99

23 11 42 58 65 74 36 94 87 99

23 11 42 58 65 36 74 87 94 99

23 11 42 58 65 36 74 87 94 99

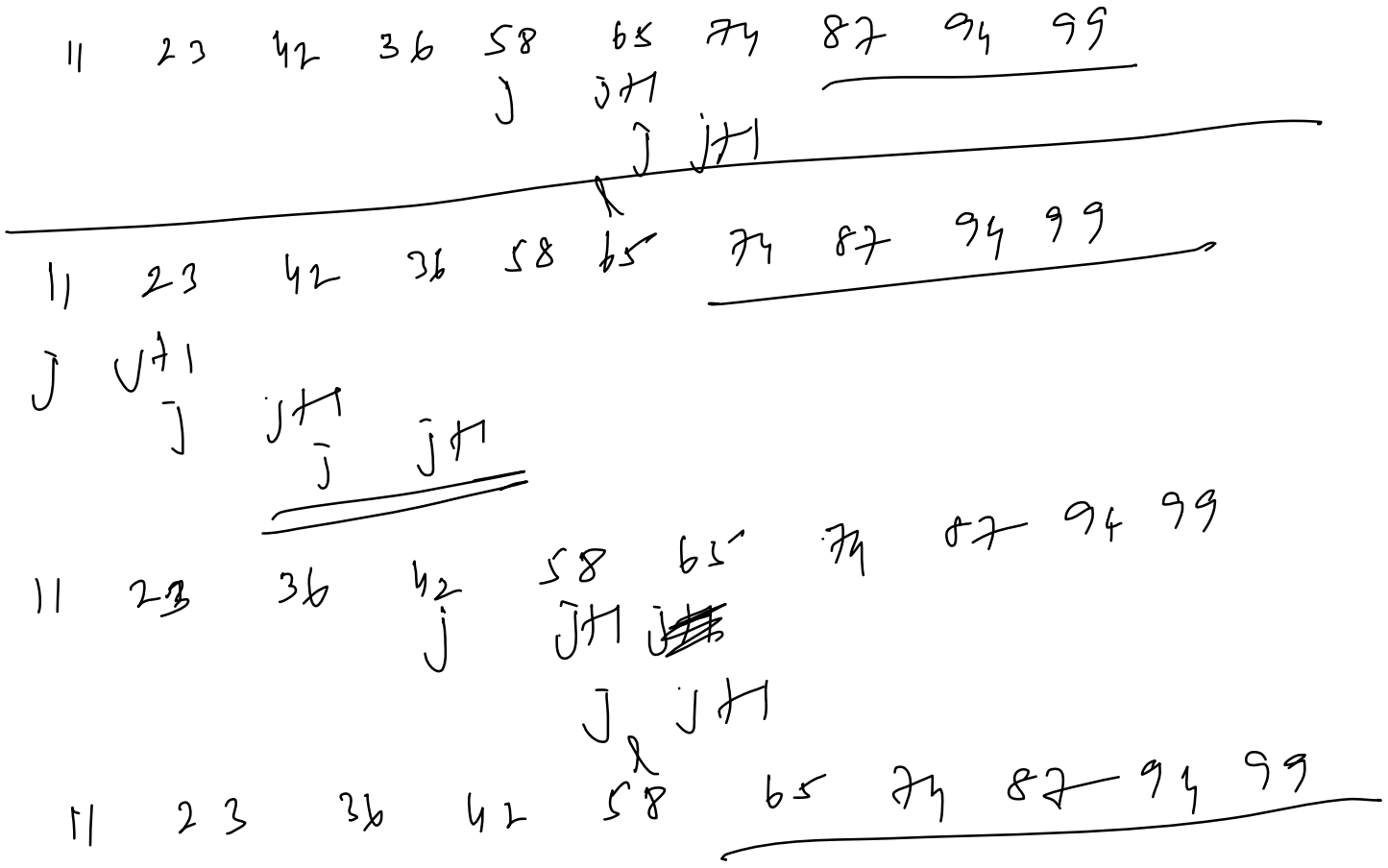
11 23 42 58 65 36 74 87 94 99

11 23 42 58 65 36 74 87 94 99

11 23 42 58 36 65 74 87 94 99

11 23 42 58 36 65 74 87 94 99

11 23 42 58 36 65 74 87 94 99



11 23 38 42 58 65 74 87 91, 93

j j+1
j j+1
j j+1
j j+1

```
for(i=0; i<n; i++)
{
    flag=0;
    for(j=0; j<l; j++)
    {
        if(a[j]>a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
            flag=1;
        }
    }
    if(flag==0)
        return;
    else
        l=l-1;
}
```

Time Complexity

Best Case = $O(n)$ → if list is already sorted

Worst Case & Avg Case = $O(n^2)$

Insertion Sort (Arrangement of books inside a Library)

42 23 74 11 65 58 36

42

j i

42 23

42 42

$x = a[i] = 23$

$j = i - 1$

$x < a[j]$

$a[j+1] = a[j]$

$j--$

23 42

$j == -1$

$\because j < 0$

$a[j+1] = x$

$a[0] = 23$

j i

23 42 74

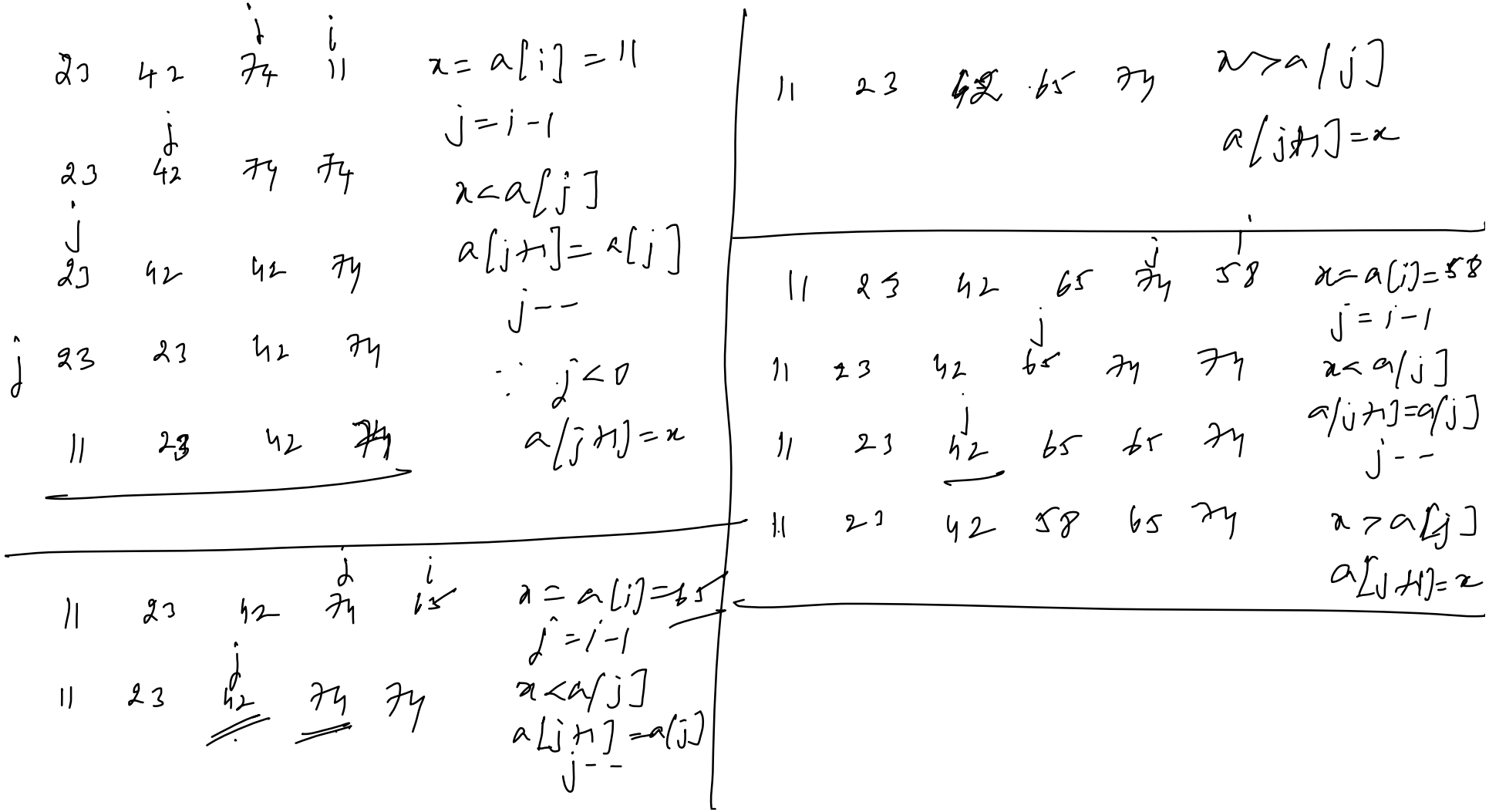
23 42 74

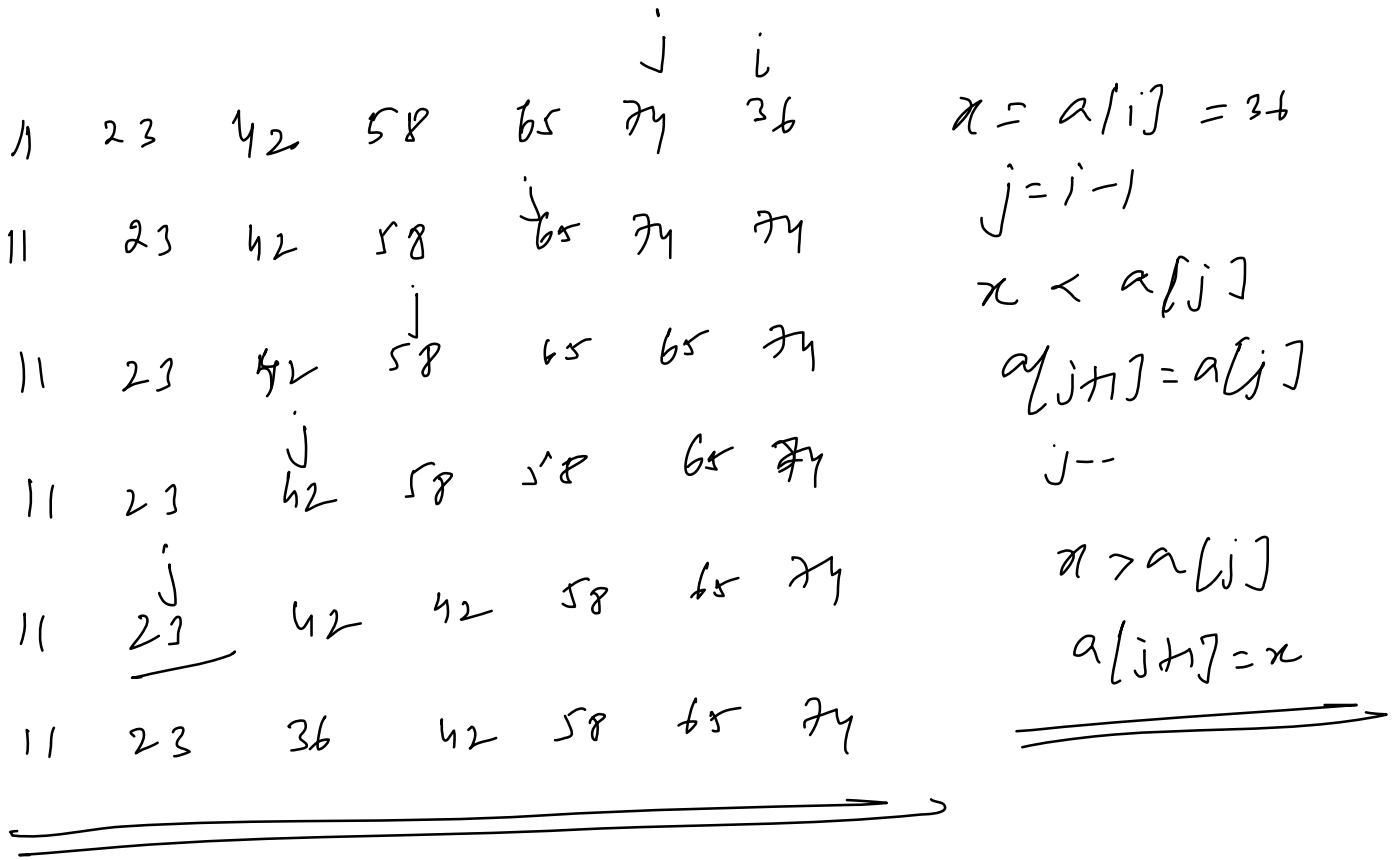
$x = a[i] = 74$

$j = i - 1$

$x > a[j]$

$a[j+1] = x$


23 42 74 11
x = a[i] = 11
j = i - 1
23 42 74 74
x < a[j]
23 42 42 74
a[j+1] = a[j]
j--
23 23 42 74
j < 0
11 23 42 74
a[j+1] = x
11 23 42 65 74 58
x = a[i] = 58
j = i - 1
11 23 42 65 74 74
x < a[j]
11 23 42 65 65 74
a[j+1] = a[j]
j--
11 23 42 58 65 74
x > a[j]
a[j+1] = x
11 23 42 74 65
x = a[i] = 65
j = i - 1
11 23 42 74 74
x < a[j]
a[j+1] = a[j]
j--
11 23 65 74 74
x > a[j]
a[j+1] = x

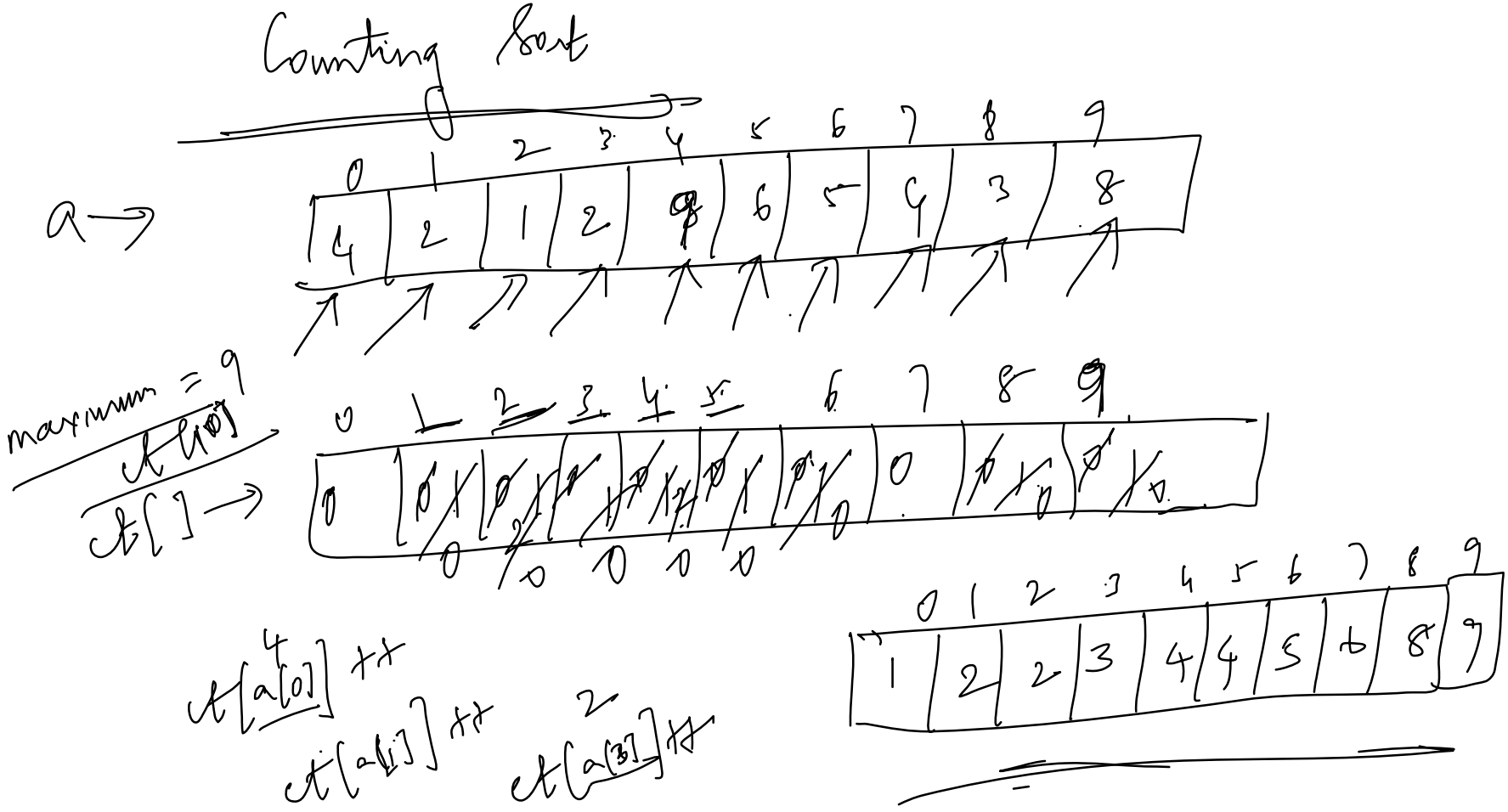

j i
11 23 42 58 65 74 36
x = a[i] = 36
j = i-1
11 23 42 58 65 74 74
x < a[j]
a[j+1] = a[j]
11 23 42 58 65 65 74
j--
11 23 42 58 58 65 74
x > a[j]
11 23 42 42 58 65 74
a[j+1] = x
11 23 36 42 58 65 74

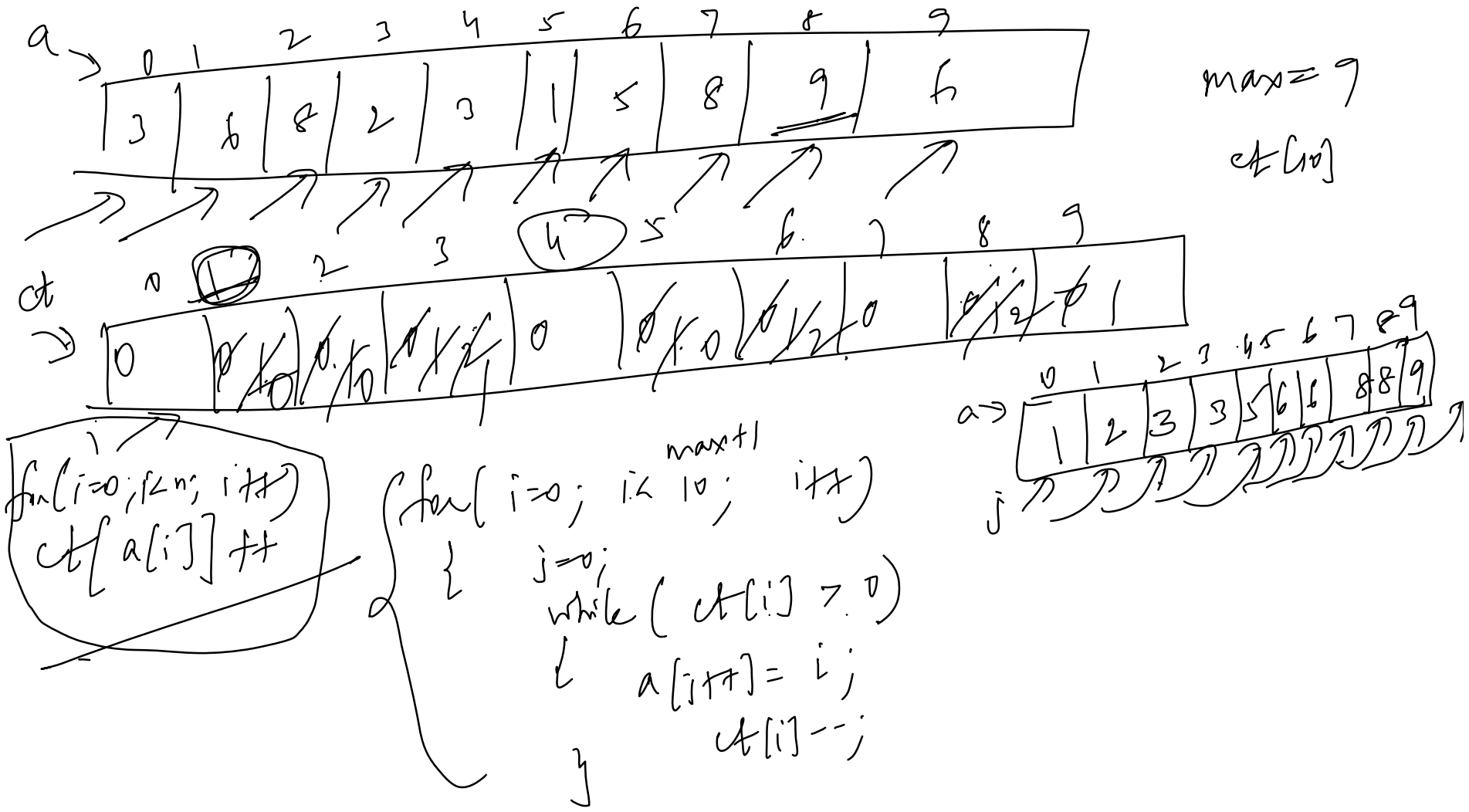
```
for( i=1; i<n; i++)
{
    x=a[i];
    for( j=i-1; x<a[j] && j>=0 ; j--)
    {
        a[j+1] =a[j];
    }
    a[j+1] = x;
}
```

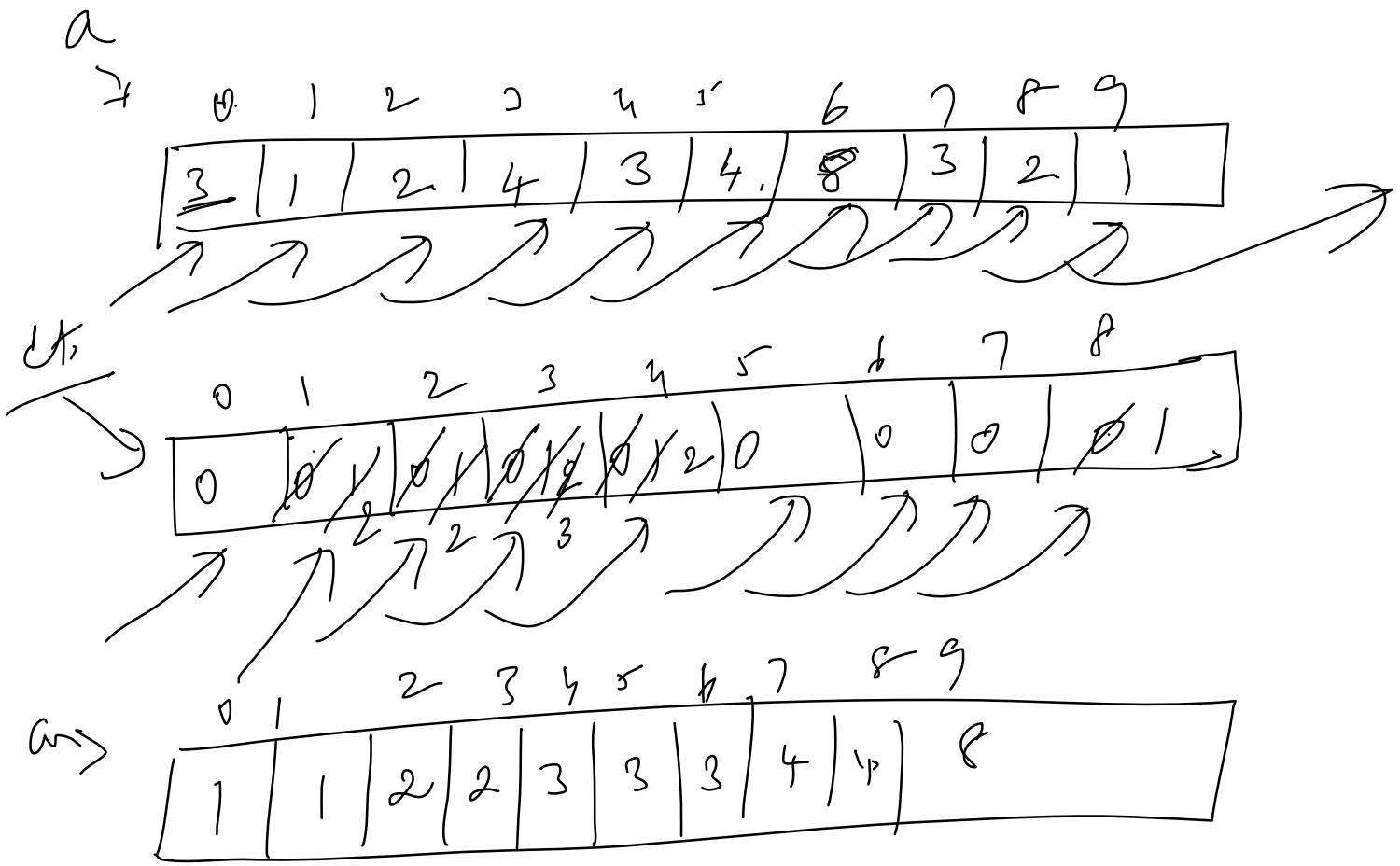
Time Complexity

Best Case
$= O(n)$
if list is already sorted

Worst Case &
Avg Case
$O(n^2)$

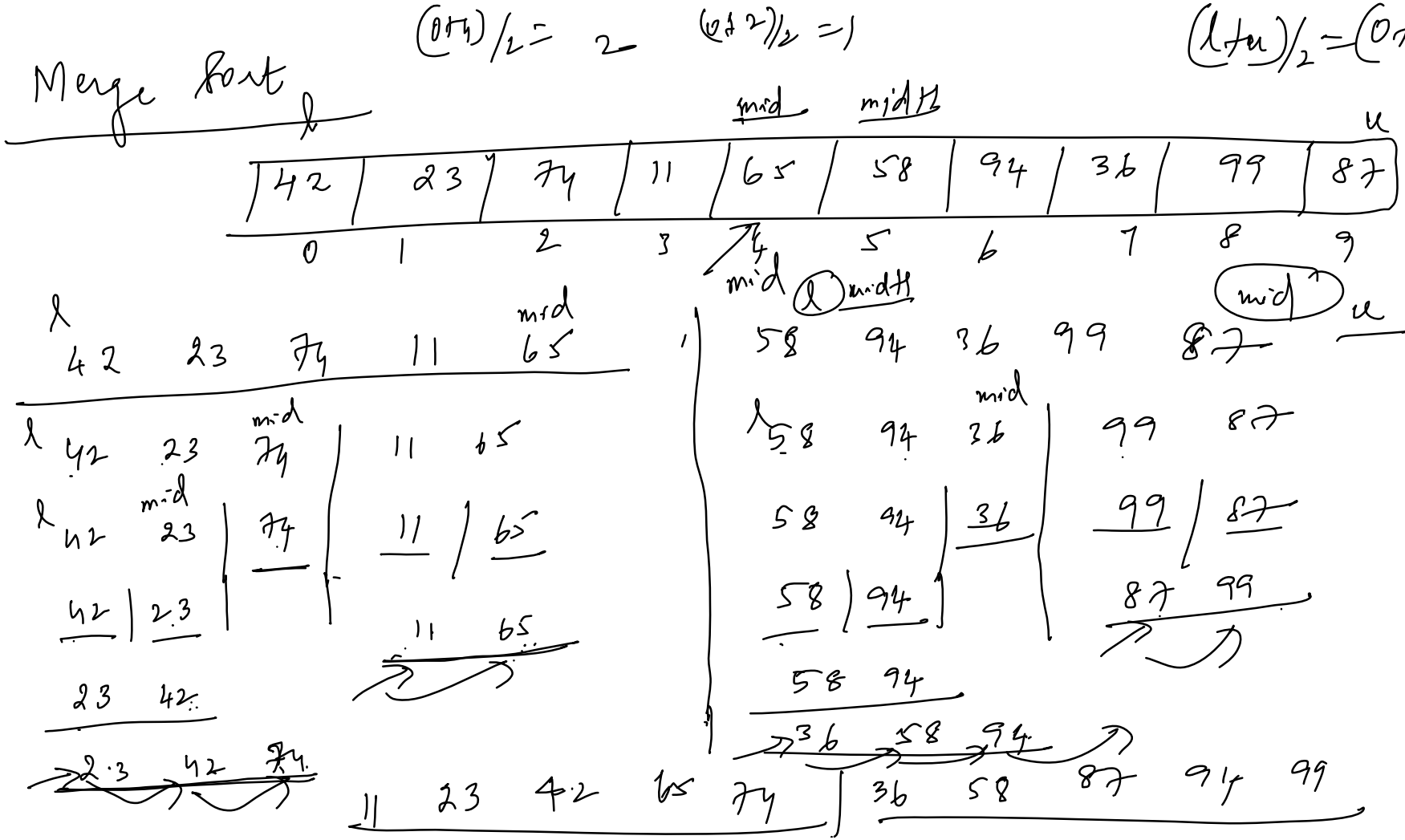

Counting Sort
a →
0 1 2 3 4 5 6 7 8 9
4 2 1 2 9 6 5 4 3 8
maximum = 9
ct[10]
ct[] →
0 1 2 3 4 5 6 7 8 9
0 0 0 0 0 0 0 0 0 0
ct[a[0]]++
ct[a[1]]++
ct[a[3]]++
0 1 2 3 4 5 6 7 8 9
1 2 2 3 4 4 5 6 8 9

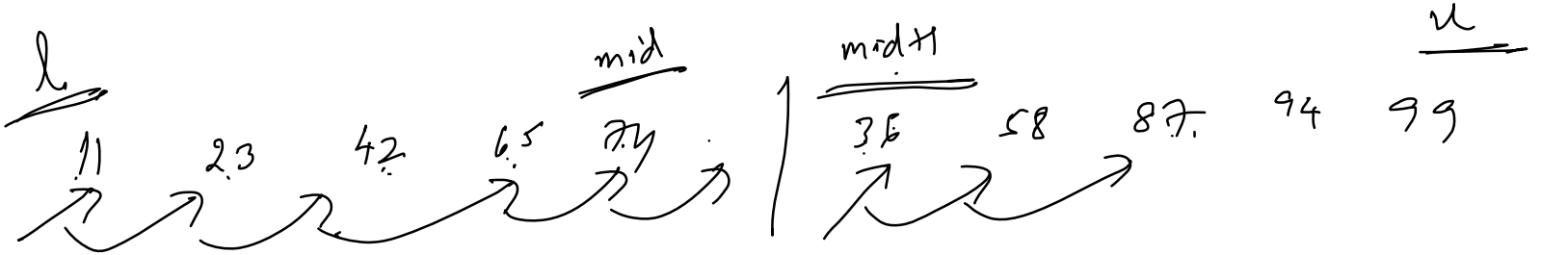

a
0 1 2 3 4 5 6 7 8 9
3 6 8 2 3 1 5 8 9 6
max=9
ct[10]
ct
0 1 2 3 4 5 6 7 8 9
0 0 0 0 0 0 0 0 0 0
for(i=0;i<n;i++)
ct[a[i]]++
for(i=0; i<max+1; i++)
{ j=0;
while(ct[i]>0)
{ a[j++]=i;
ct[i]--;
}
a
0 1 2 3 4 5 6 7 8 9
1 2 3 3 5 6 6 8 8 9
j


a
0 1 2 3 4 5 6 7 8 9
3 1 2 4 3 4 8 3 2 1
dp
0 1 2 3 4 5 6 7 8
0 0 0 0 0 0 2 0 0 0 1
2 2 3
ans
0 1 2 3 4 5 6 7 8 9
1 1 2 2 3 3 3 4 4 8

Merge Sort

$(0+4)/2 = 2$ $(0+2)/2 = 1$ $(l+u)/2 = (0+9)/2 = 4$





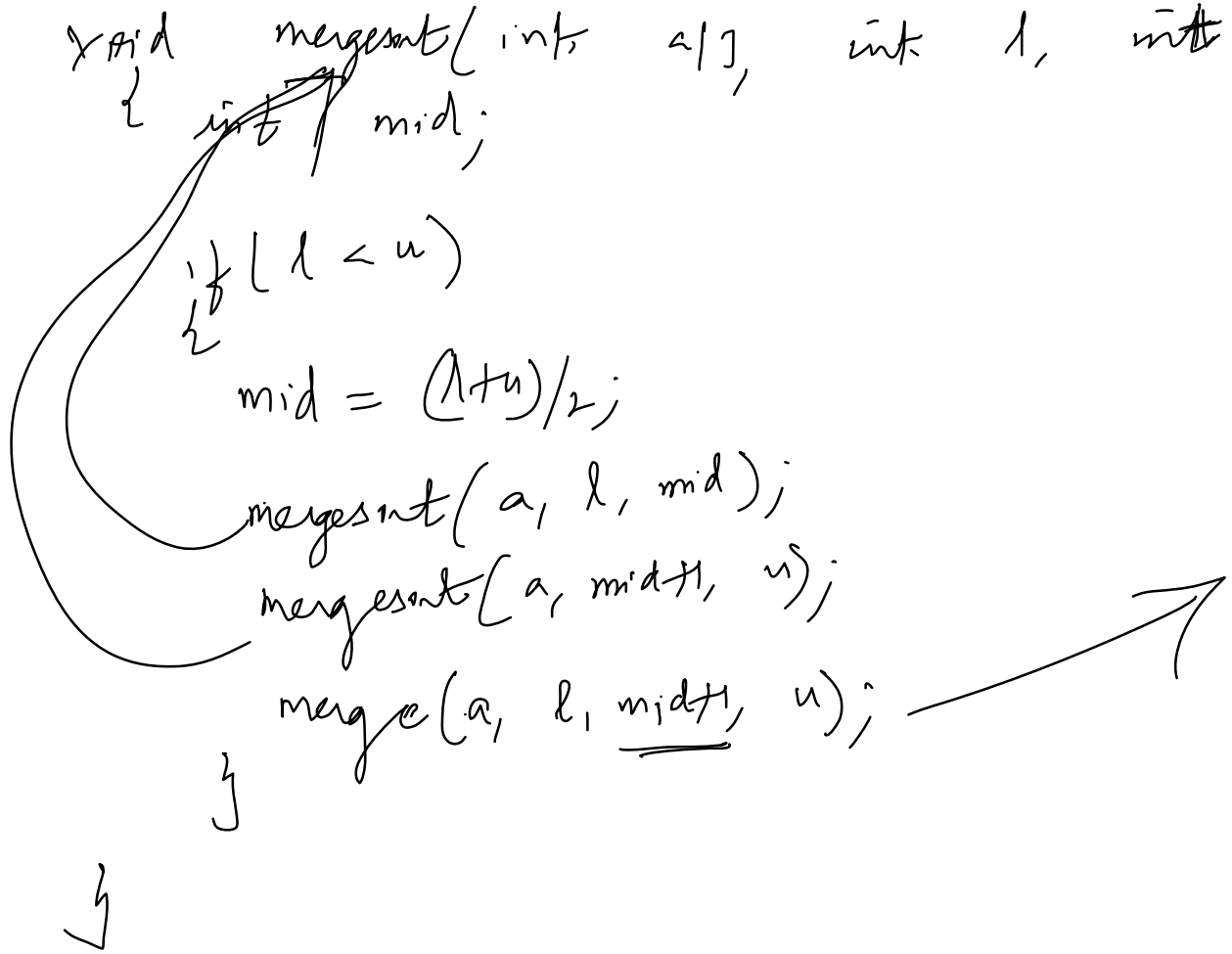
11 23 37 42 58 65 74 87 94 99

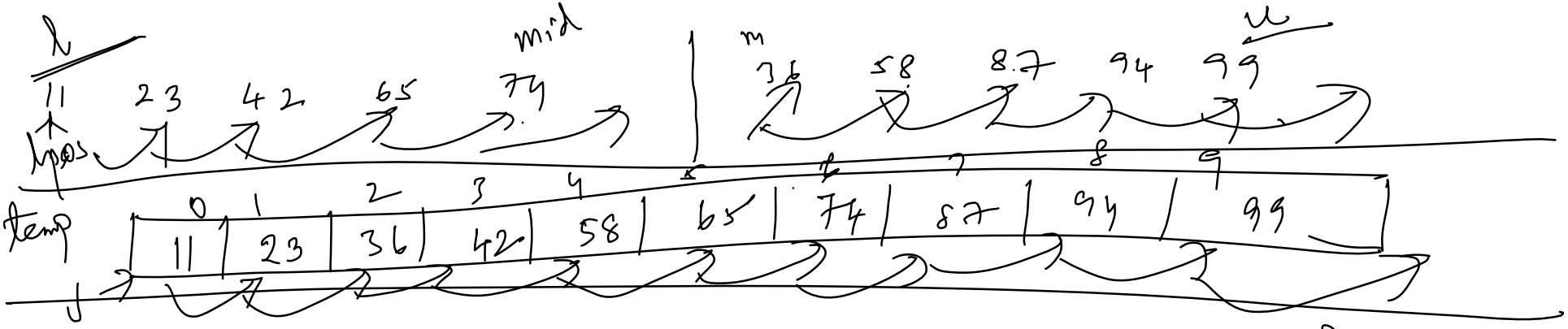
Divide & Conquer

Time Complexity

$O(n \log_2 n)$

- Best Case
- Worst Case
- Avg Case


void mergesort(int a[], int l, int u)
{ int mid;
if(l < u)
{
mid = (l+u)/2;
mergesort(a, l, mid);
mergesort(a, mid+1, u);
merge(a, l, mid+1, u);
}
}




```
void merge ( int a[], int l, int m, int u)
{
    int lpos = l, mid = m-1, j = 0, i; temp [20];
    while ( lpos <= mid && m <= u)
    {
        if (a[lpos] <= a[m])
        {
```

```
temp[j] = a[lpos];
j++;
lpos++;
}
else
{
temp[j] = a[m];
j++;
m++;
}
}
```

```
while(m<=u)
{
temp[j] = a[m];
j++;
m++;
}
```

if right list is left over

```
while(lpos<=mid)
{
temp[j] = a[lpos];
j++;
lpos++;
}
```

if left list is left over

```
for(i=l, j=0; i<=u; i++,j++)
 {
      a[i] = temp[j];
 }
}
```

Quick Sort :- Divide and Conquer policy

1) Assume first pos as lowerbound (l) and last pos as upperbound (u)

2) Assume position i as l+1, position j as u

3) Assume pivot as value of l

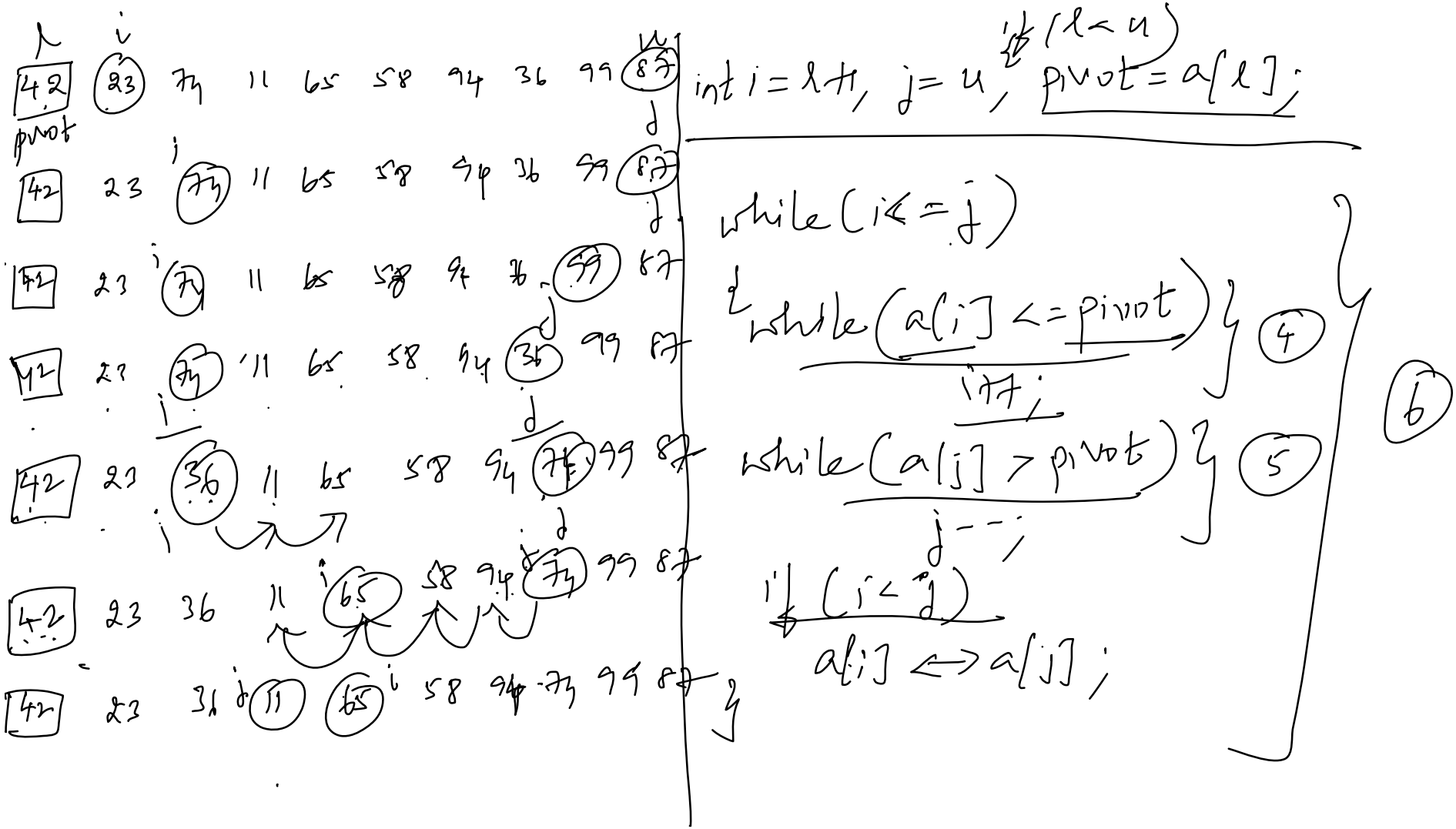
4) If value present in ith pos is <= pivot increment i by 1. Repeat till value in ith pos > pivot

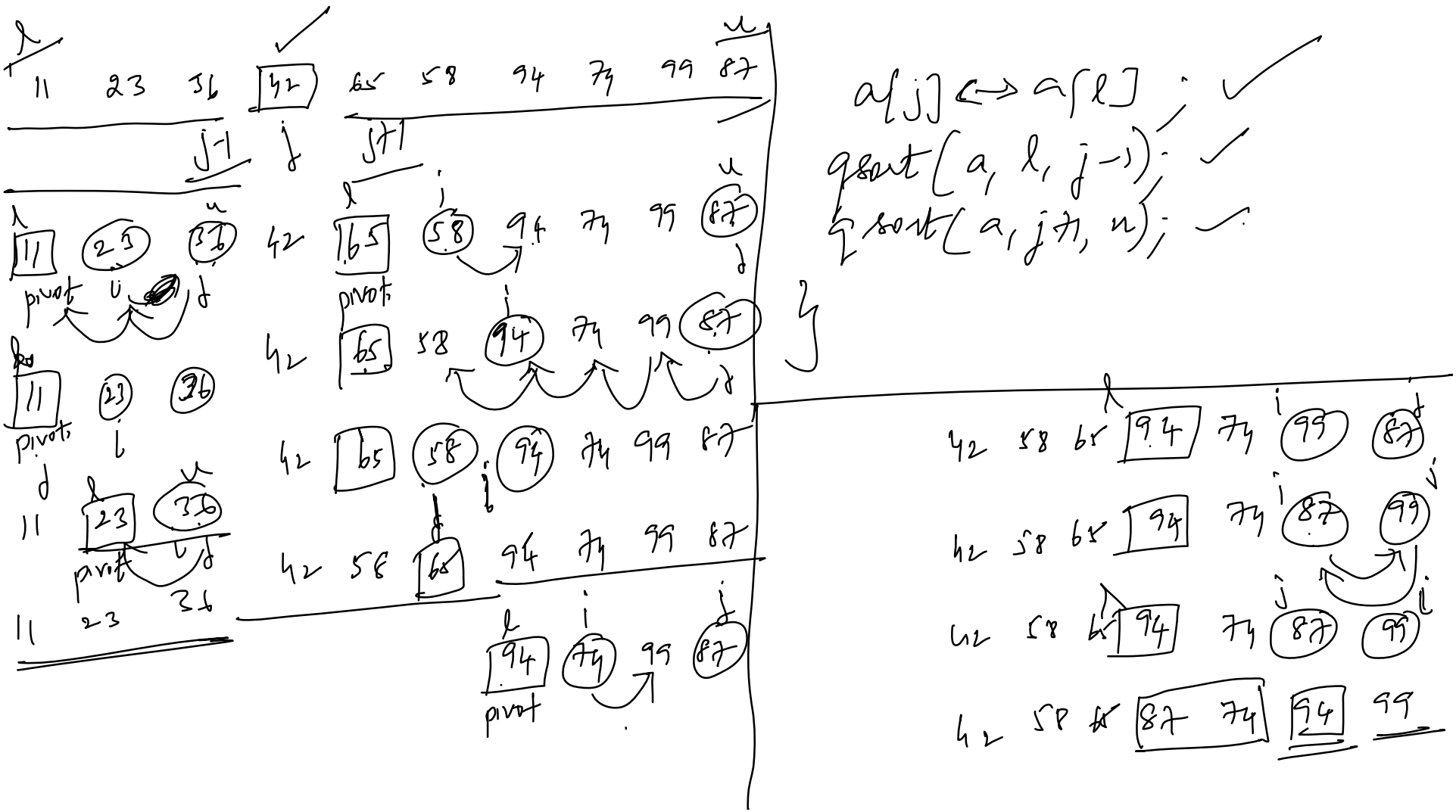
5) If the value present in j^{th} pos is > pivot decrements j by 1. Repeat till j^{th} pos val < pivot.

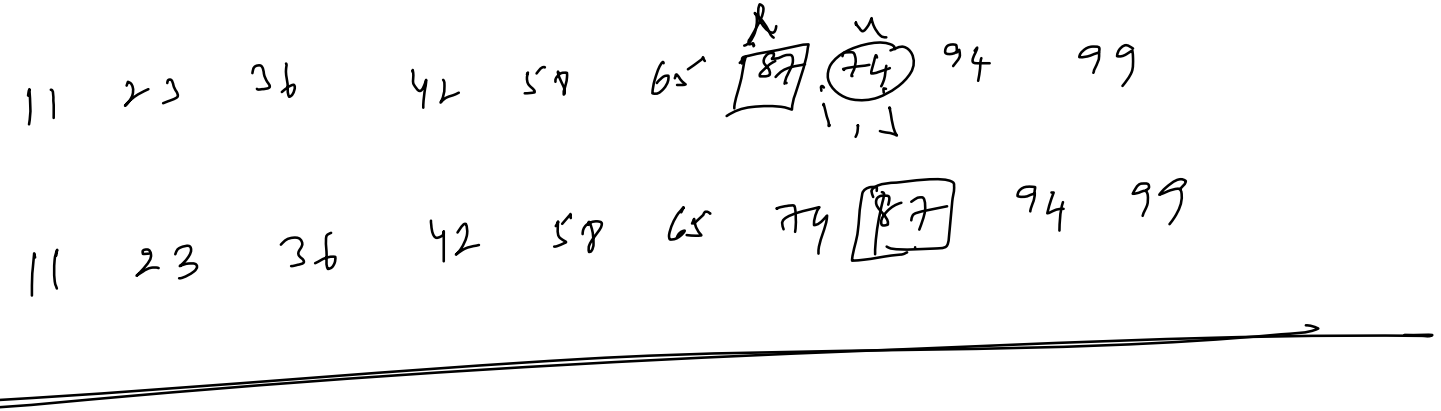
6) If i < j swap values present in i & j and repeat steps 4, 5, 6

7) If i > j swap values present in j & l positions (l position value means pivot) which makes pivot sit in its original position

8) repeat steps 2 to 7 for l to j-1 positions and j+1 to u positions of the array if l < u


l i
42 23 74 11 65 58 94 36 99 87
pivot
42 23 74 11 65 58 94 36 99 87
42 23 74 11 65 58 94 36 99 87
42 23 74 11 65 58 94 36 99 87
42 23 36 11 65 58 94 74 99 87
42 23 36 11 65 58 94 74 99 87
42 23 36 11 65 58 94 74 99 87
int i=l+1, j=u, pivot=a[l];
while(l<u)
while(i<=j)
while(a[i]<=pivot) 4
i++;
while(a[j]>pivot) 5
j--;
if(i<j)
a[i]↔a[j];
6


a[j] ↔ a[l] ;
qsort(a, l, j-1);
qsort(a, j+1, u);



Quick sort Best & Avg Case → $O(n \log_2 n)$

if list is not sorted

Worst Case → $O(n^2)$ → if list is already sorted

Radix Sort

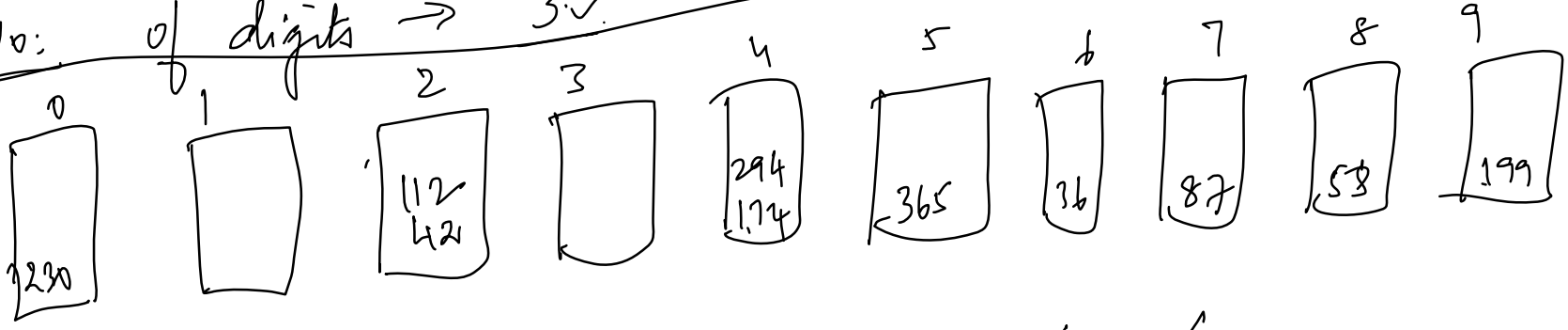
42 230 174 112 365 58 294 36 199 87

Max value → 365

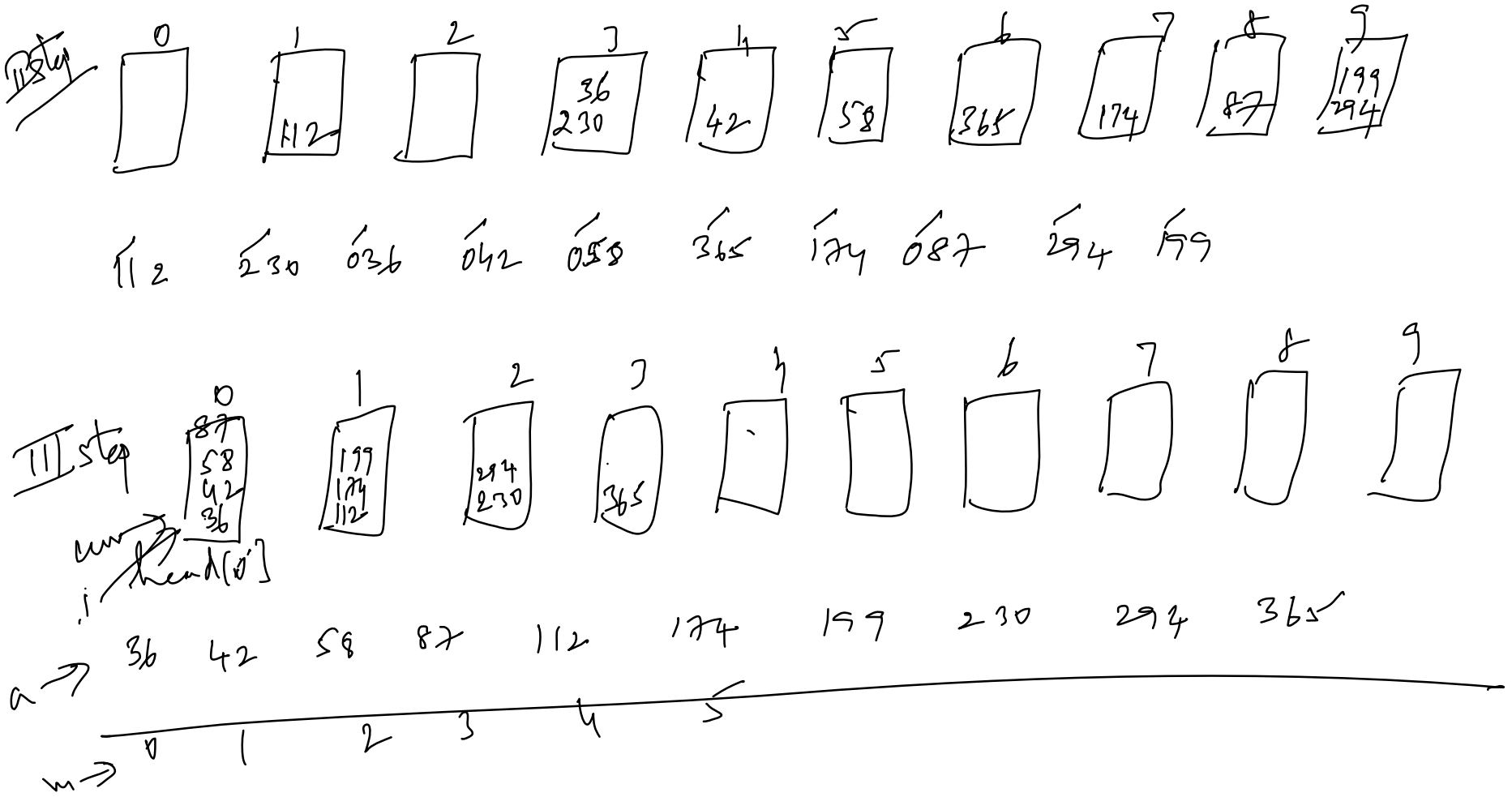
No: of digits → 3 → steps

365 % 10 → 5
36 % 10 → 6
3 % 10 → 3

Pstep



230 42 112 294 174 365 36 87 58 199

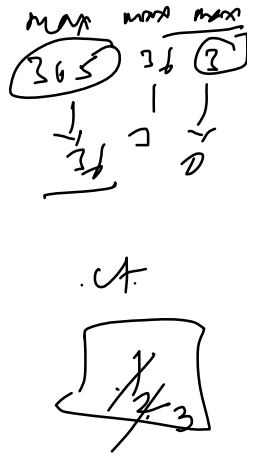

II step
0 1 2 3 4 5 6 7 8 9
112
36
230
42
58
365
174
87
199
294
112 230 036 042 058 365 174 087 294 199
III step
0 1 2 3 4 5 6 7 8 9
87
58
42
36
199
174
112
294
230
365
num[0']
i
a→
36 42 58 87 112 174 199 230 294 365
0 1 2 3 4 5
m→

```
int count(int a[], int n)
{
    int max = a[0], i, ct=0
    for(i=1; i<n; i++)
    {
        if(max < a[i])
            max = a[i];
    }
```

→ Maximum no

```
    while(max > 0)
    {
        max = max/10;
        ct++;
    }
    return ct;
}
```

→ No: of digits in max no:



```
struct node
{
    int data;
    struct node *next;
} *head[10];
```

```
struct node* append( struct node *n, int x)
{
    struct node *nn, *temp=n;
nn= (struct node *) malloc (sizeof( struct node));
    nn-> data ==x;
    nn-> next= NULL;
  if(n== NULL)
  {
      n= nn;
      return n;
  }
```

```
while(temp->next!=NULL)
  temp =temp->next;
temp->next =nn;
return n;
}
```

```
int digit(int x, int p)
{
    int i=1;
    while(i<p)
    {
        x=x/10;
        i++;
    }
    return x%10;
}
```

x => 365

365%10

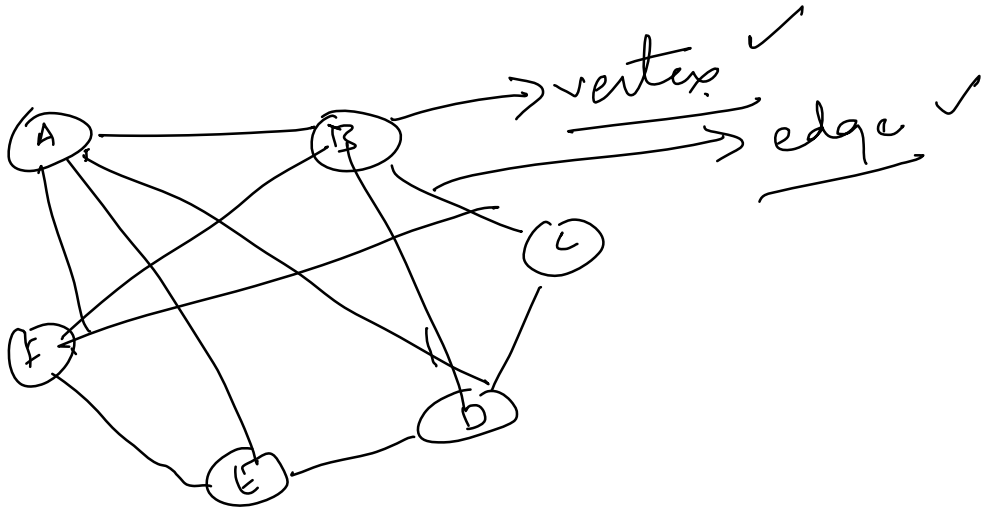
5

3

36%10

```
struct node *cur;
int ct, p, i, m, k;
for(i=0; i<10; i++)
{
  head[i] = NULL;
}
ct = count(a, n);
for(p=1; p<=ct; p++)
{
  for(j=0; j<n; j++)
  {
    k = digit(a[j], p)
    head[k] = append(head[k], a[j]);
  }
  for(i=0, m=0; i<10; i++)
  {
    cur = head[i];
    while(cur != NULL)
    {
      a[m] = cur->data;
      m++;
      cur = cur->next;
    }
    head[i] = NULL;
  }
}
```

Graph :- It is a collection of vertices & edges

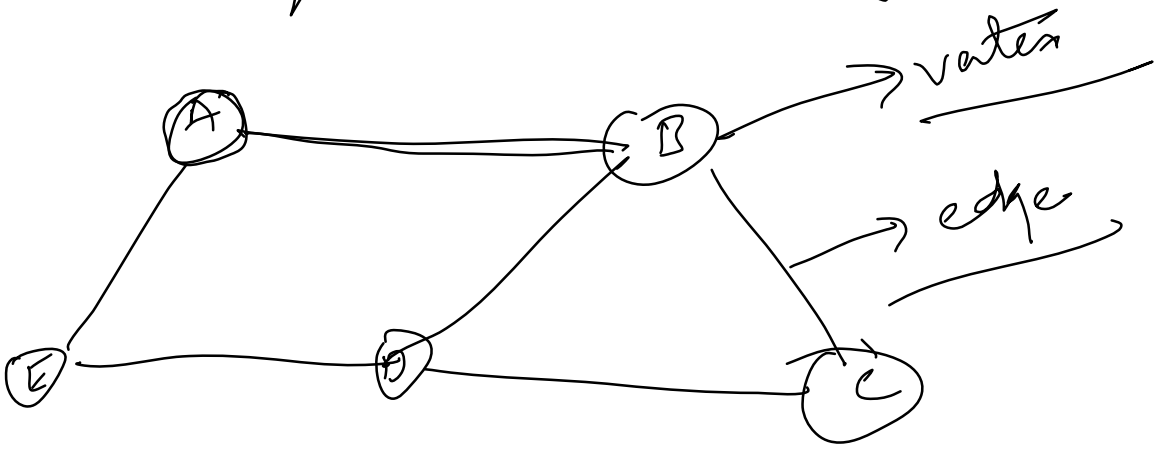


Graph

Non Linear data structure

hierarchical

It is a collection of vertices and edges



Shortest Path b/w any 2 cities

Hamiltonian Cycles

Graph Coloring

Spanning Trees

Travelling Salesperson Problems etc

✓ Social N/w's

✓ Navigation systems

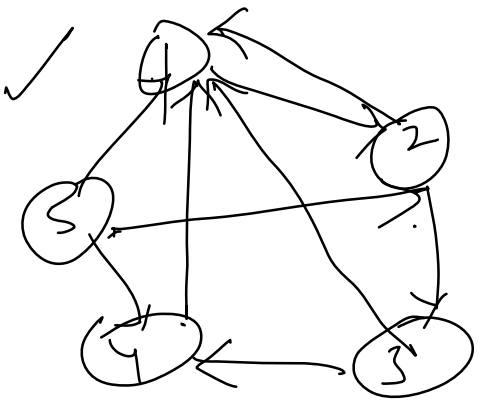
✓ Biological systems

Terminology

Directed Graph

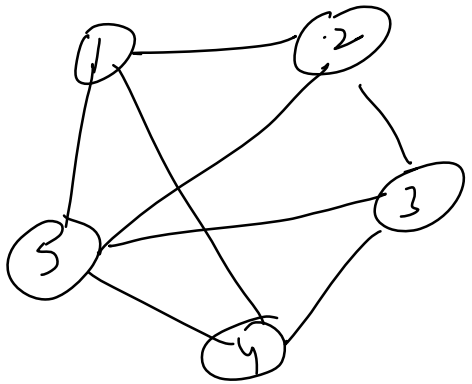
every edge of the graph has some direction

✓



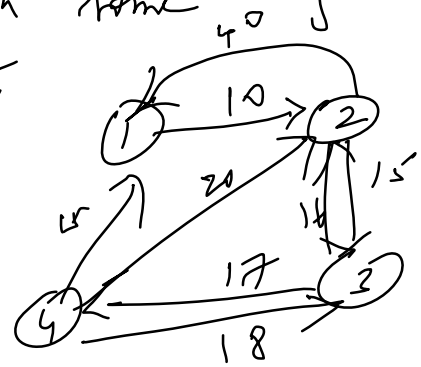
Undirected Graph

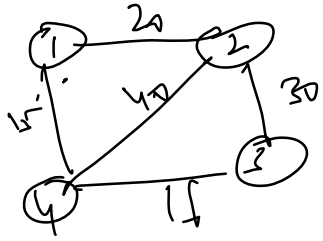
every edge of the graph is without direction



Weighted Graph

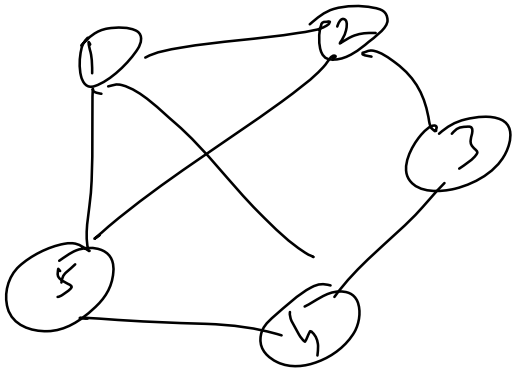
every edge is associated with some weight or cost





Adjacency

Two vertices are said to be adjacent to each other if an edge exists b/w the 2 vertices



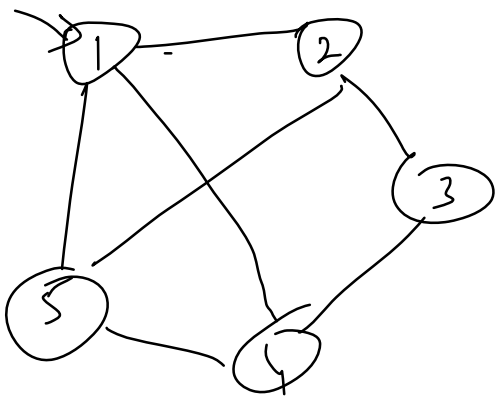
1 is adjacent to 2, 5, 4

5 is adjacent to 1, 4, 2.

Path :- If we can reach a vertex y from vertex x using some of the edges then there exists a path b/w x & y.

1-5-4 / 1-4 / 1-2-3-4 / 1-2-5-4

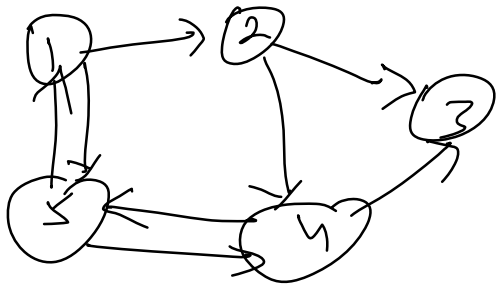
Cycle:- / Closed Path — If we can reach a vertex x by using some edges starting from vertex x then the path is called cycle



1-2-3-4-5-1

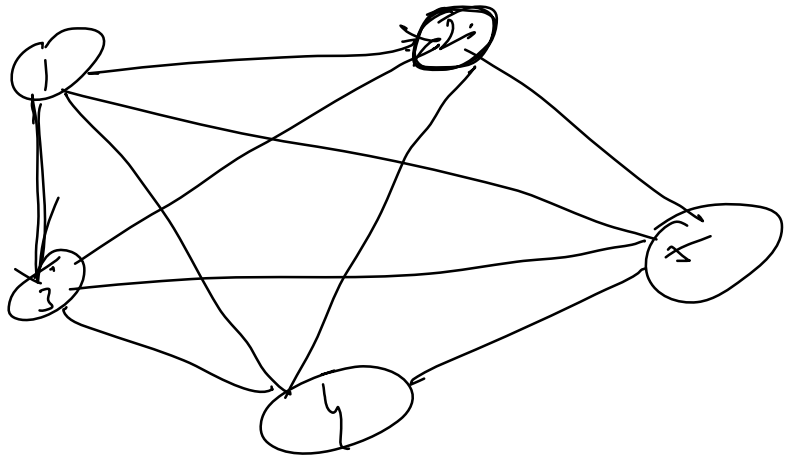
1-4-5-1

1-2-5-1

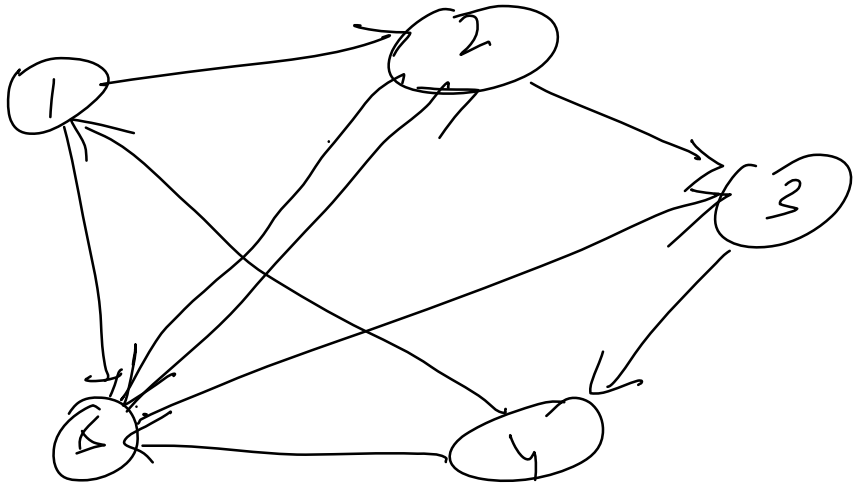


1-2-4-5-1

Complete Graph:- A graph in which every vertex has an edge with all the remaining vertices of the graph is called complete graph



Outdegree, Indegree, Total degree of a vertex



Outdegree(5) = 2

Indegree(5) = 3

Total degree(5) = 5

No: of edges ending at 5 is indegree(5)

No: of edges leaving 5 is outdegree(5)

Indegree + Outdegree

↓

Total degree

Representation of Graphs

1) Adjacency Matrix

2) Adjacency List

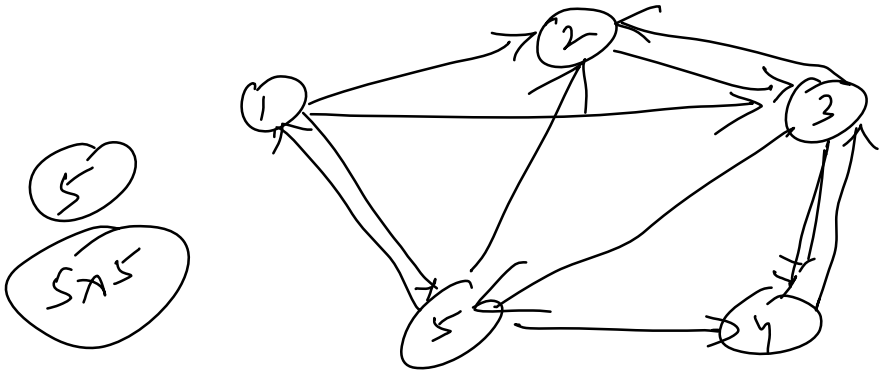
Adjacency Matrix :- Assuming graph has n vertices we construct an n×n matrix where the matrix values are either 1 or 0 in case of a directed graph or undirected graph (or) cost or ∞ in case of a weighted graph.

↓ if edge exists

↓ if edge does not exist

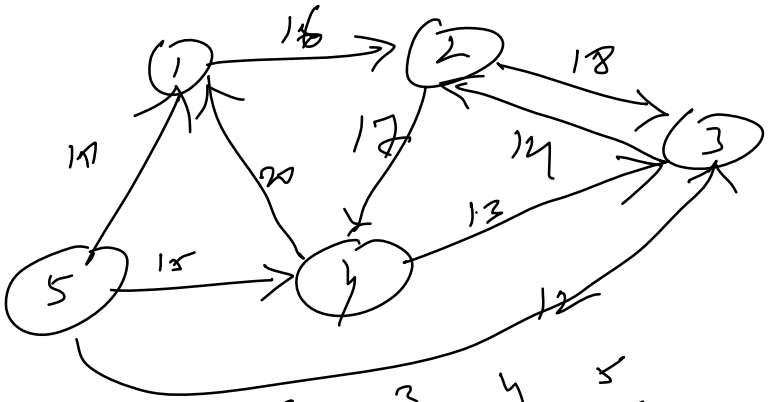
In case of directed graph or undirected graph we store 1 if edge exists b/w 2 vertices, otherwise we store 0.

Directed Graph



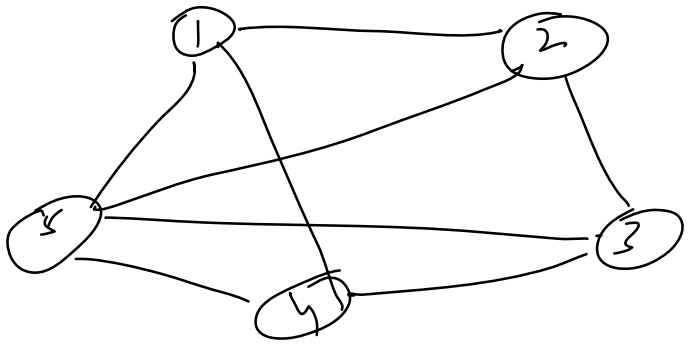
	1	2	3	4	5
1	0	1	1	0	1
2	0	0	1	0	0
3	0	1	0	1	1
4	0	0	1	0	0
5	1	1	0	1	0

Weighted Directed Graph



	1	2	3	4	5
1	∞	16	∞	∞	∞
2	∞	∞	18	17	∞
3	∞	14	∞	∞	∞
4	20	∞	13	∞	∞
5	10	∞	12	15	∞

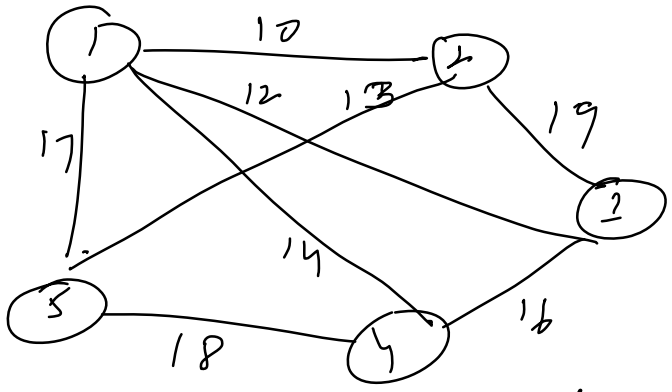
Undirected Graph



	1	2	3	4	5
1	0	1	0	1	1
2	1	0	1	0	1
3	0	1	0	1	1
4	1	1	1	0	1
5	1	1	1	1	0

Symmetric Matrices

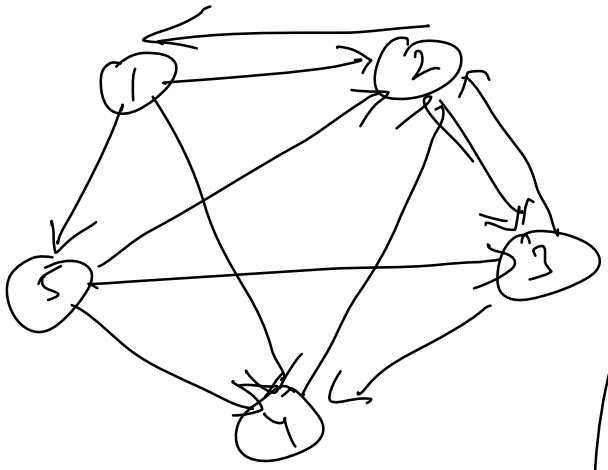
Undirected Weighted Graph

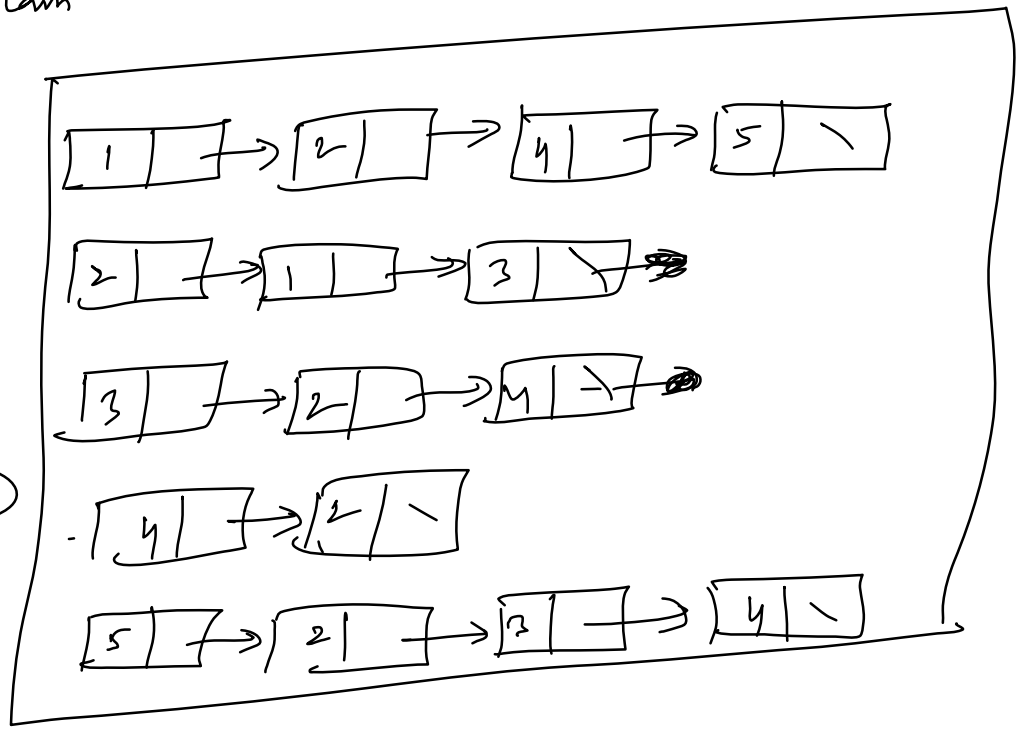


	1	2	3	4	5
1	∞	10	12	14	17
2	10	∞	19	∞	13
3	12	19	∞	16	∞
4	14	∞	16	∞	18
5	17	13	∞	18	∞

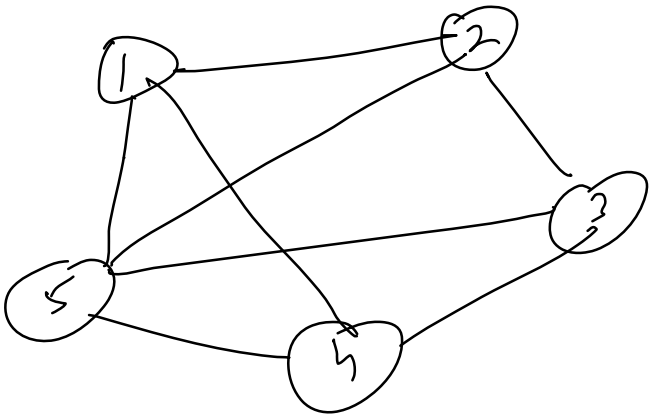
Adjacency List:- Array of linked lists where for every vertex we maintain a linked list.

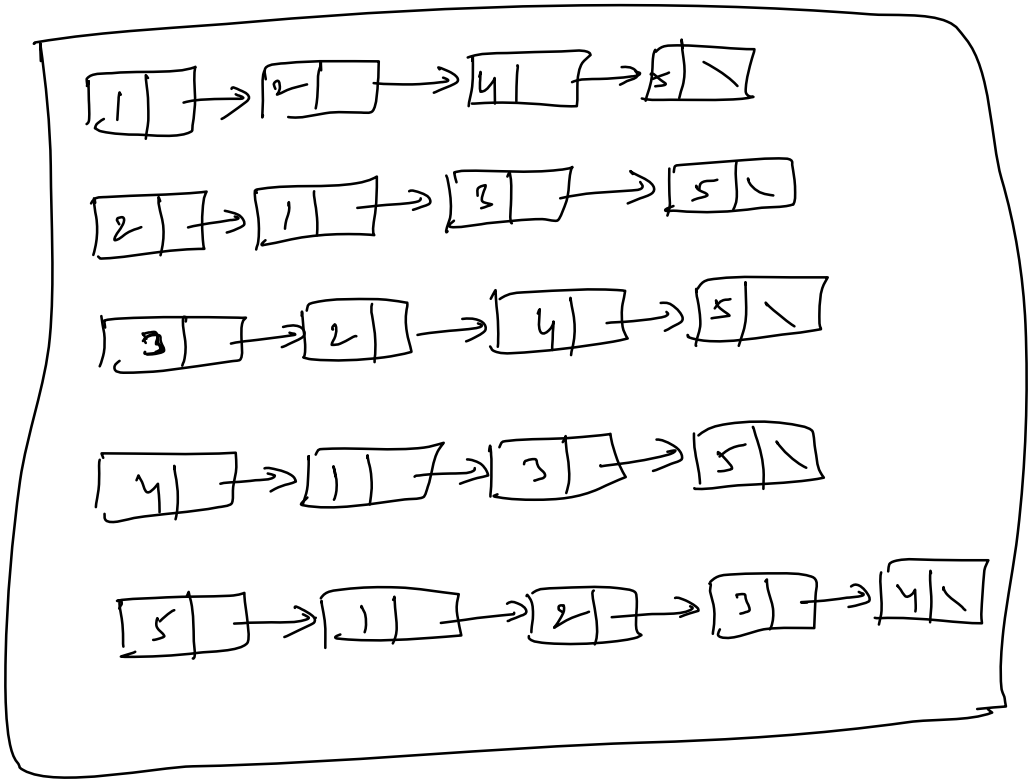
Directed Graph



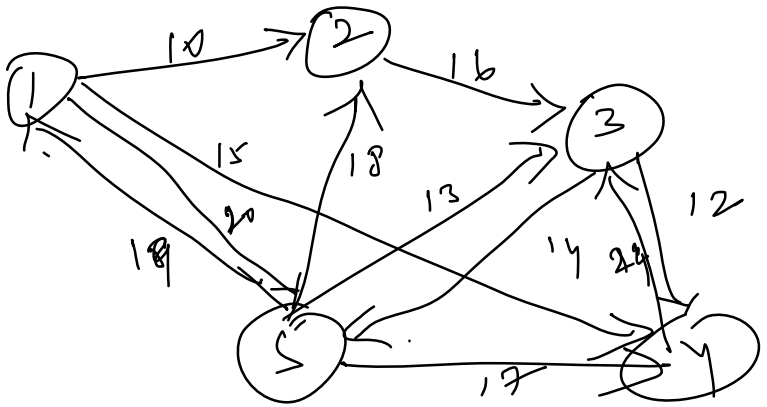


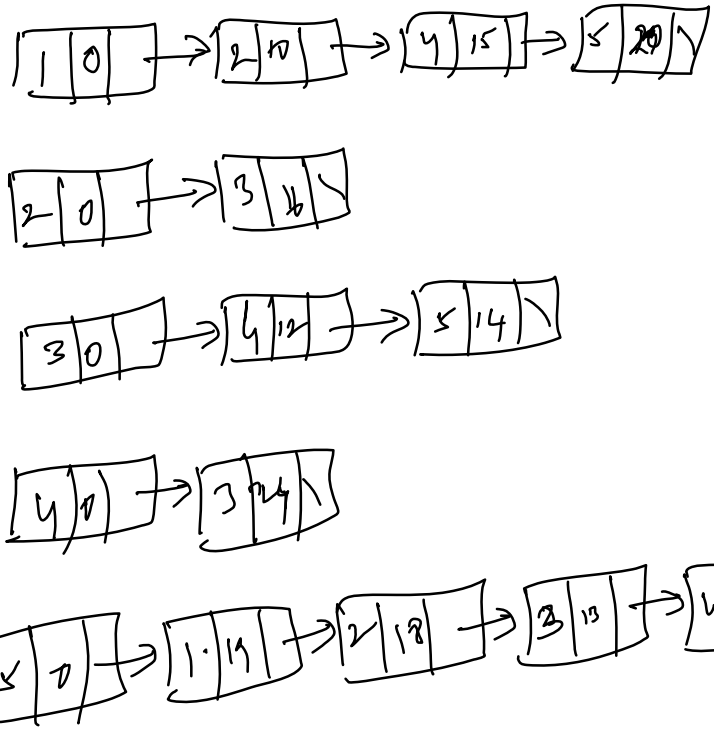
Undirected Graph





Weighted Directed Graph





Graph Traversals:- It is the process of visiting each & every vertex of the graph atleast & atmost once starting from an assumed source vertex.

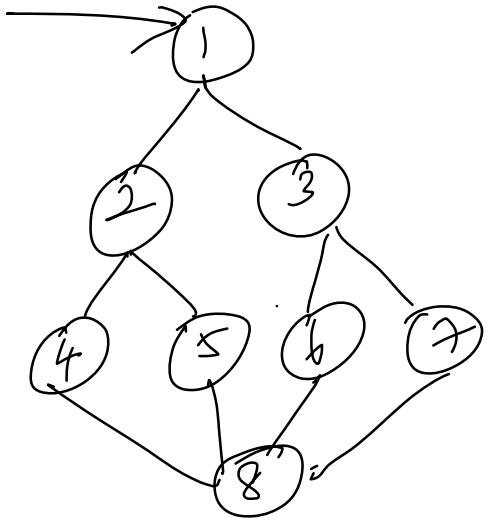
Two Graph Traversal Techniques

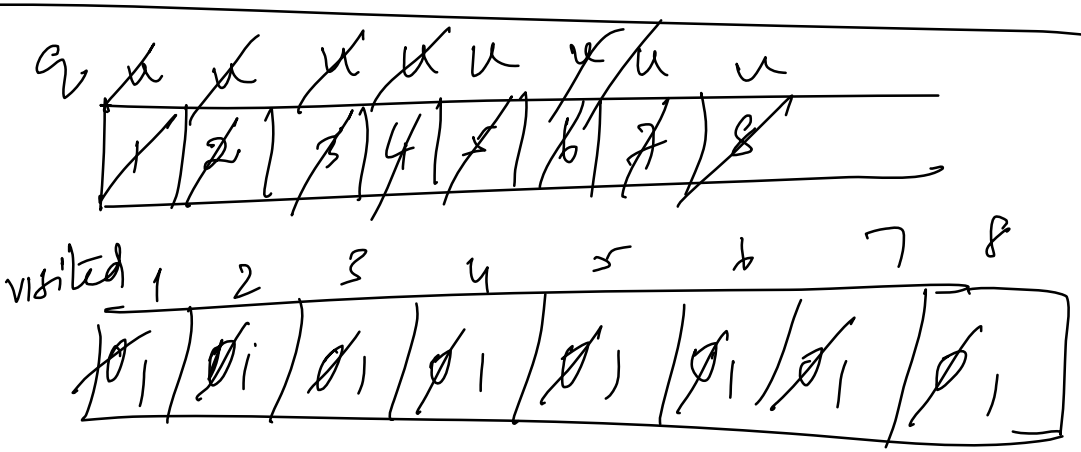
1) Breadth First Search (queue)
2) Depth First Search (stack)

Breadth First Search

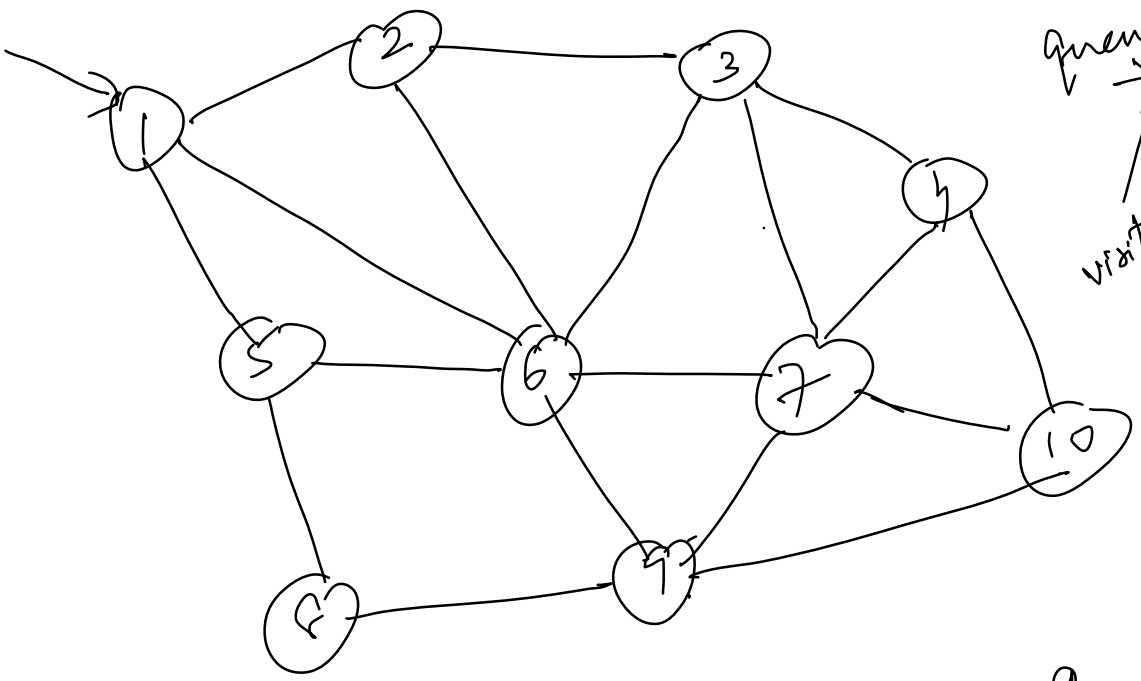
1) Uses a queue ✓, visited[] array which initially stores 0's
2) Assume a vertex (u) to be a source vertex ✓
3) Add that vertex (u) to the queue and assume that vertex is visited by making visited[vertex] as 1
4) Identify the adjacent vertices of u and if their visited[] value is 0, add them to the queue and make their visited[] values as 1
5) delete (u) → front element from the queue → and print it; and repeat steps 4 & 5

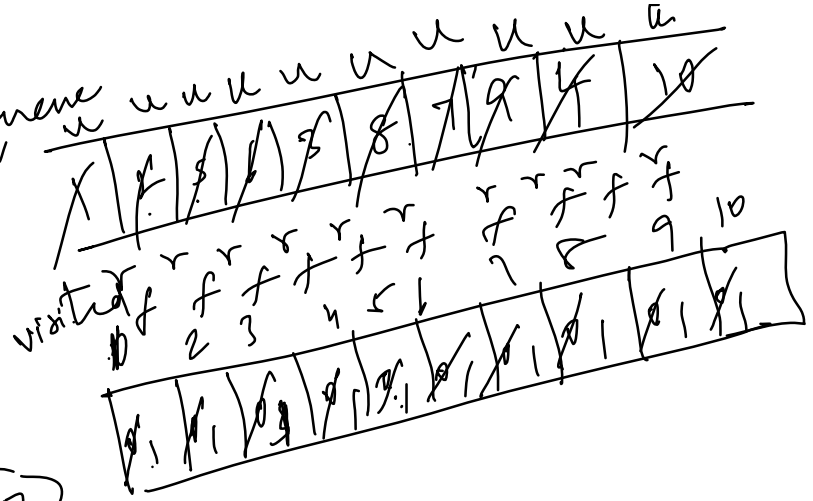
by making the front element of the queue as u until the queue becomes empty.





1 2 3 4 5 6 7 8

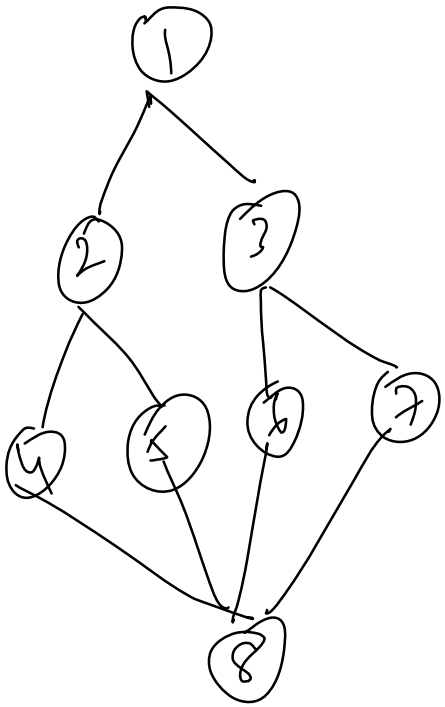




1 2 5 6 3 8 7 9 4 10

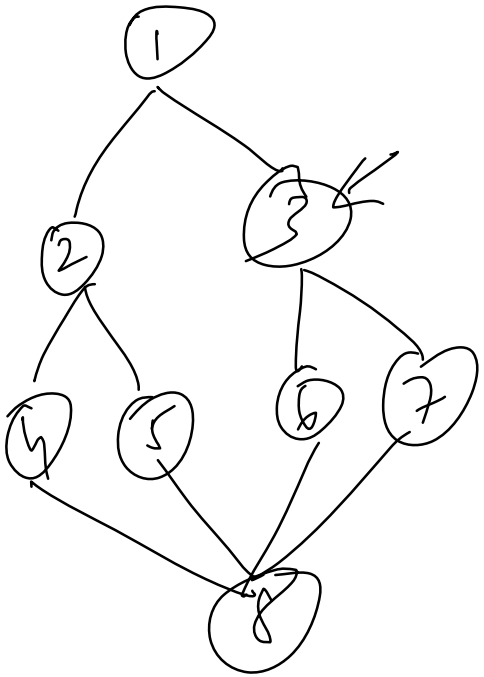
```
void qinsert(int x)
{
    r++;
    q[r] = x;
    if (f == -1)
        f = 0;
}
```

```
int qdelete()
{
    int x;
    if (f == -1)
        return 0;
    x = q[f];
    if (f == r)
    {
        f = r = -1;
    }
    else
        f++;
    return x;
}
```



	1	2	3	4	5	6	7	8
1	0	1	1	0	0	0	0	0
2	1	0	0	1	1	0	0	0
3	1	0	0	0	0	1	1	0
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	1	0	0	0	0	1
8	0	0	0	1	1	1	1	0

```
void bfs(int v, int n)
{
    qinsert(v);
    visited[v] = 1;
    u = v;
    while(1)
    {
        for(i = 1; i <= n; i++)
        {
            if(a[u][i] == 1 && visited[i] == 0)
            {
                qinsert(i);
                visited[i] = 1;
            }
        }
        x = qdelete();
        if(x != 0)
            printf("%d\t", x);
        if(f == -1)
            break;
        else
            u = q[f];
    }
}
```



visited

1	2	3	4	5	6	7	8
0,1	0	0,1	0,1	0,1	0,1	0,1	0

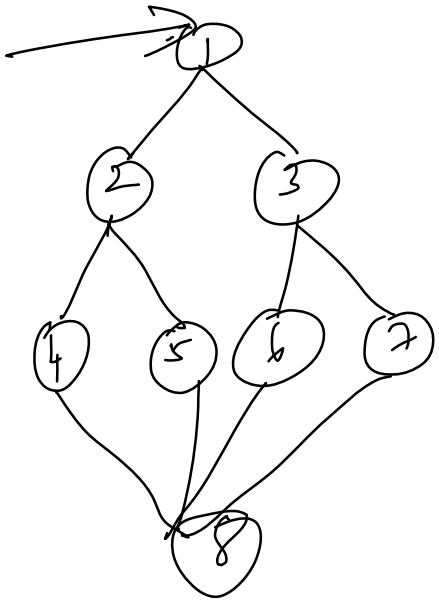
3 1 6 7 2 8 4 5

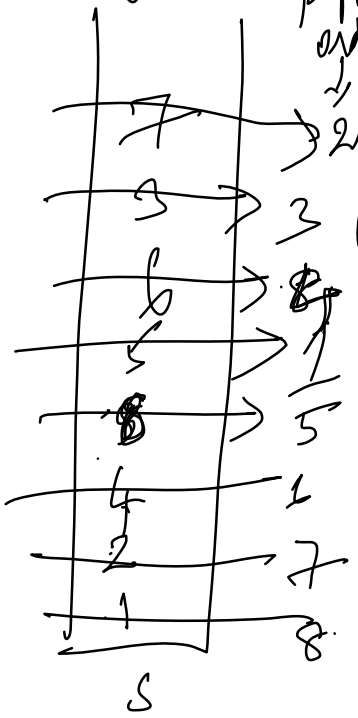
Depth first Search :- uses a stack

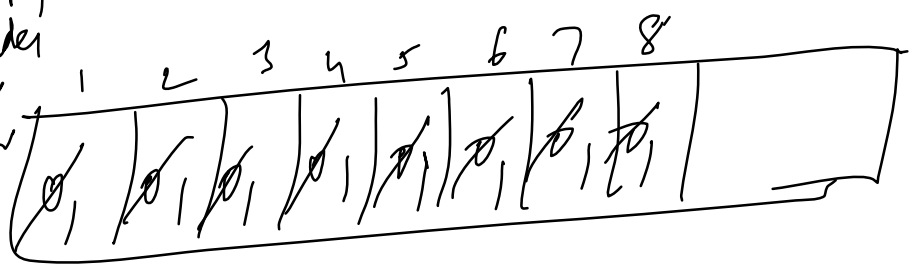
1) Assumes a source vertex and pushes it into stack
2) Identifies adjacent vertices of source vertex and pushes an adjacent vertex into the stack and repeat step 2 with the stacks top most element until a vertex whose all adjacent vertices are pushed into stack and if found pop the element.

Repeat the process of popping element from stack until it becomes empty.

The order of pushing elements into stack is DFS

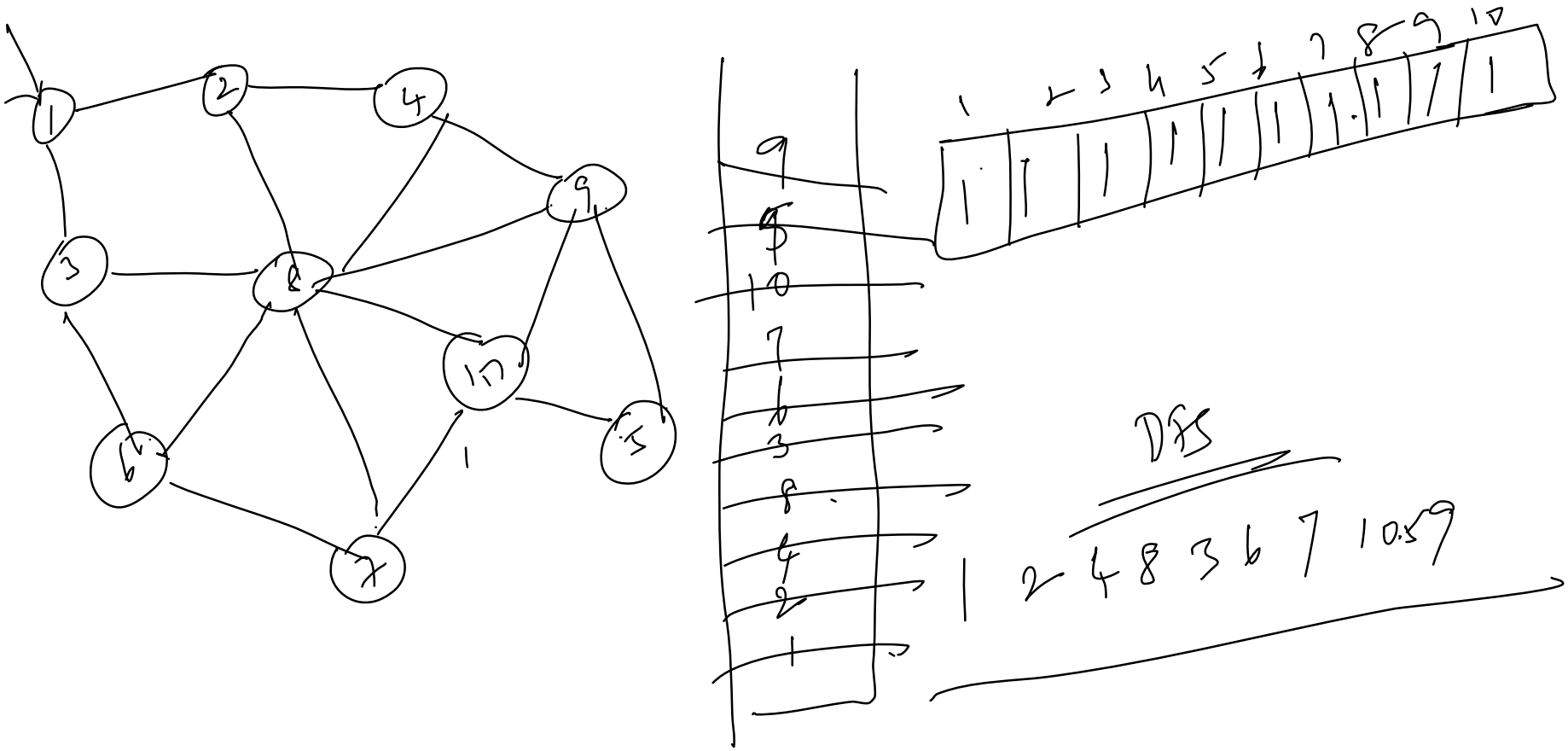




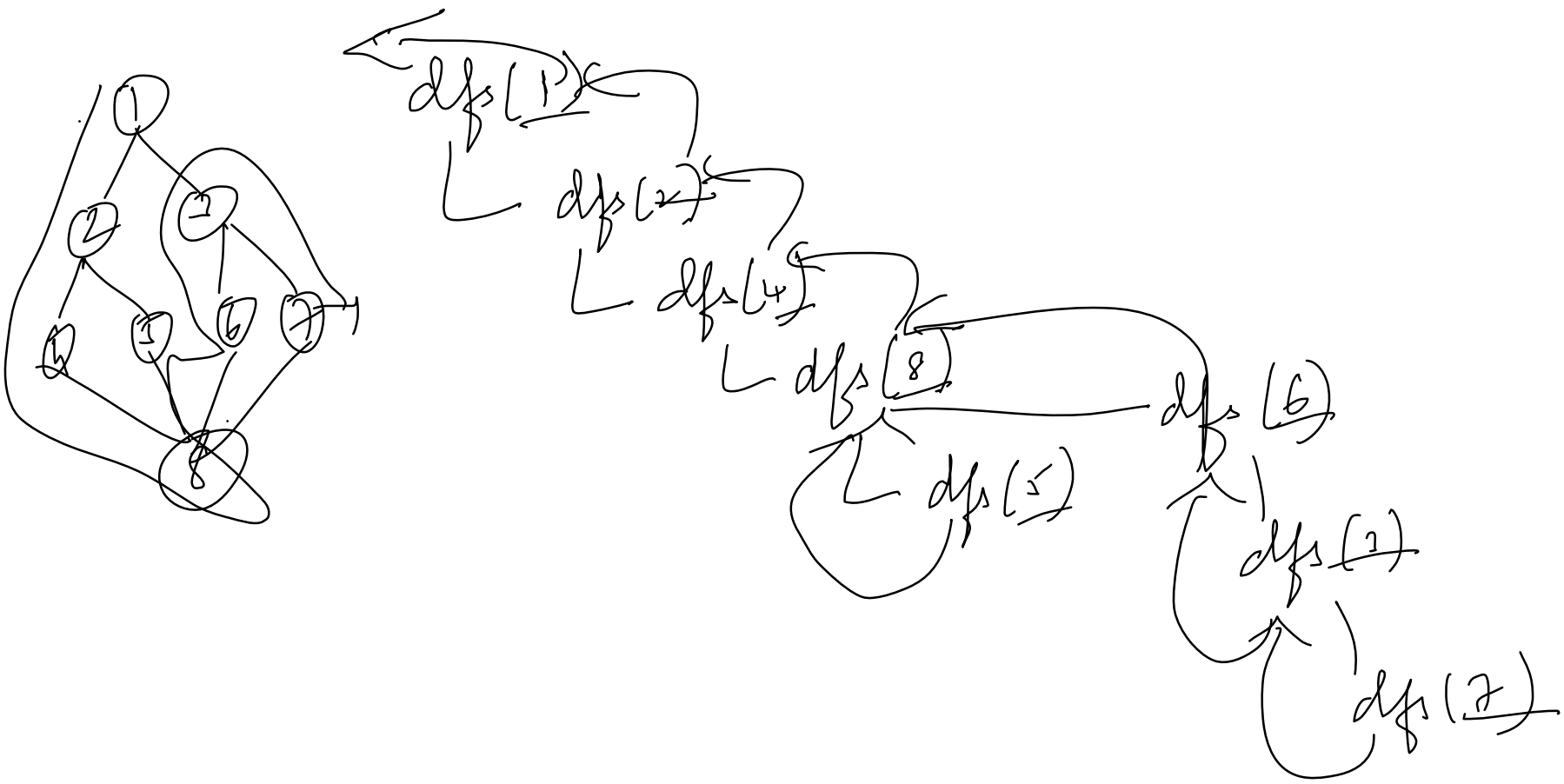


DFS

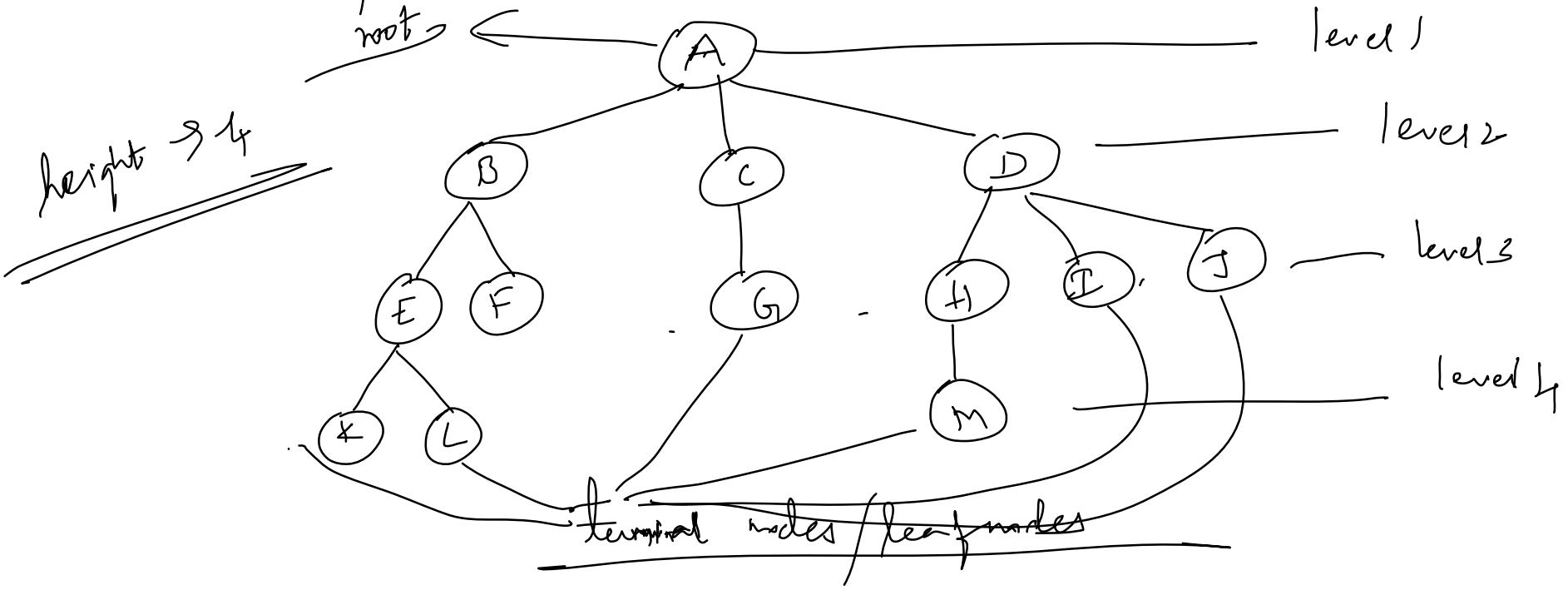
1, 2, 4, 8, 5, 6, 3, 7


DFS
1 2 4 8 3 6 7 10 5 9

```
void dfs( int v, int n)
{
    int i;
    visited[v] = 1;
    printf( "%d \t", v);
    for( i = 1, i <= n; i++)
    {   if [a[v][i] == 1 && visited [i] == 0)
            dfs ( i, n);
}   }
```


dfs(1)
dfs(2)
dfs(4)
dfs(8)
dfs(5)
dfs(6)
dfs(7)
dfs(3)

Trees :- Collection of vertices and edges where one vertex or node is designated as root node and all other vertices are partitioned in $n \geq 0$ disjoint sets or trees



The no: of subtrees of a particular node is called degree of that node

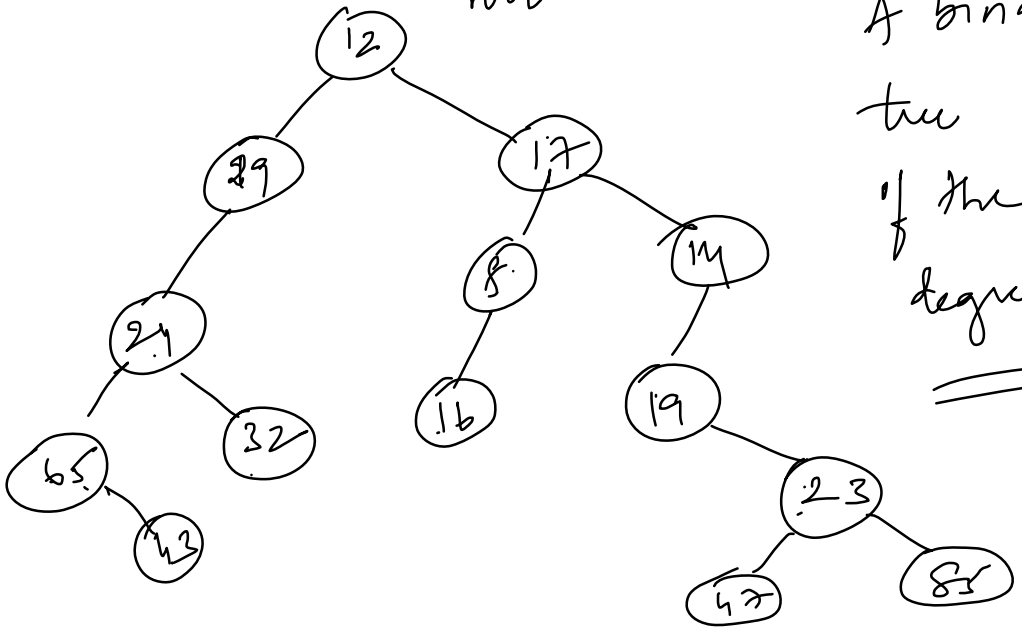
Eg:- degree of A is 3, B is 2, K is 0, L is 0

Nodes with degree 0 are called leaf nodes / terminal nodes

Since we are assuming root node is at level 1, its child nodes are at level 2, ----

The highest level of the tree is called height/depth of the tree.

Binary Tree :- A binary tree is a finite set of nodes that can be either empty or consists of a root node and 2 disjoint binary trees called as left & right subtrees.

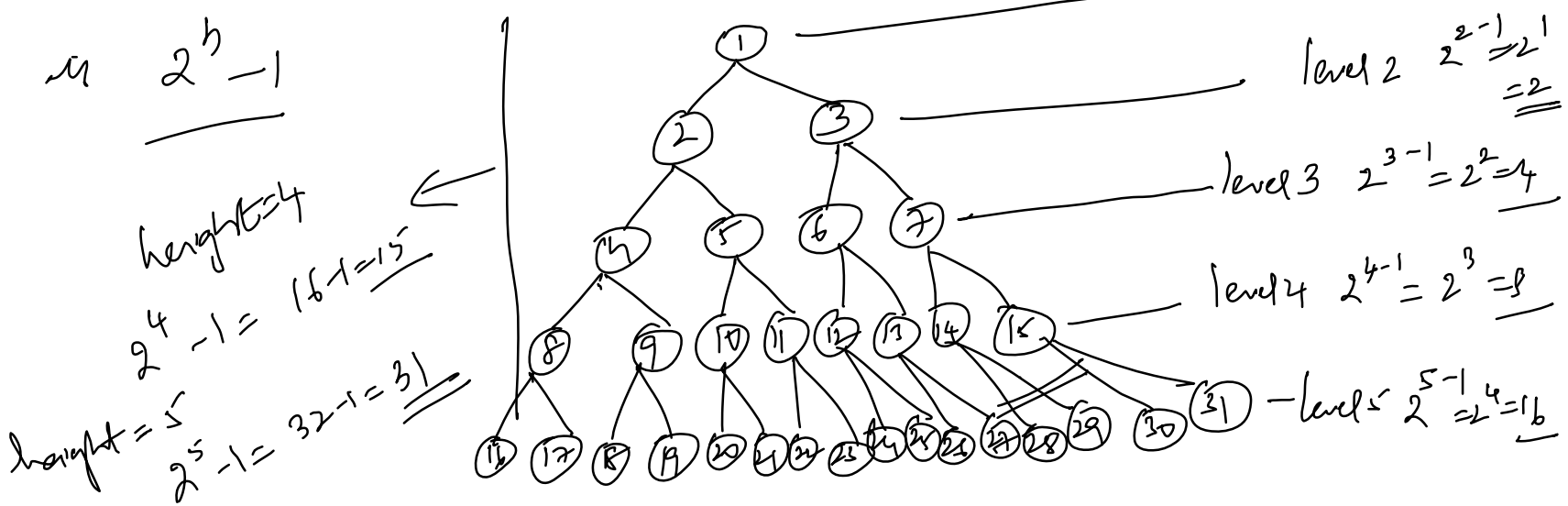


A binary tree is a tree where every node of the tree has a degree of 0/1/2

Properties

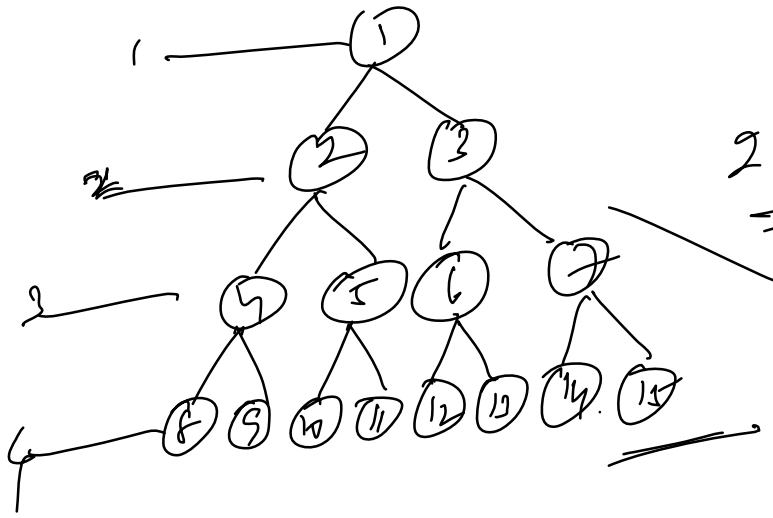
1) The max no: of nodes in a binary tree at level i is 2^{i-1} $i \geq 1$

2) The max no: of nodes in a binary tree of height h (depth (or) level) is $2^h - 1$



A binary tree of height h consisting of $2^h - 1$ nodes is called a full binary tree

height = 4



$2^4 - 1 = 15$

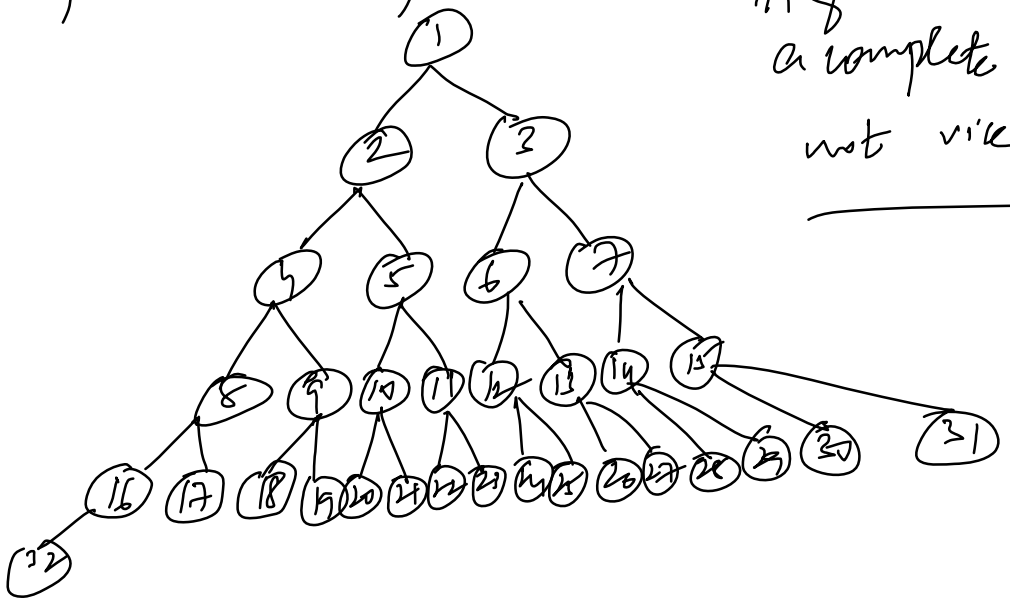
full binary tree

Complete Binary Tree :- A binary tree of height h which is a full binary tree till level h-1 and in level h the nodes are filled from left to right without any gaps is called a complete binary tree.

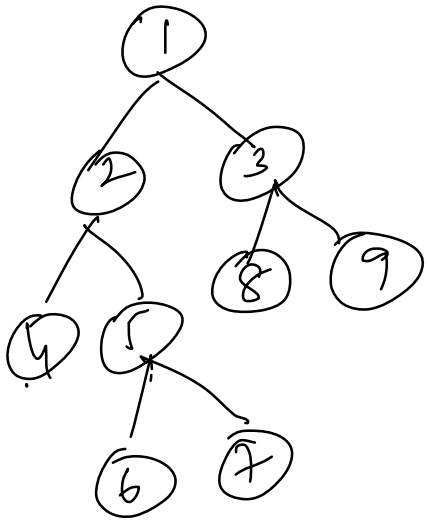
A full binary tree is a complete binary tree but not vice versa →

height = 5

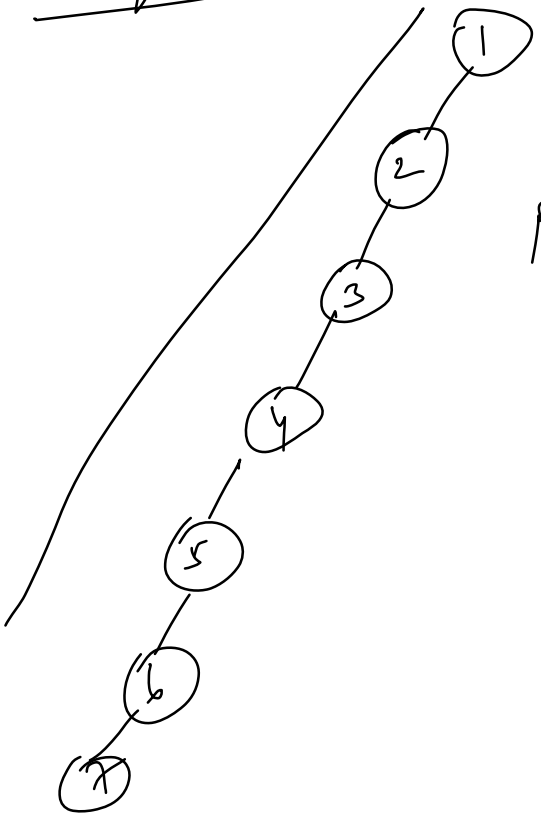
height = 4



Strictly binary tree:- A binary tree where every node has either 0 or 2 child nodes

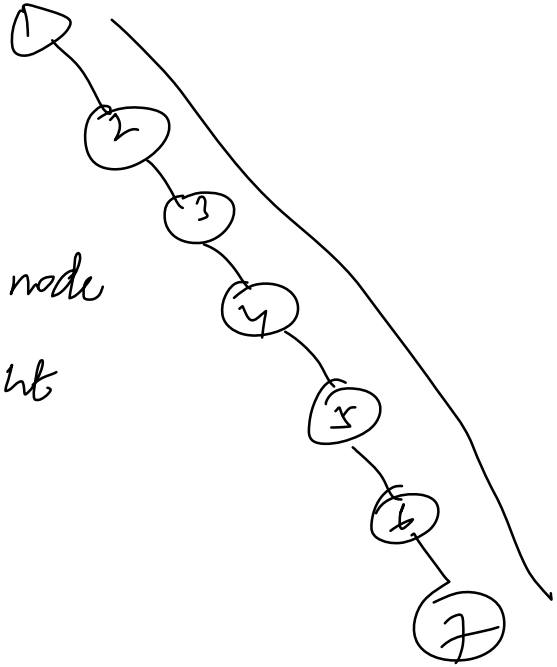


left Skewed Binary Tree



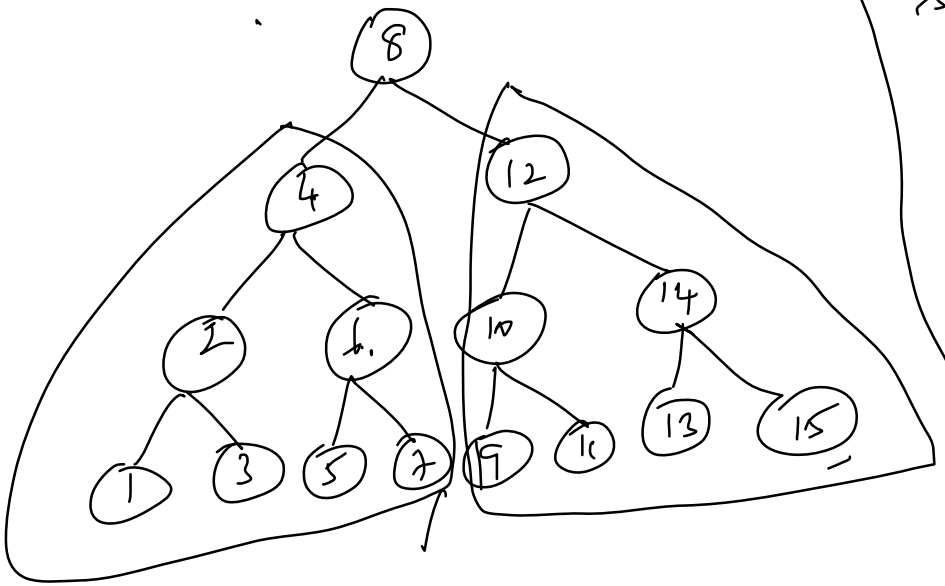
Binary tree where every node has only left child nodes

Right Skewed Binary Tree



Binary tree where every node has only right child nodes

Binary Search Tree :- It is a binary tree where every node of the tree has a value greater than the values in its left subtree and lesser than the values in its right subtree.




```
struct tnode
{ int data;
  struct tnode *lchild;
  struct tnode *rchild;
};
```

Node structure

Operations on a Binary Search Tree

1) Create
2) Traversals (Display)
   - Preorder
   - Inorder
   - Postorder
3) Search
4) MaxElement
5) MinElement
6) Height ✓
7) Deletion

Create a Binary Search Tree

1) create a tree node
2) store data into that node and left child & right child as NULL. as the new node which is created will always be a leaf node.
3) If bstree is not present the newly created node will become the binary search tree.
4) If bstree is already present we move to the required position in the tree and adds the new node as leaf node to the already present BST

Define the node

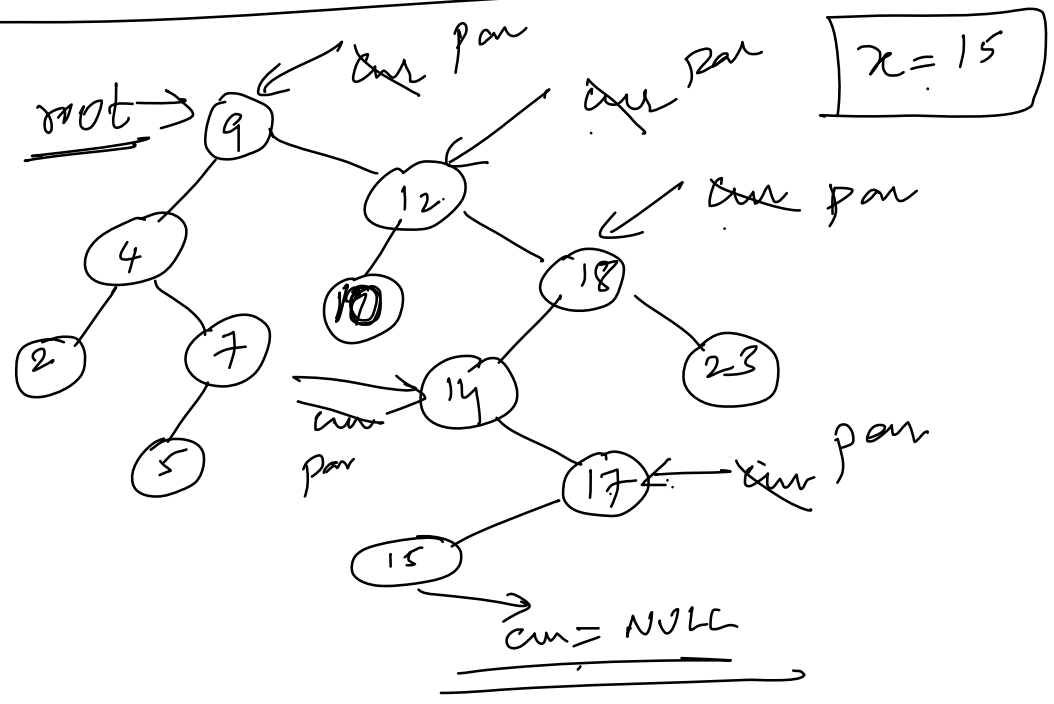
```
struct tnode
{
  int data;
  struct tnode *lchild;
  struct tnode *rchild;
};

typedef struct tnode* Node;
```

root = NULL.

root → 15

nn



```
Node create(Node root, int x)
{
    Node nn, cur, par;
nn = (Node) malloc(sizeof(struct tnode));
  nn->data = x;
    nn->lchild = NULL;
    nn->rchild = NULL;
    if(root == NULL)
    {
        root = nn;
        return root;
    }
    cur = root;
    while(cur != NULL)
    {
        par = cur;
        if(x > cur->data)
            cur = cur->rchild;
        else
            cur = cur->lchild;
    }
```

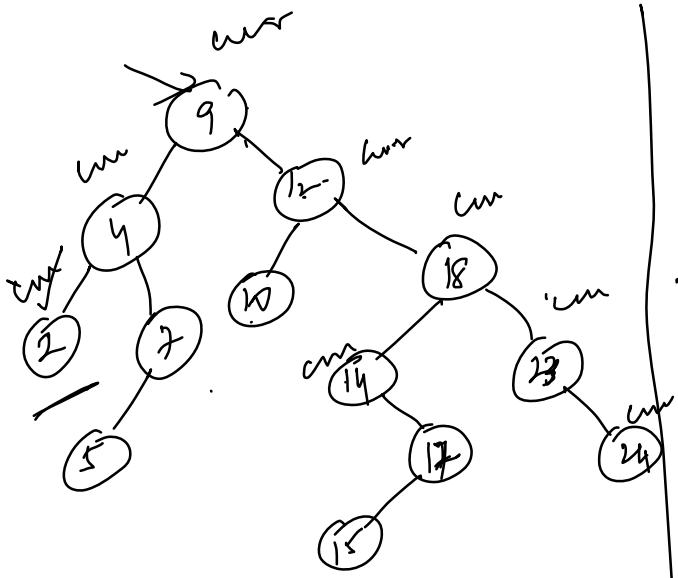
```
if( x > par->data)
    par->rchild = nn;
else
    par->lchild = nn;
return root;
}
```

Traversal:- It is the process of displaying the elements of a binary search tree

Preorder Traversal → visit the root node (print the value)
first Traverse the left subtree
Traverse the right subtree

Inorder Traversal → Traverse the left subtree
middle visit the root node
Traverse the right subtree

Postorder Traversal → Traverse the left subtree
last Traverse the right subtree
visit the root node



Preorder → root – left – right

9, 4, 2, 7, 5, 12, 10, 18, 14, 17, 15, 23, 24

Inorder → left – root – right

2, 4, 5, 7, 9, 10, 12, 14, 15, 17, 18, 23, 24

Postorder → left – right – root

2, 5, 7, 4, 10, 15, 17, 14, 24, 23, 18, 12, 9

Search → 17

curr = root;

x > curr → data

curr = curr → rchild;

else if (x < curr → data)

curr = curr → lchild;

else

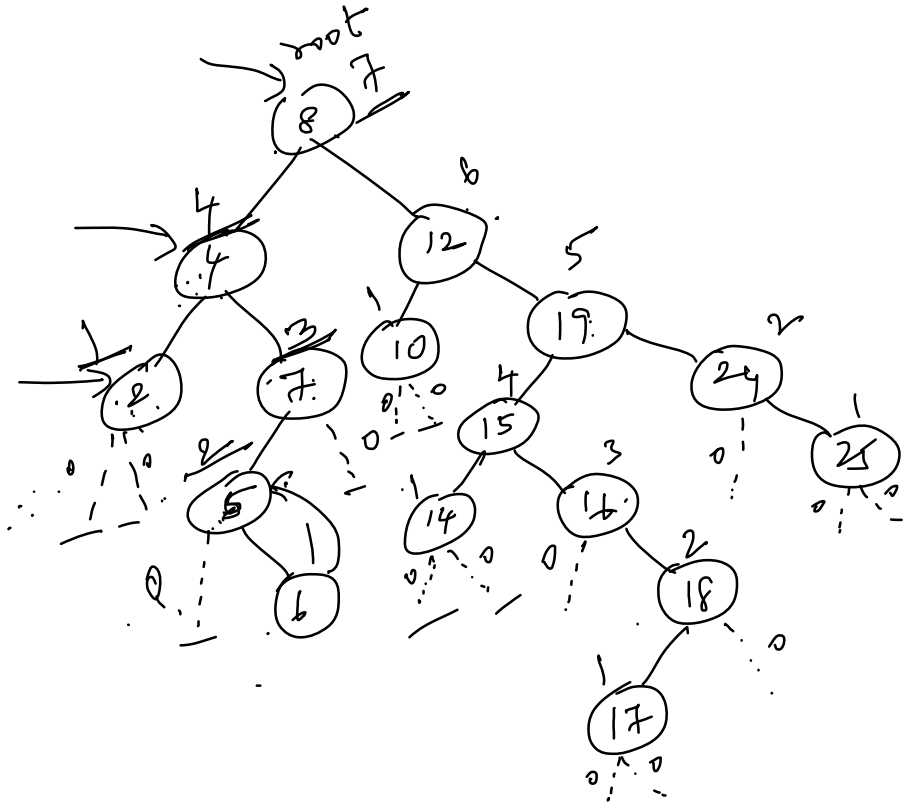
min

```
cur = root;
while (cur->lchild != NULL)
{
cur = cur->lchild;
}
return cur->data
```

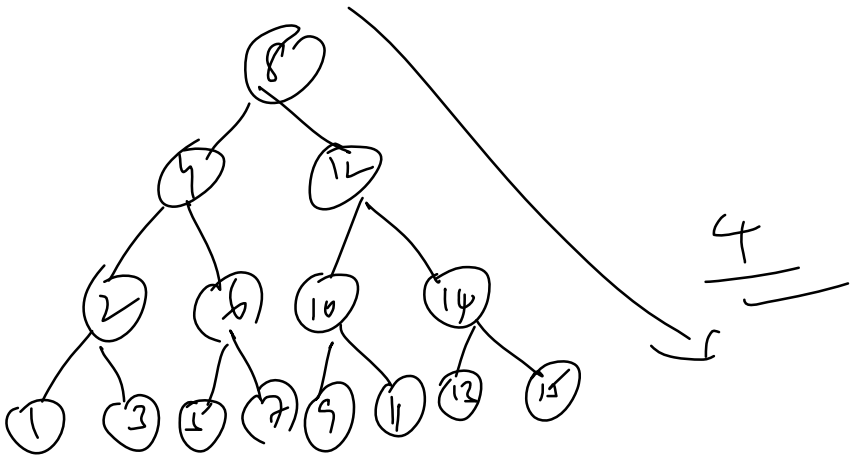
max

```
cur = root;
while (cur->rchild != NULL)
{
  cur = cur->rchild;
}
return cur->data;
```

Height




```
int lh, rh;
if(root == NULL)
    return 0;
else
{
  lh = height(root->lchild);
  rh = height(root->rchild);
return (lh > rh ? lh : rh) + 1;
}
```


8
4
12
2
6
10
14
1
3
5
7
9
11
13
15
4

Delete a node from a binary search tree

1) bstree is not present root == NULL
2) only one node is present in bst
3) bst is present then search for the node to be deleted
4) In the search process if NULL is encountered means element to be deleted is not found
5) node to be deleted is found
   a) node is the leaf node

b) node to be deleted has no left subtree

c) node to be deleted has no right subtree

d) node to be deleted has both left & right subtrees

↓ replace the node with its inorder successor

(1)

```
// Tree is not present
if ( root == NULL)
{
    printf ("Tree is empty");
    return root;
}
```

root → (7)

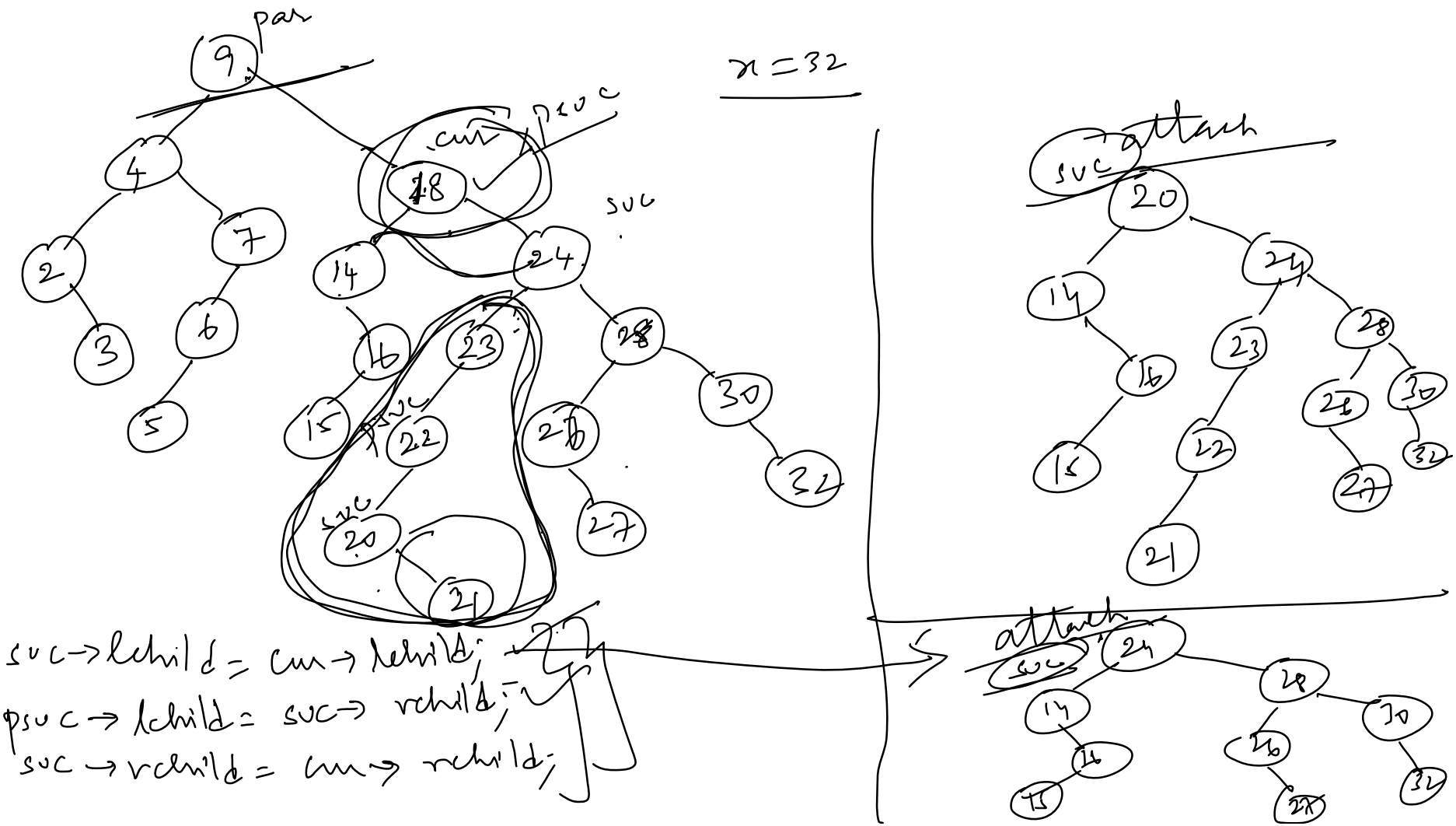
```
// Only one node is present
```

(2)

```
if ( root->lchild == NULL & root->rchild == NULL)
{
    if ( root -> data == x)
    {
        root = NULL;
        printf ("Element deleted");
    }
    return root;
}
else
{
    printf ("Element not found");
    return root;
}
```

(3) // Tree is present & search for the node

```
cur = root;
while( cur -> data != x && cur != NULL)
{
    par = cur;
    if( x > cur -> data )
        cur = cur -> rchild;
    else if( x < cur -> data)
        cur = cur -> lchild;
}
// required node is not present
if( cur == NULL)
{
    printf("Node not found");
    return root;
}
```


x=32
par
cur
psuc
suc
attach
suc->lchild = cur->lchild;
psuc->lchild = suc->rchild;
suc->rchild = cur->rchild;

```
//Node to be deleted is a leafnode
    if( cur->lchild ==NULL && cur->rchild ==NULL)
    {
        if( par->lchild ==cur)
            par->lchild =NULL;
        else
            par->rchild =NULL;
        printf("Node deleted");
        return root;
    }
```

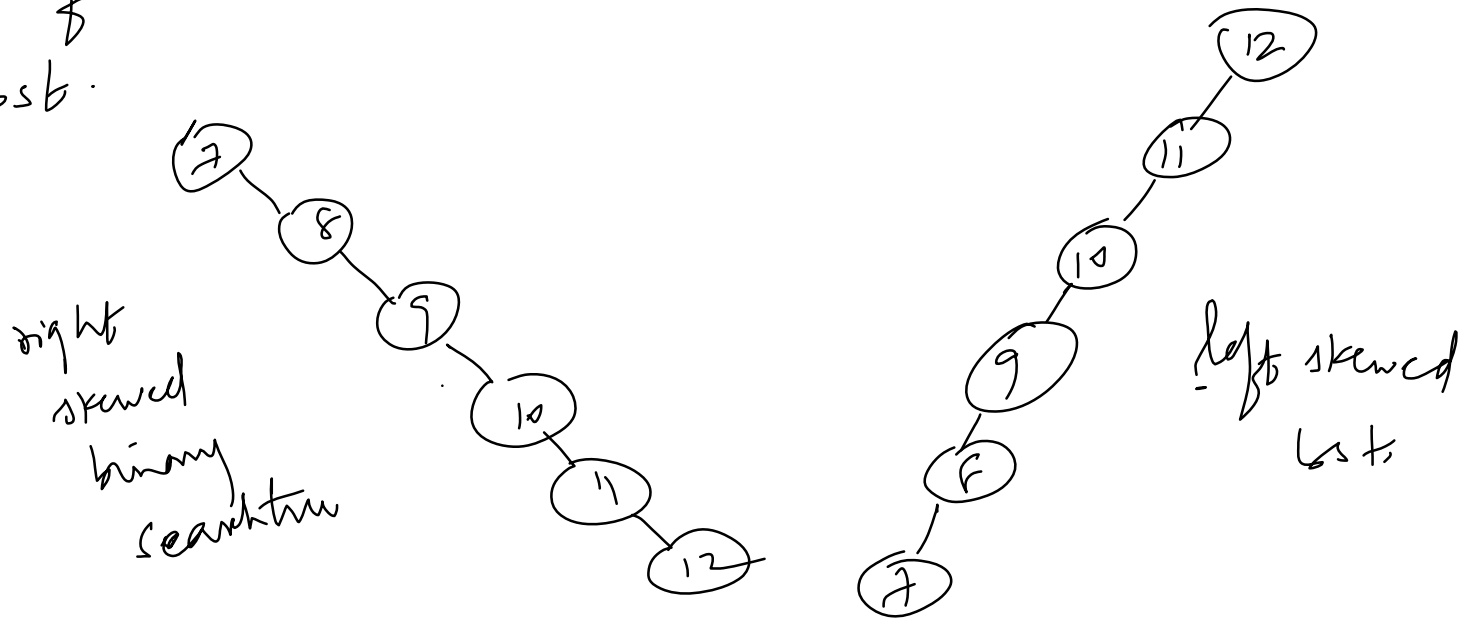
```
if(cur -> rchild == NULL)        // right child is not present
{
   attach = cur -> lchild;
}
else if(cur -> lchild == NULL)   // left child is not present
 {
   attach = cur -> rchild;
 }
else                             // both left & right are present.
 {
    psuc = cur;
    suc = cur -> rchild;
```

```
while (suc->lchild != NULL)
{
    psuc = suc;
    suc = suc->lchild;
}
if( psuc == cm)
{
  suc->lchild = cm->lchild;
}
else
{
  suc->lchild = cm->lchild;
  psuc->lchild = suc->rchild;
  suc->rchild = cm->rchild;
}
attach = suc;
}
```

```
if (root == cur)
{
  root = attach;
}
else if (par -> rchild == cur)
{ par -> rchild = attach;
}
else
{   par -> lchild = attach;
}
printf ("Node deleted");
return root;
}
```

AVL Trees

The problem with Binary Search tree is it takes $O(n)$ units of time for an operation (search, insert, delete) if the bst is a left skewed bst or right skewed bst.



AVL Trees :- Height Balanced Binary Search trees where the operations (insert, delete, search) take $O(\log_2 n)$ units of time.

Adelson - Velskii - Landis

AVL tree is based on the concept of balancing factor of a node

Balancing factor = height of left subtree of the node
- height of right subtree of the node

If every node's balancing factor is between (inclusive) -1 and $+1$ $[-1 \leq bf \leq +1]$ then we can say the tree is balanced and an AVL tree.

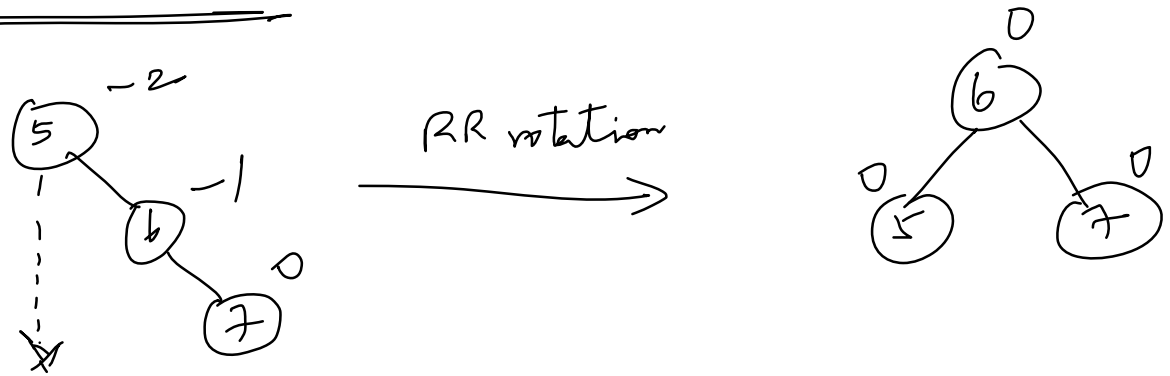
If the balancing factor of a node is < -1 or $> +1$ we must perform rotations wrt the node in order to convert the tree into an AVL tree

. We have four types of rotations

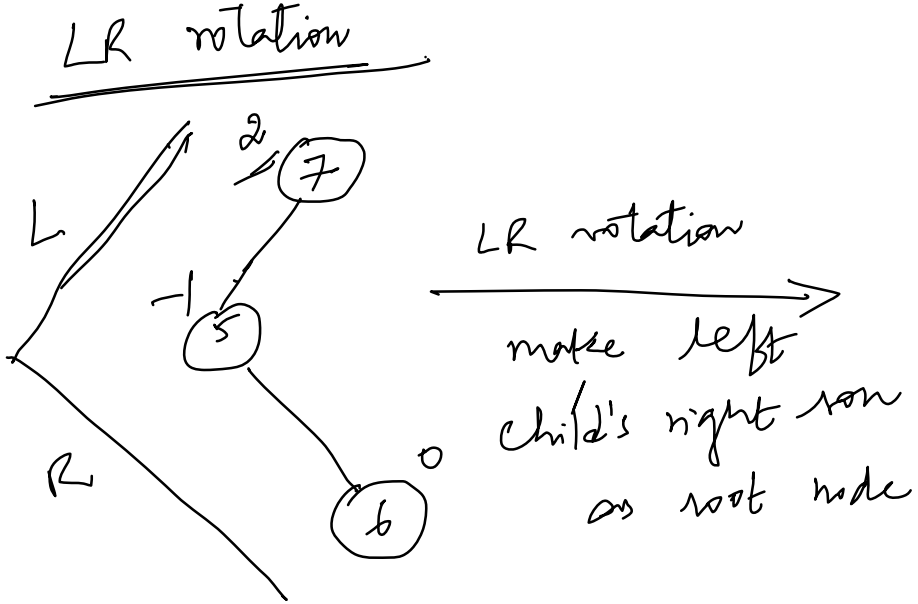
1) LL rotation



2) RR rotation

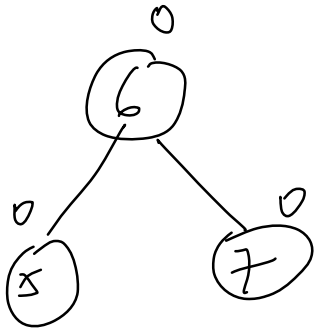


LR rotation

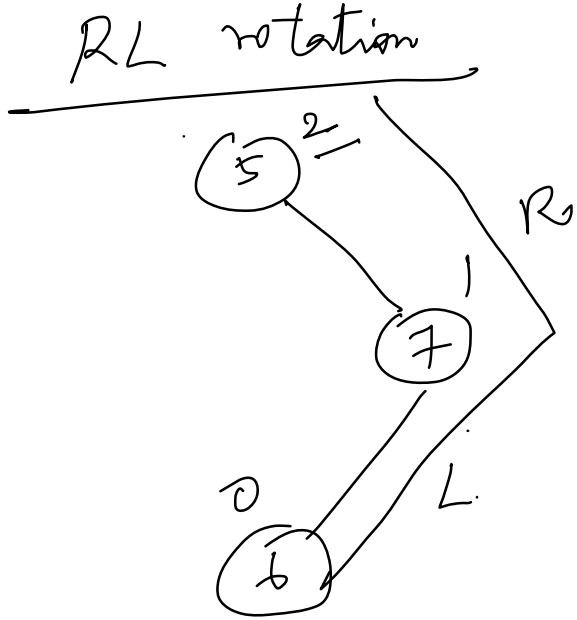


LR rotation

make left child's right son as root node

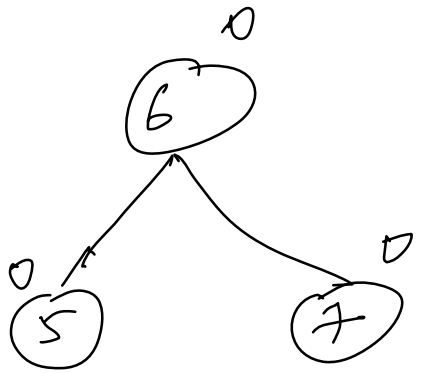


RL rotation



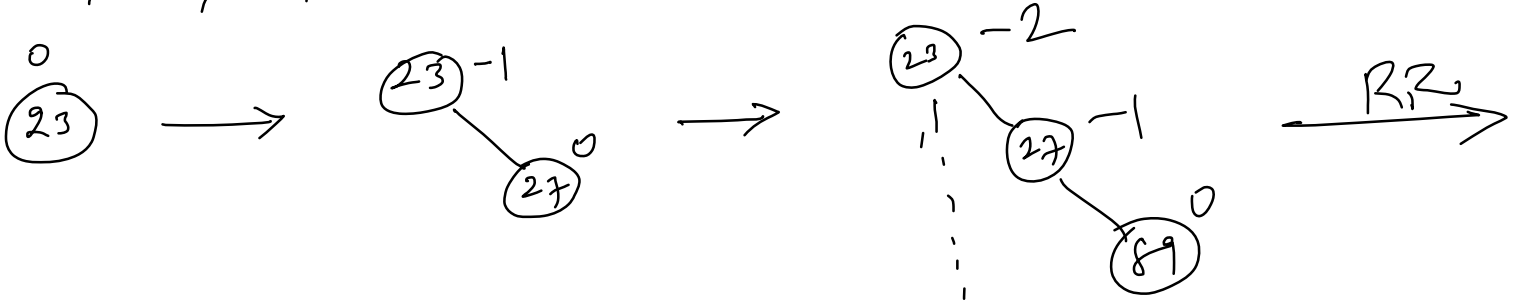
RL rotation

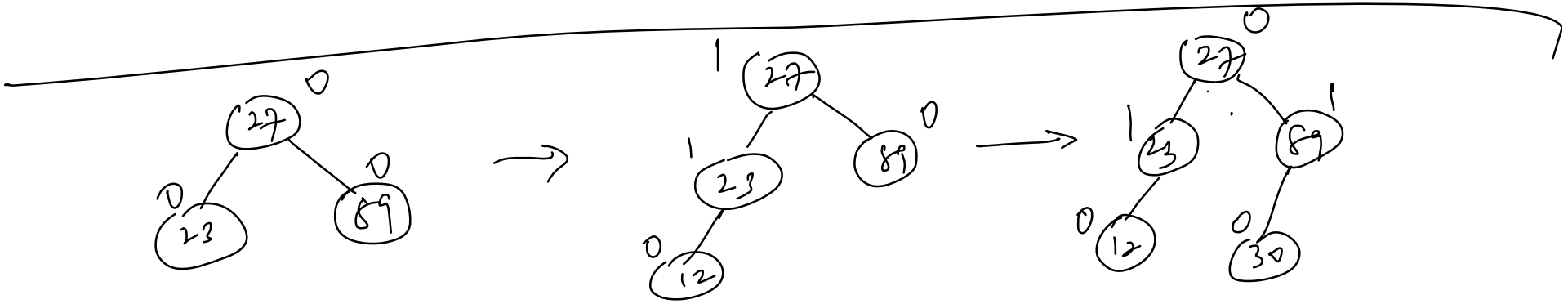
make right child's left son as root node

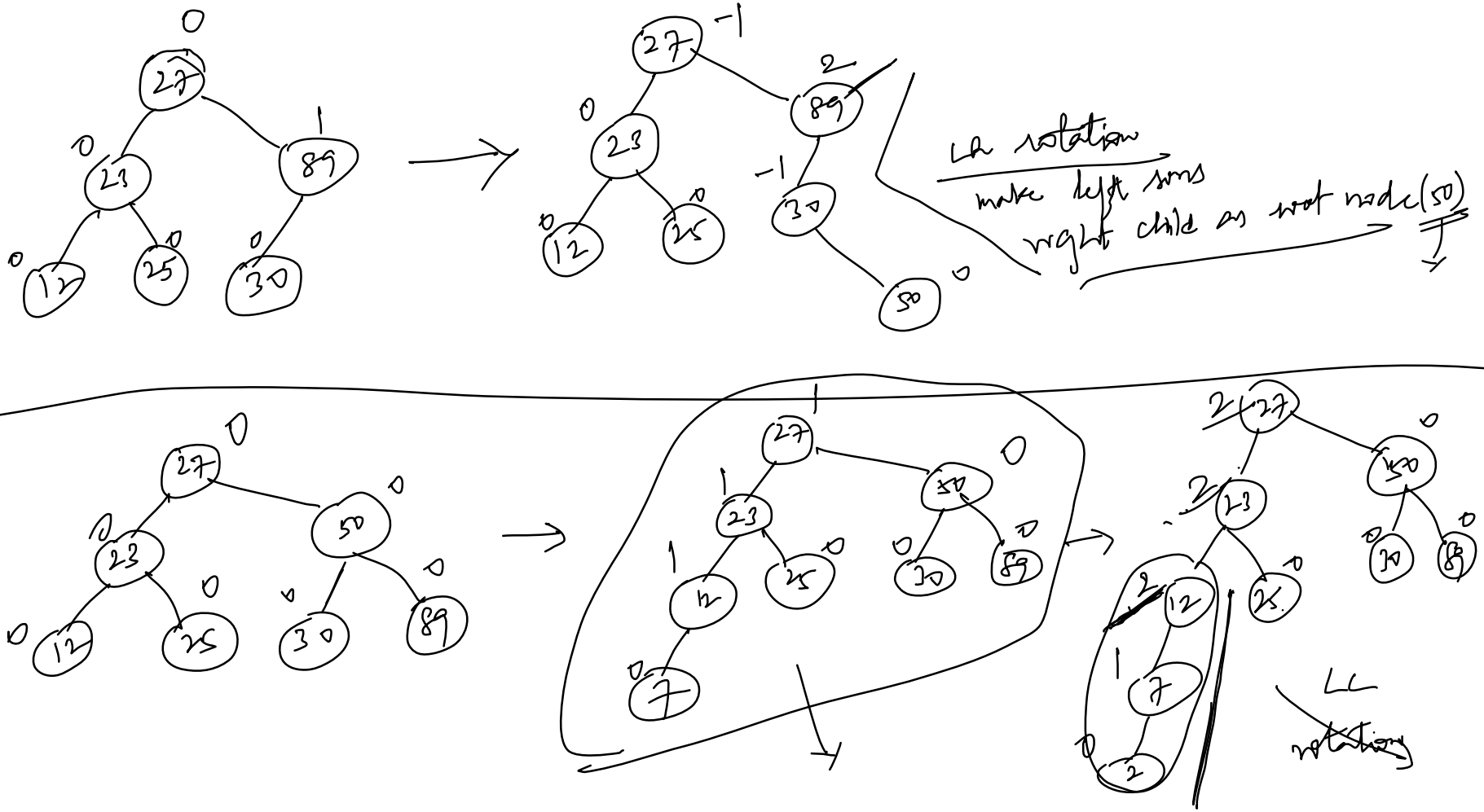


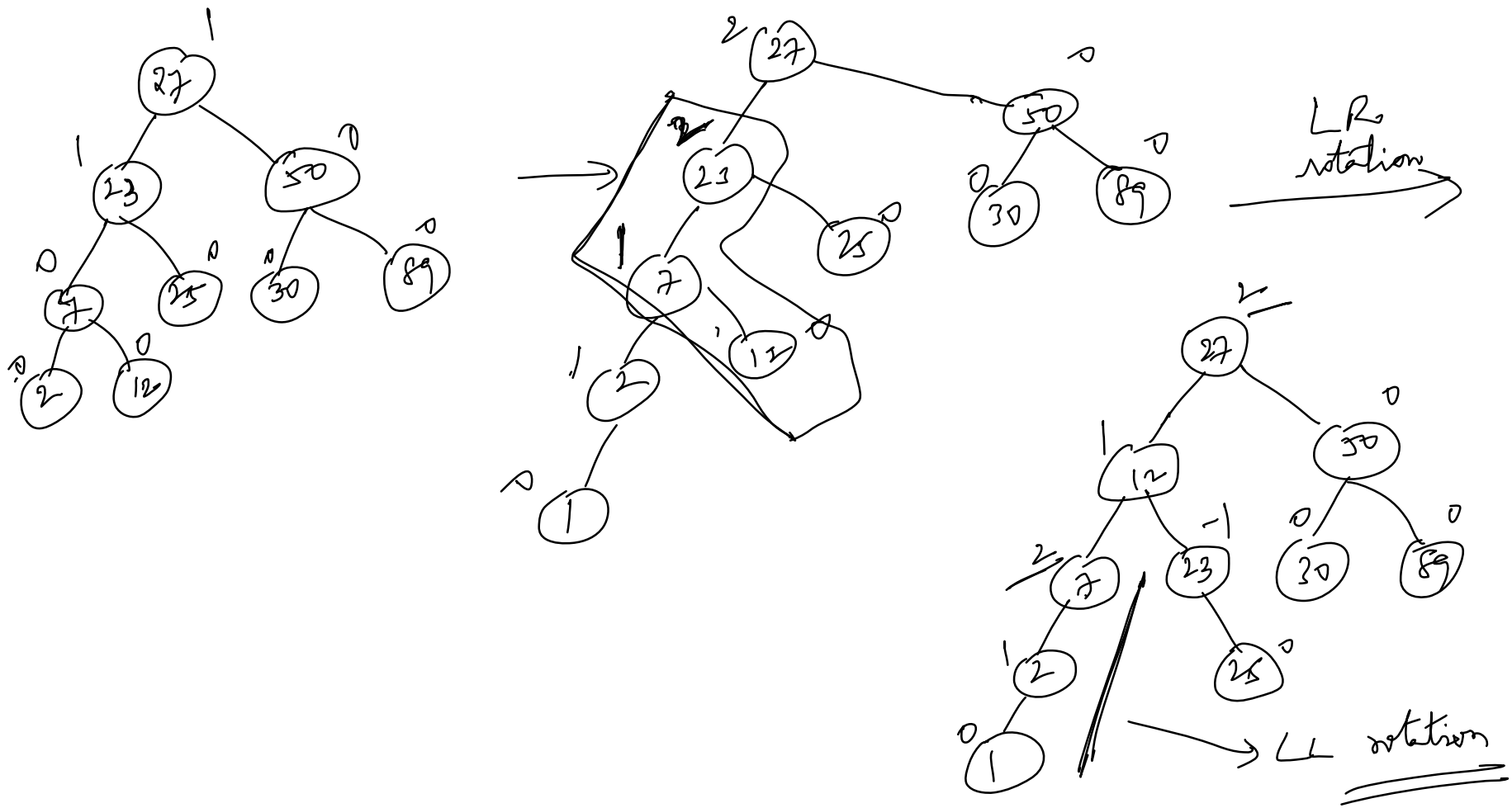
Construct an AVL tree for the following elements

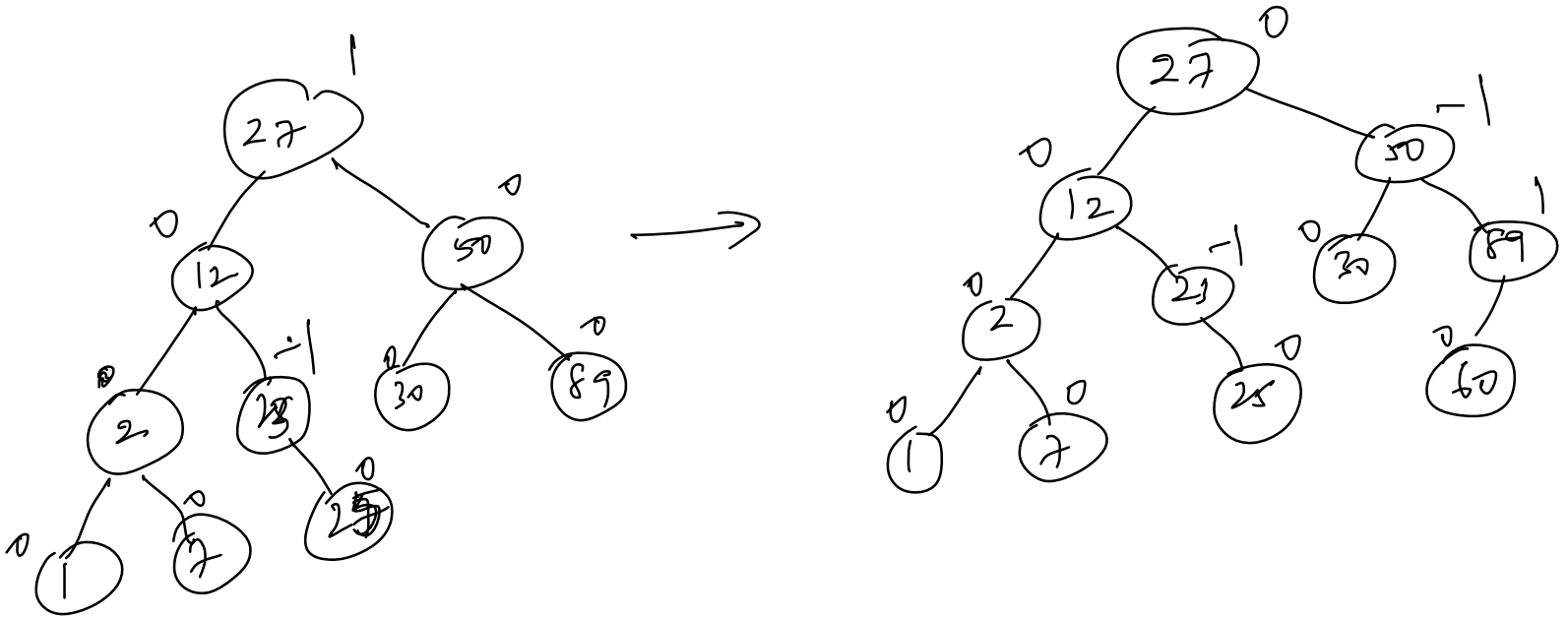
23, 27, 89, 12, 30, 25, 50, 7, 2, 1, 60, 10

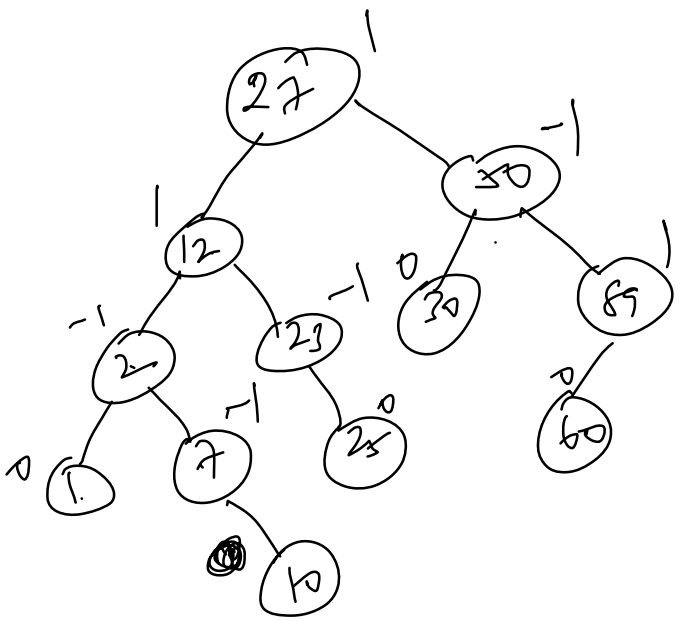





LR rotation
make left sms
right child as root node(50)
LL
rotation


27
23
50
7
25
30
89
2
12
1
LR
rotation
LL
rotation

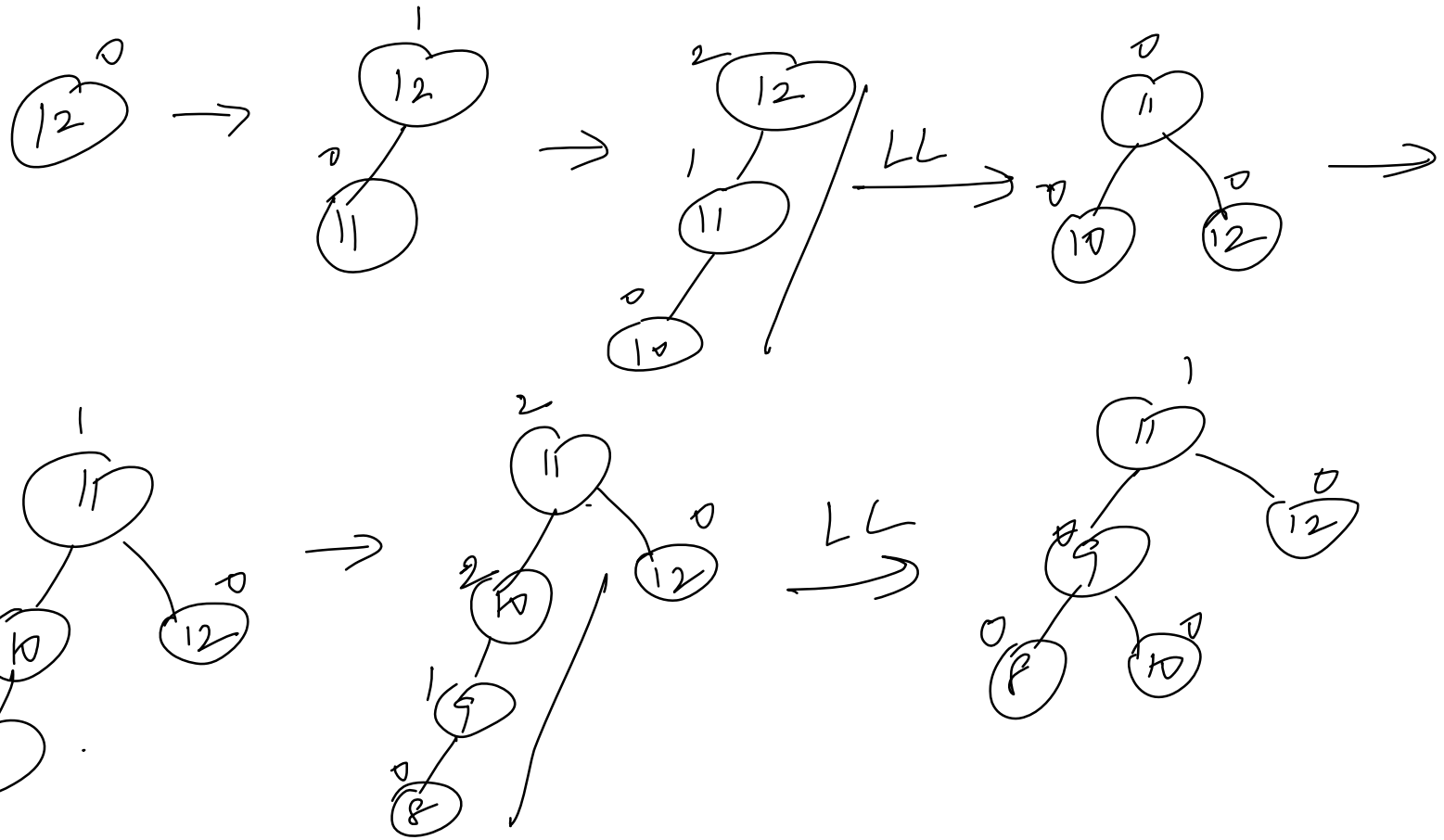

27
12
50
2
25
30
89
1
7
25

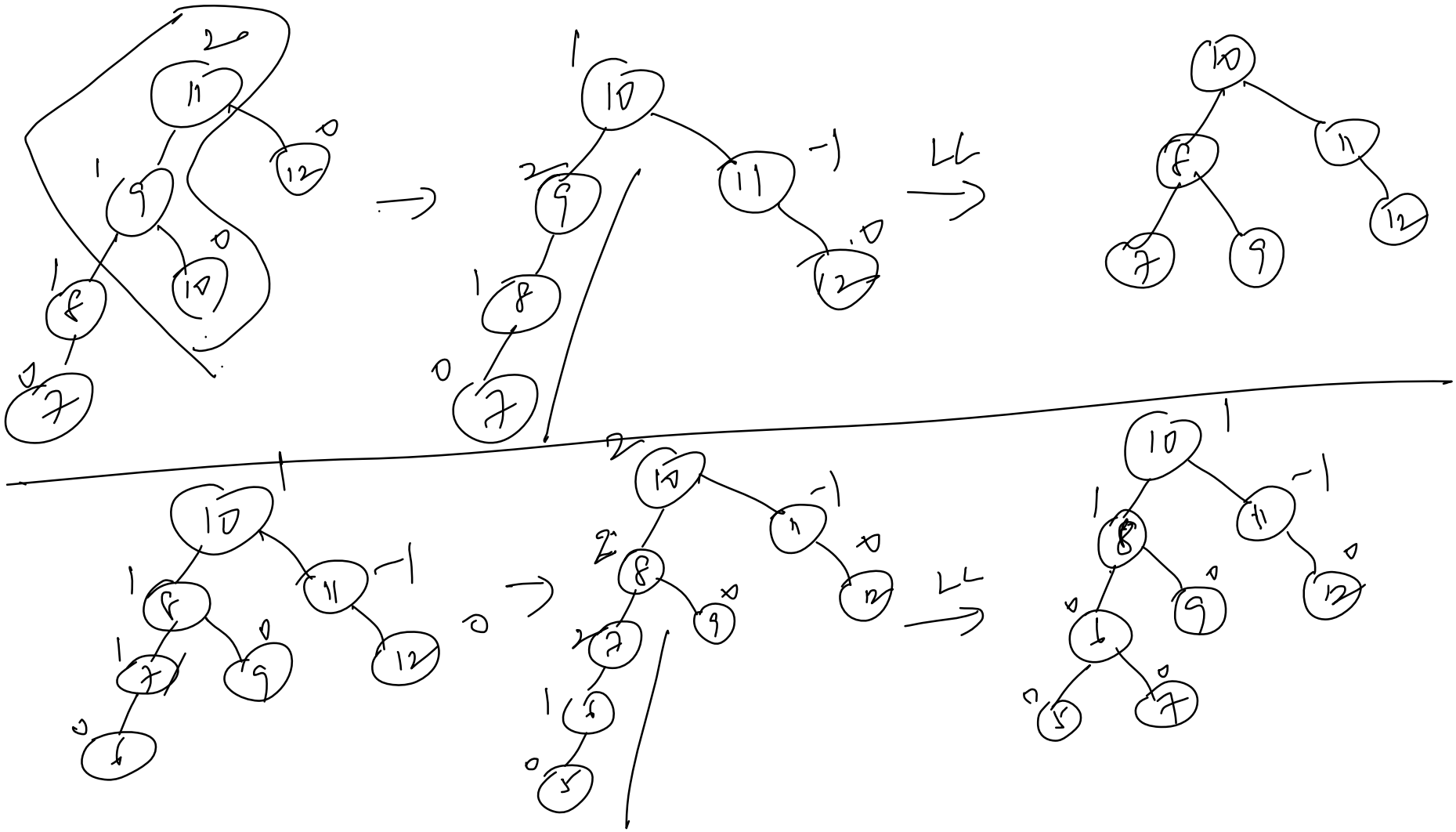


Construct AVL tree for following elements

a) 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

b) 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1


12
11
12
11
10
LL
11
10
12
11
10
9
12
11
10
9
8
12
LL
11
9
12
8
10


LL
LL

Red Black Trees:- It is a binary search tree where every node of the bst is colored either as red or black.

Properties wrt nodes

RB1:- Root node and external nodes (null pointers) are colored black

RB2:- No root node to external node path has 2 consecutive red nodes.

RB3:- All root to external node paths have same number of black nodes.

Properties w.r.t pointers/edges

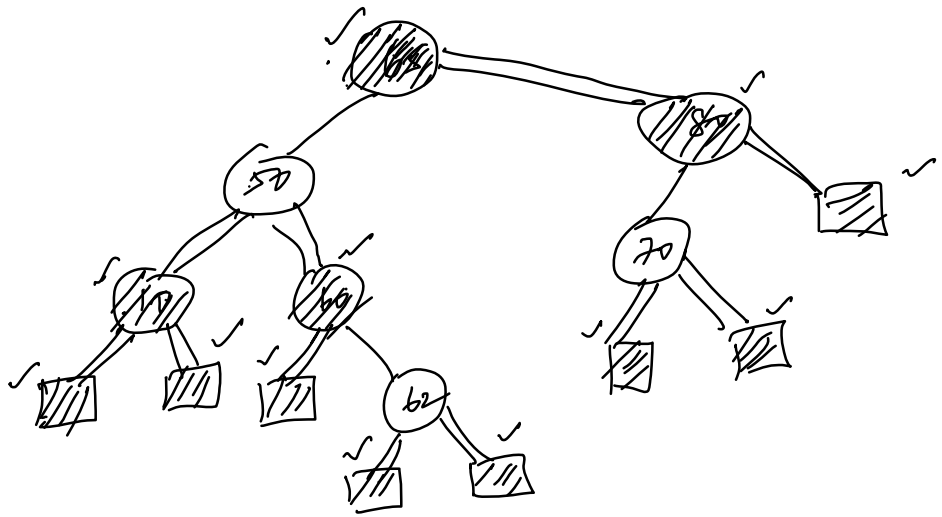
RB1' :- A red node has a red pointer from its parent and a black node has a pointer from its parent. Pointer from internal to external node must be black.

RB2' :- No root to external node path has 2 consecutive red pointers.

RB3' :- All root to external node paths must have same number of black pointers.

Fig 1 → red node → black node → black pointer → red pointer.

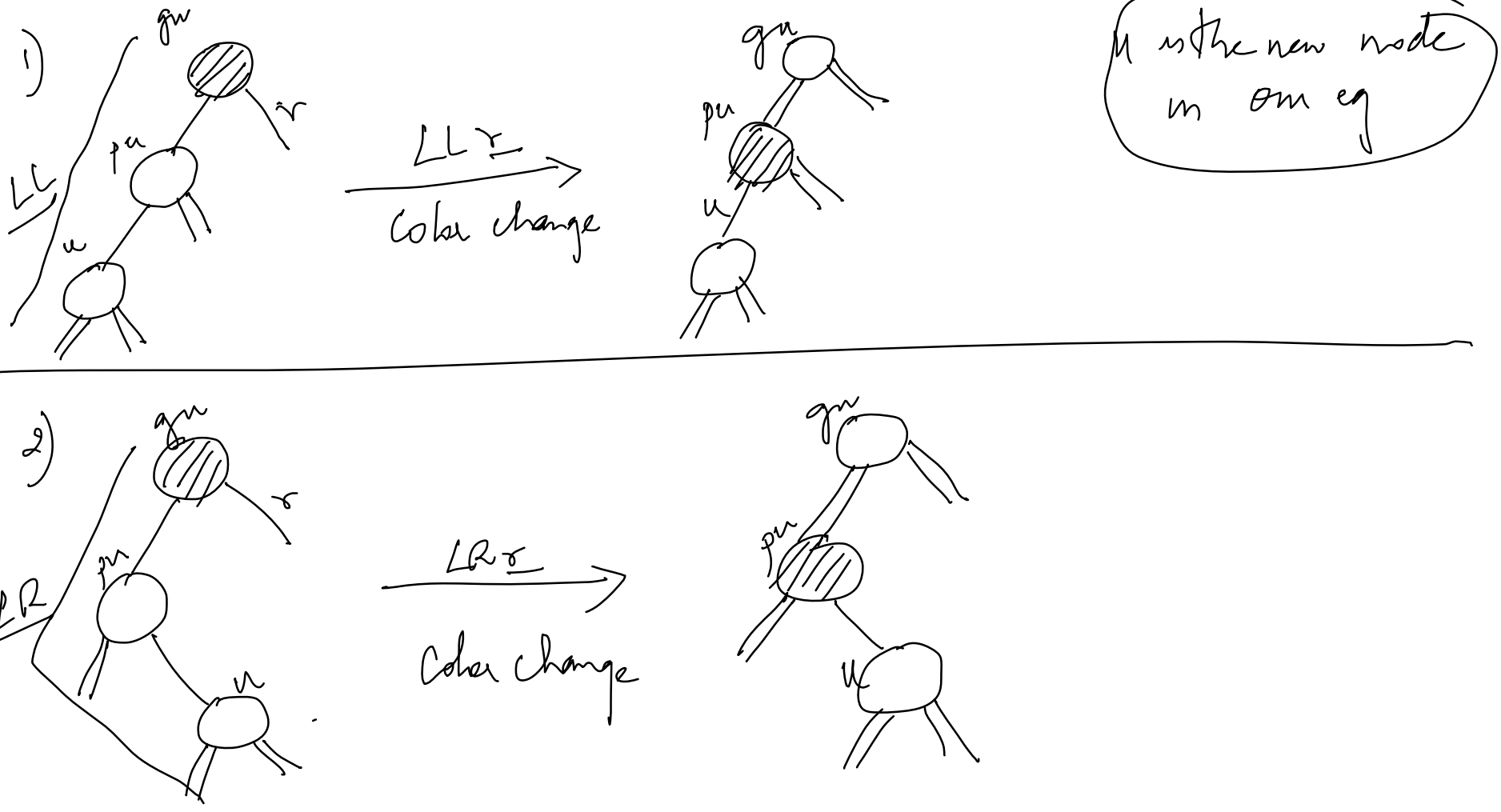
→ external nodes

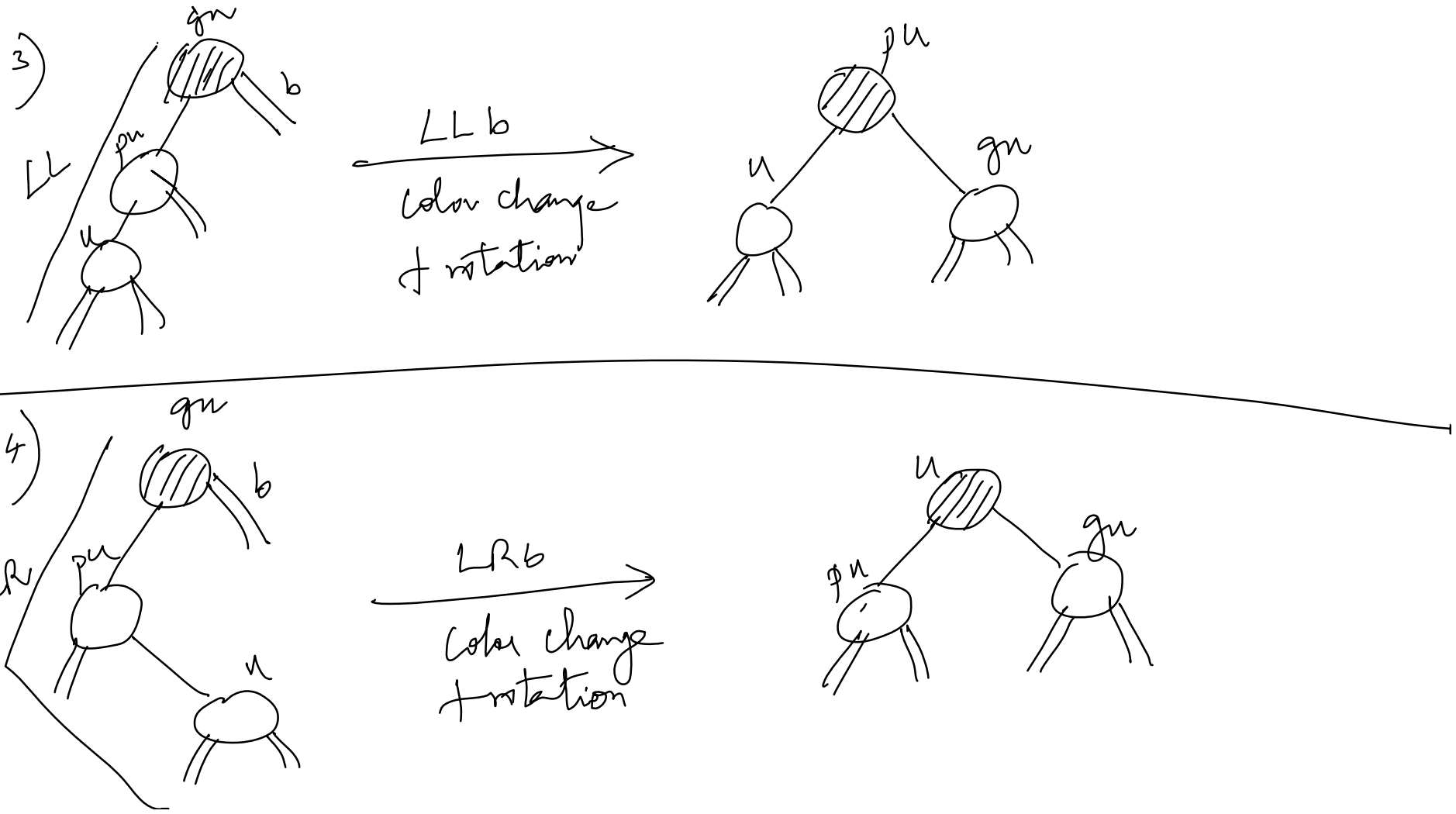


Construction :- If we insert a black node RB3 & RB3' will be violated which cannot be solved

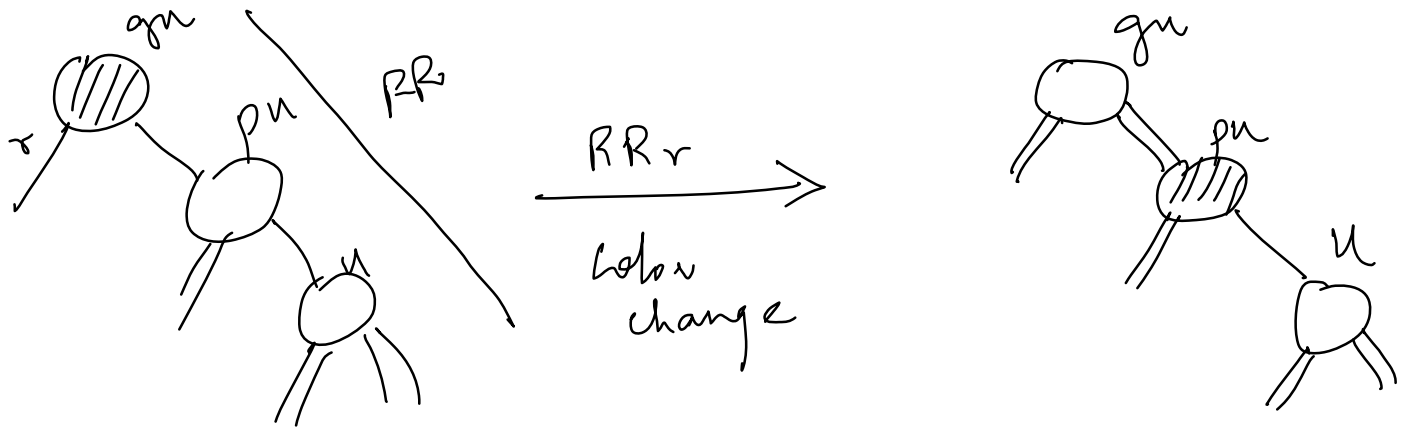
Hence we insert a red node which might violate RB2 & RB2'

If RB2 and RB2' are violated we perform color change or color change with rotations in order to convert the tree into a Red Black Tree.

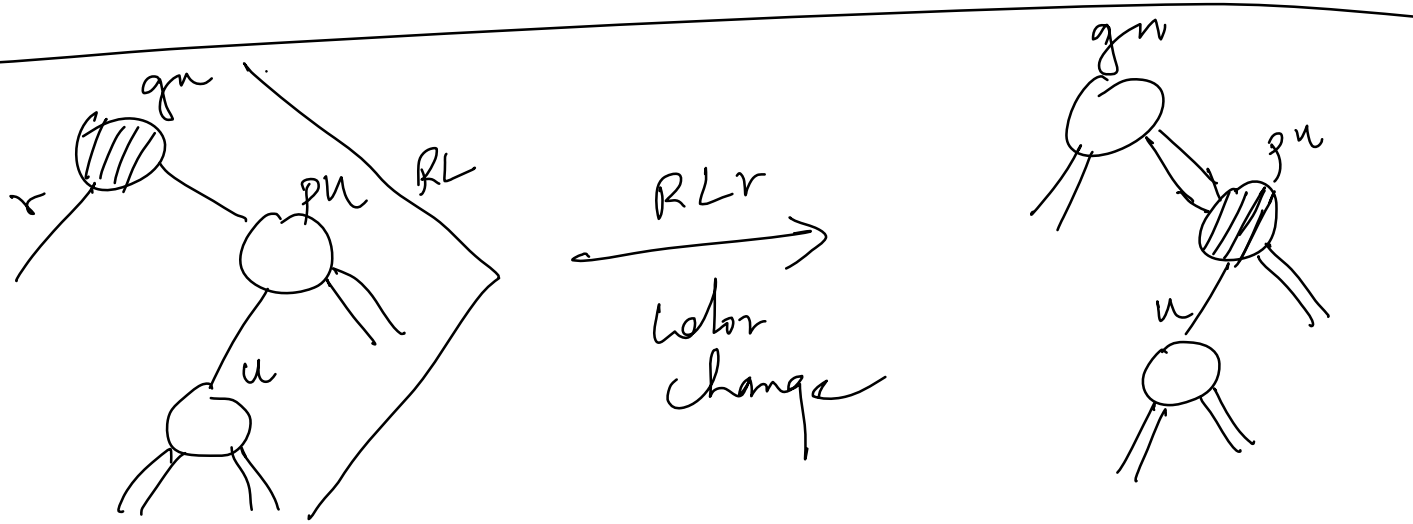

1)
gn
r
pn
LL
u
LL 회전
Color change
gn
pn
u
u is the new node in our eq
2)
gn
r
pn
LR
u
LR 회전
Color change
gn
pn
u


3)
gn
b
LL
pn
u
LL b
color change
+ rotation
pn
u
gn
4)
gn
b
LR
pn
u
LR b
color change
+ rotation
u
pn
gn

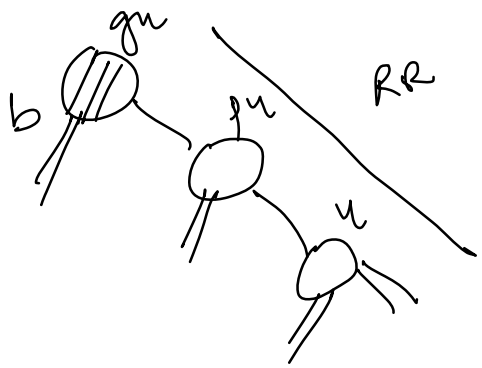
5)


gn
r
pn
u
RR
RRr
Color
change
gn
pn
u

6)

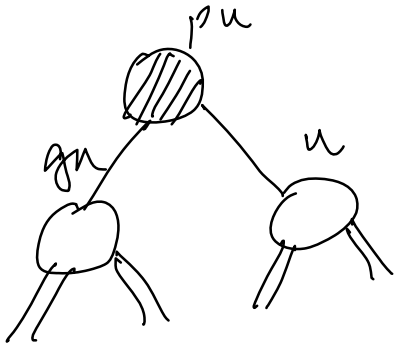

gn
r
pn
u
RL
RLr
Color
change
gn
pn
u

7)

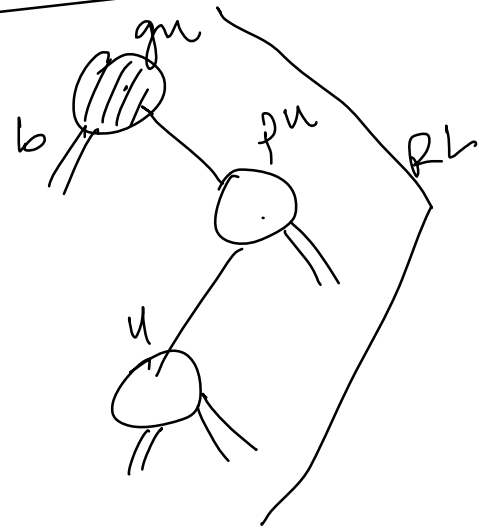


RRb

color change
rotation

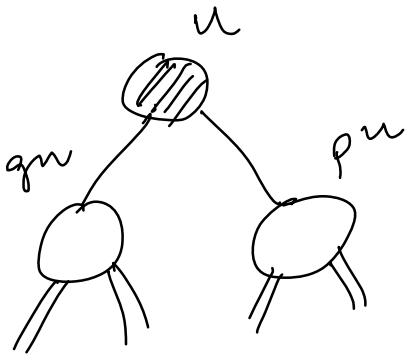


8)

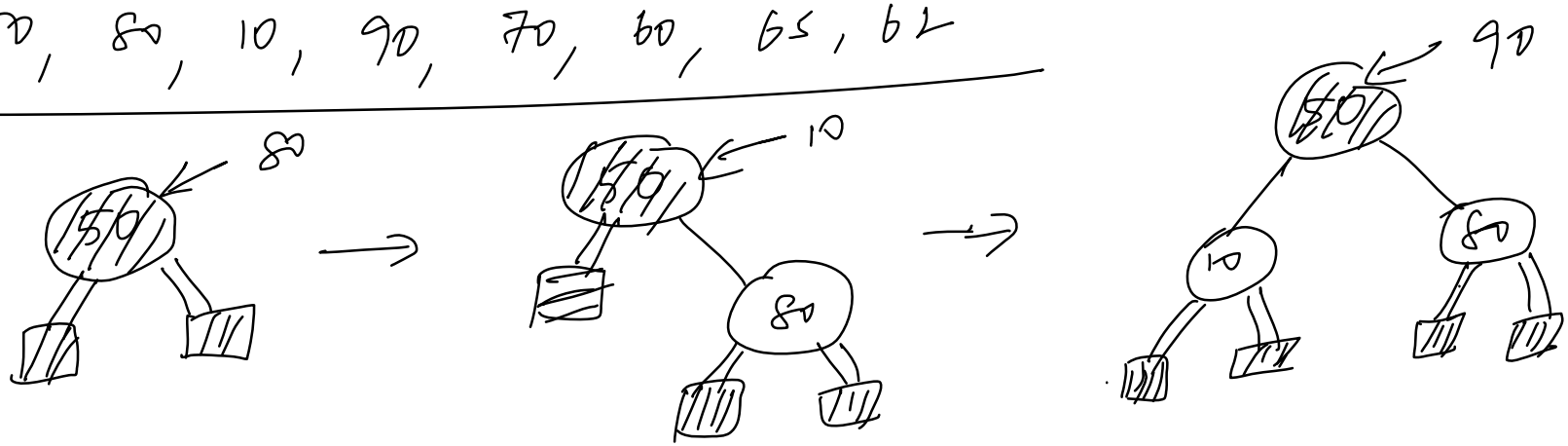


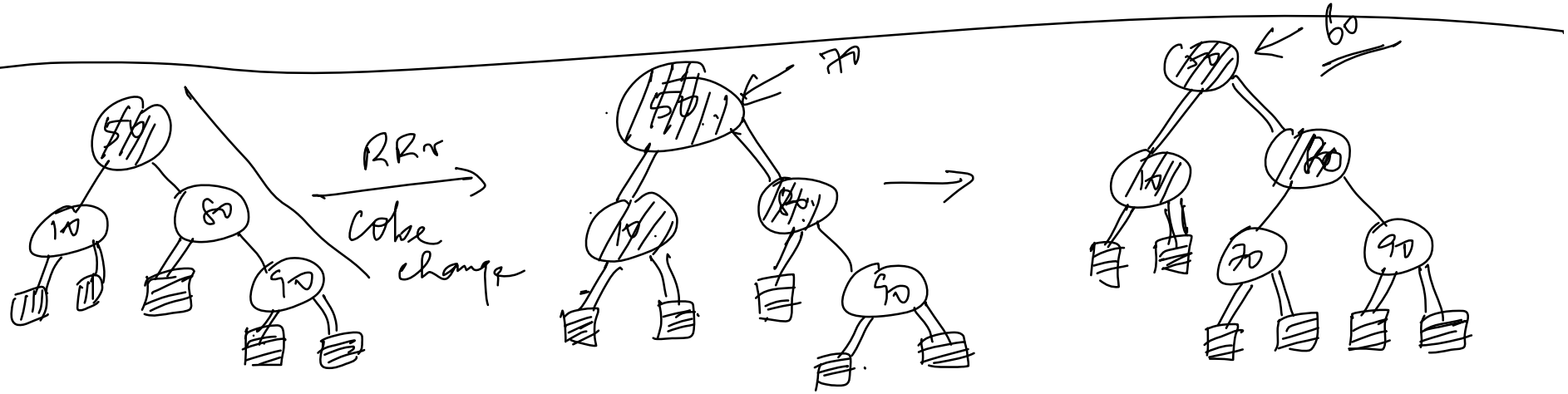
RLb

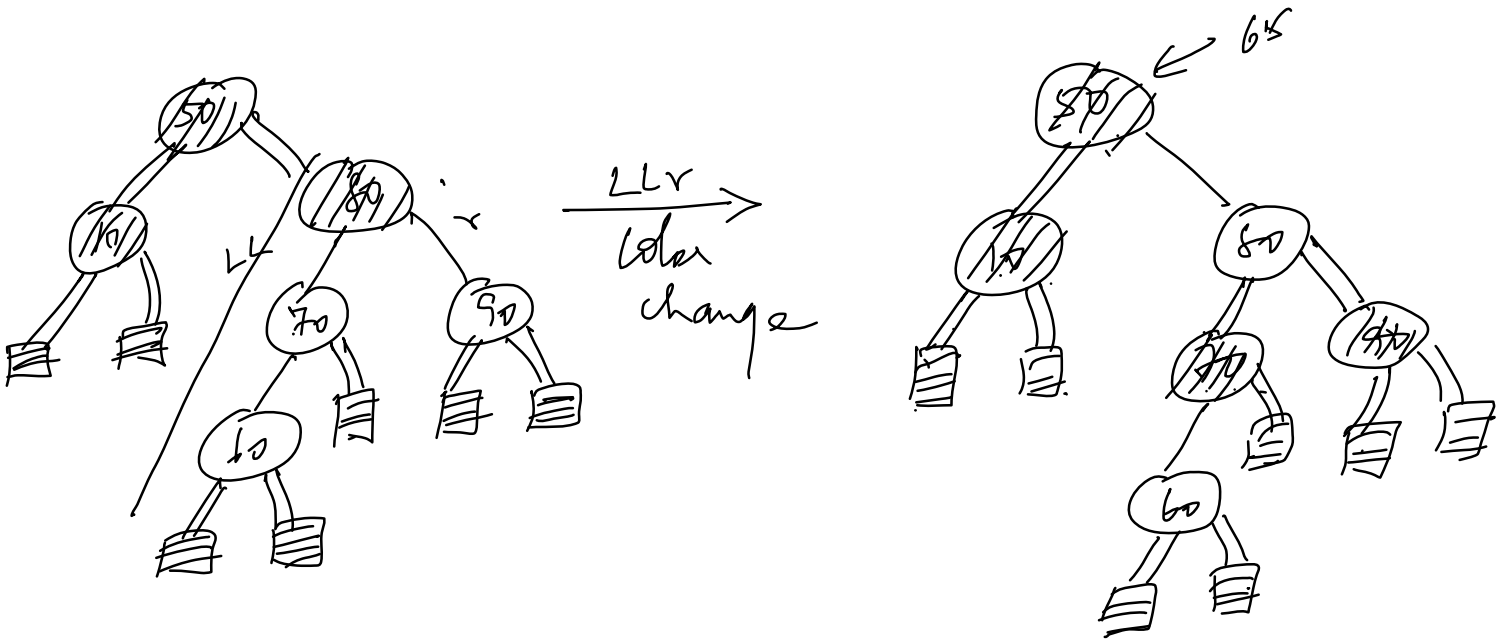
color change
rotation



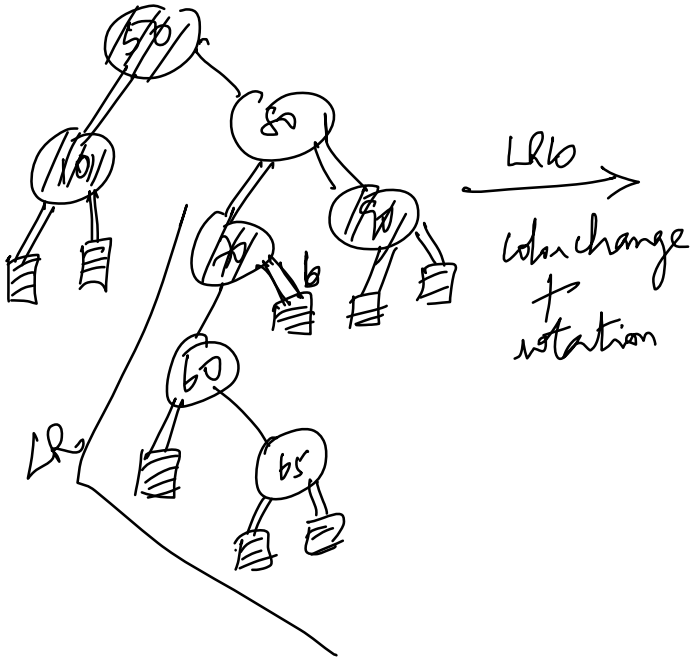
50, 80, 10, 90, 70, 60, 65, 62

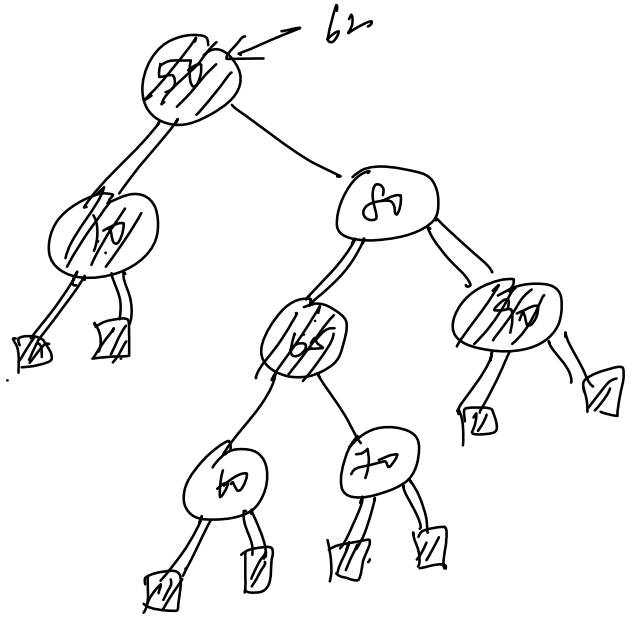


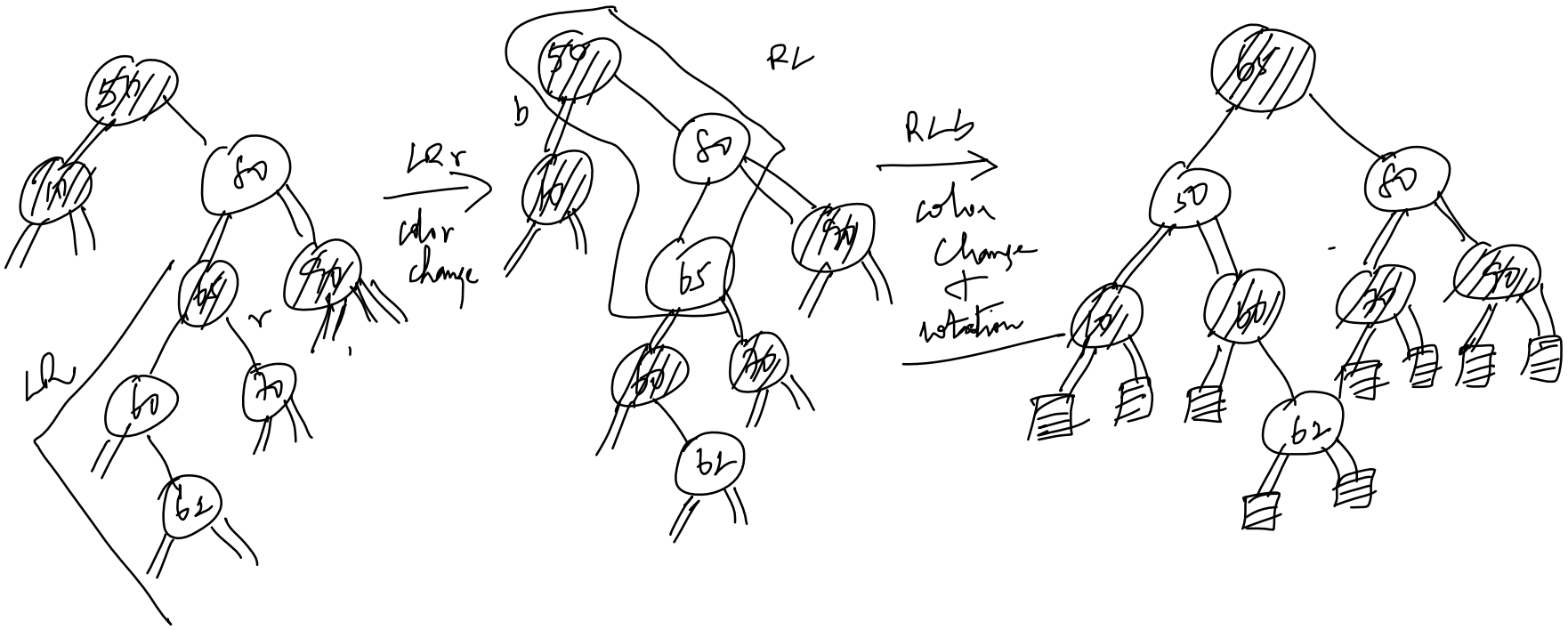



50
10
80
70
90
10
LL
r
LLr
color
change

65
50
10
80
70
90
10


LR(b)
color change
+
rotation


b


LR
r
color change
b
RL
RL
color change + rotation

B-Trees:- It is an m-way search tree where the time complexity to search a value in the tree is $O(\log_m n)$ where n is the number of values, m is the order of the tree.

Every node of B-Tree will be as follows

max no: of pointers → m
max no: of keys/values → m−1
} → construction

min no: of pointers → $\lceil \frac{m}{2} \rceil$
min no: of keys/values → $\lceil \frac{m}{2} \rceil - 1$
} → deletion

In B-tree all the leaf nodes are at the same level.

Balanced m- way search tree.

Construct a B-tree of order 5 for the following elements

2, 12, 22, 5, 15, 25, 6, 16, 26, 1, 11, 21, 7, 17, 27, 28, 29, 30

node

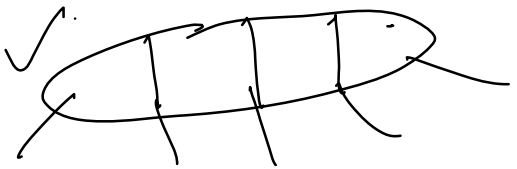
max no: of pointers = 5

max no: of keys/values = 4

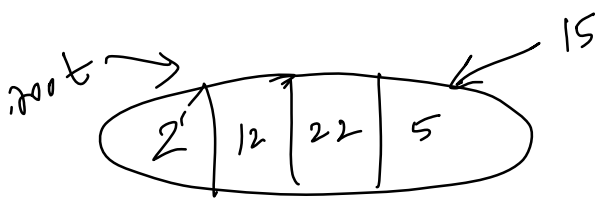
min no: of pointers = $\lceil \frac{5}{2} \rceil = 3$

min no: of keys/values = 3 - 1 = 2

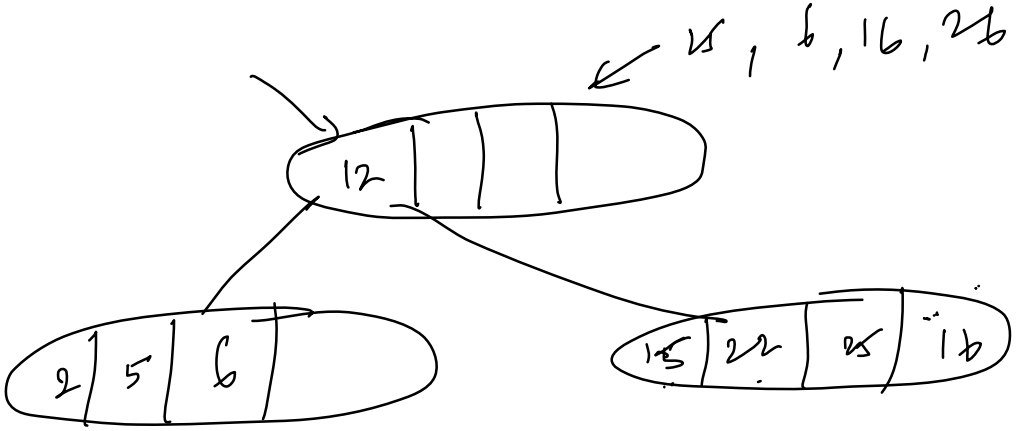




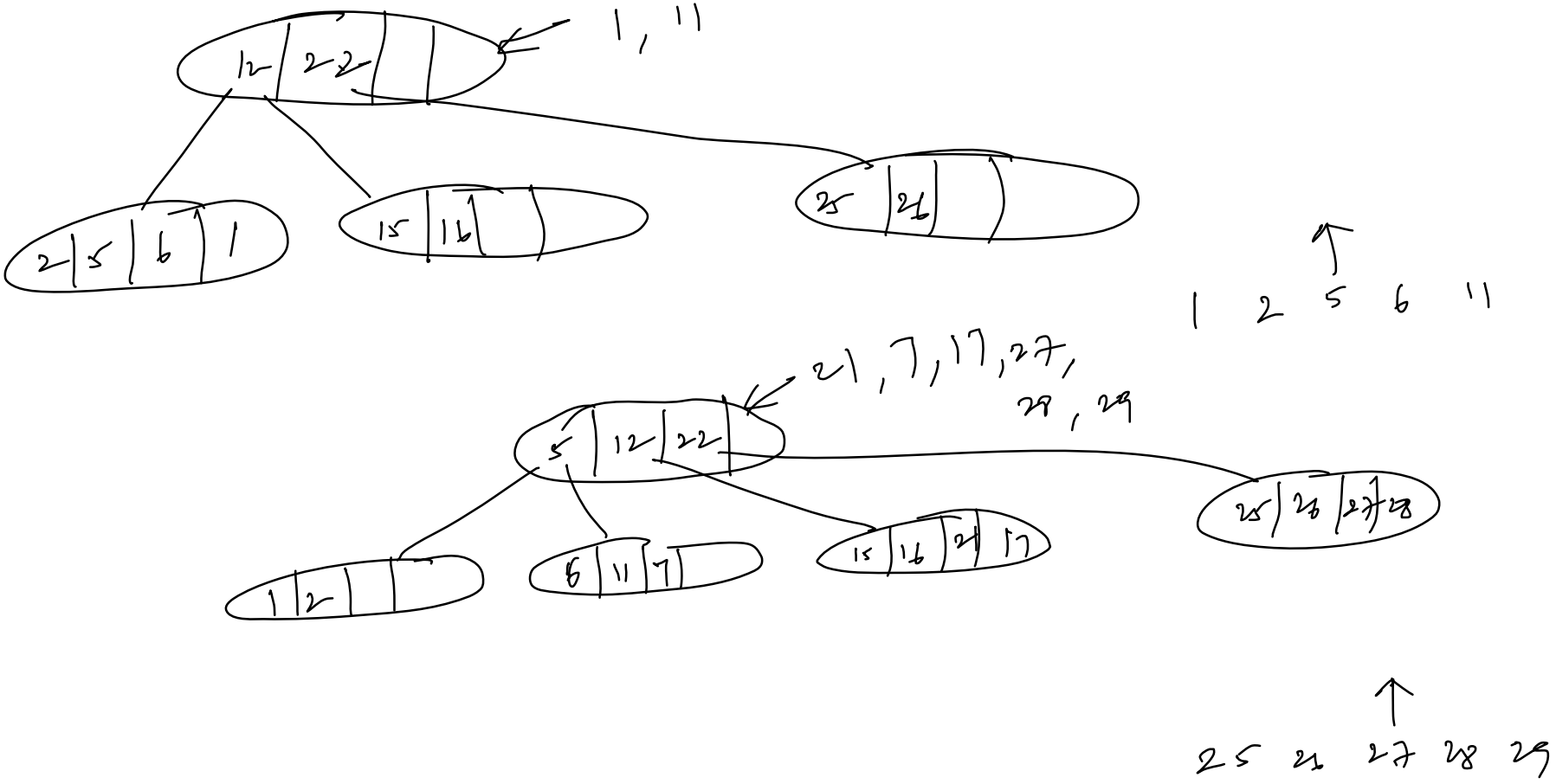
2, 12, 22, 5, 15, 25, 6, 16, 26, 1, 11, 21, 7, 17, 27, 28, 29, 30, 18, 18

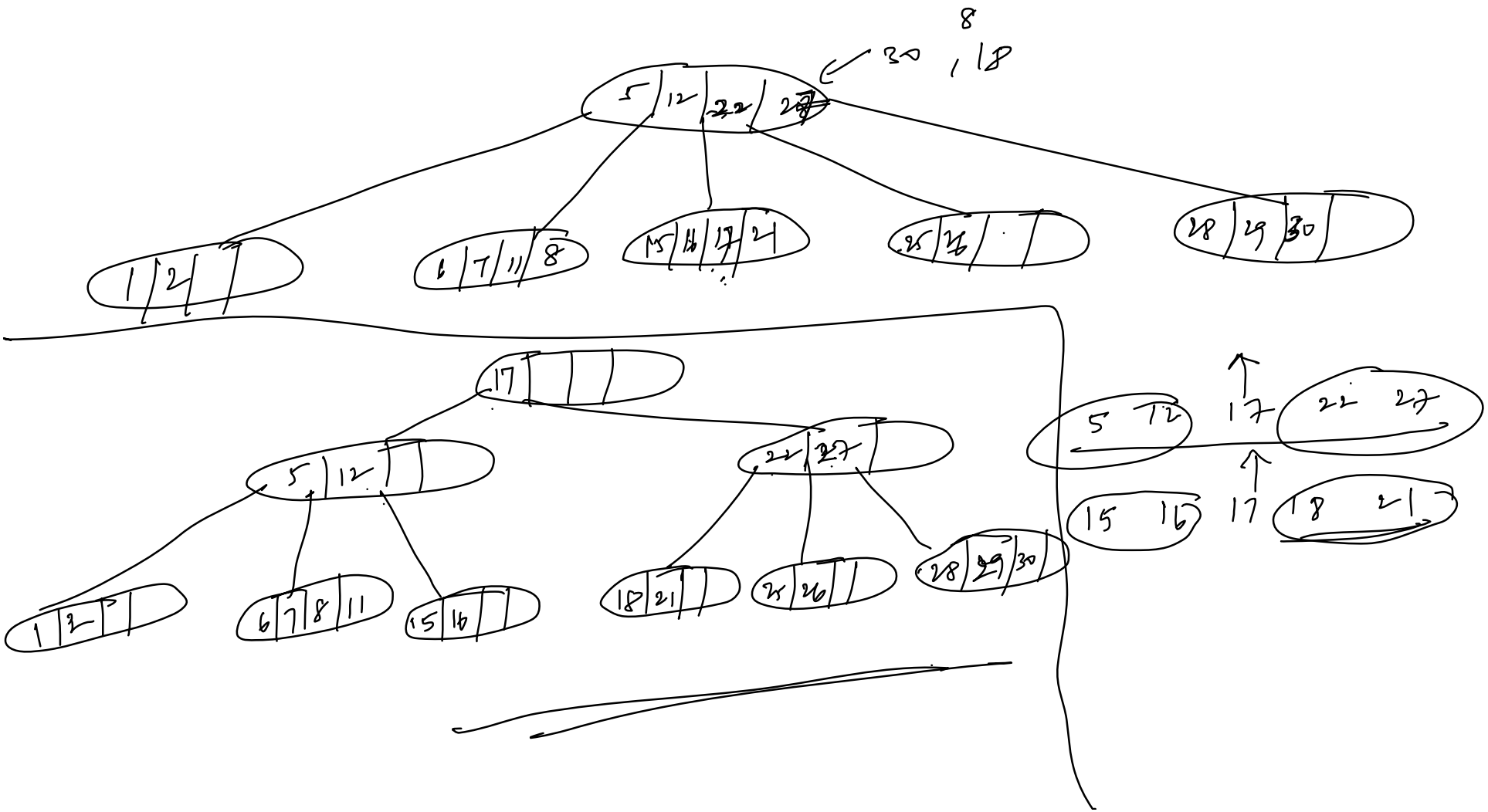


2 5 12 15 22



15 16 22 25 26


1, 11
12 22
2 5 6 1
15 16
25 21
1 2 5 6 11
21, 7, 17, 27, 29, 29
5 12 22
1 2
6 11 7
15 16 21 17
25 26 27 28
25 21 27 28 29


8
30 , 18
5 12 22 27
1 2
6 7 11 8
15 16 17 21
25 26
28 29 30
17
5 12
22 27
1 2
6 7 8 11
15 16
18 21
25 26
28 29 30
5 12
17
22 27
15 16
17
8 21

Construct a B-tree of order 6 for the following elements

2, 12, 22, 5, 15, 25, 1, 11, 21, 6, 16, 26, 4, 14, 24, 7, 17, 27, 28, 29, 30

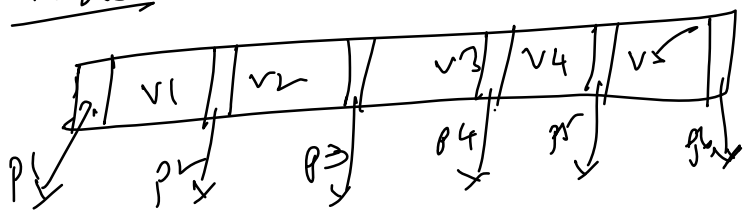
max no: of pointers = 6

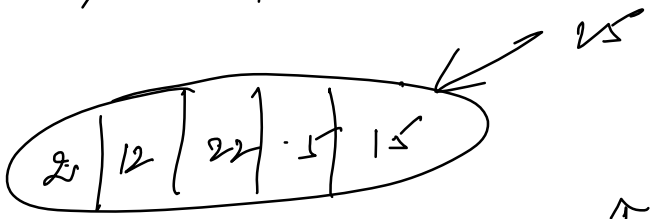
max no. of keys = 5

min no: of pointers = $\lceil \frac{6}{2} \rceil = 3$

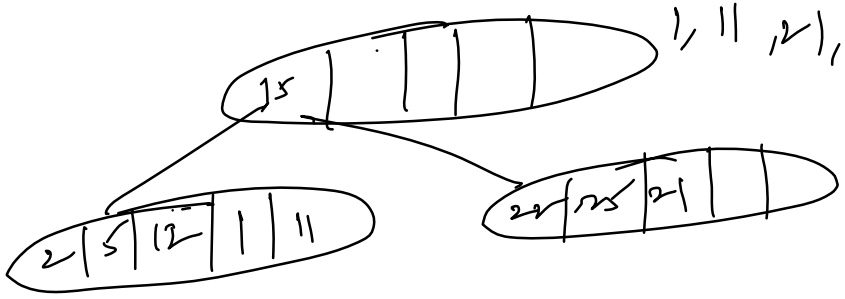
min no: of keys = $\lceil \frac{6}{2} \rceil - 1 = 2$

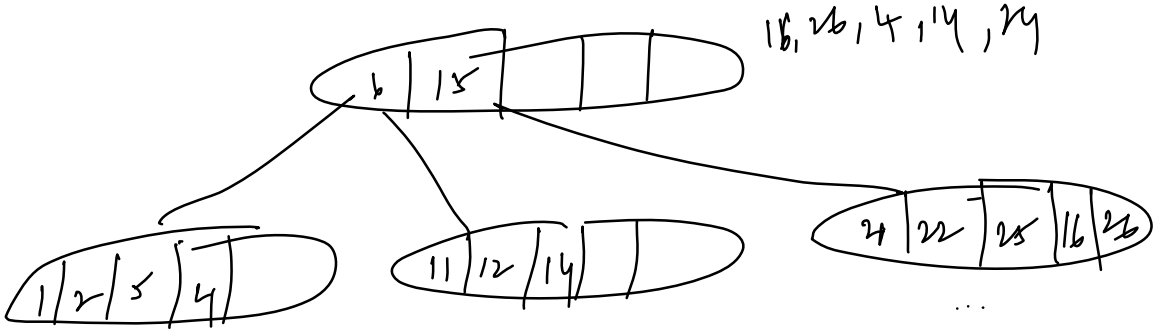
Node





2 5 12 15 22 25

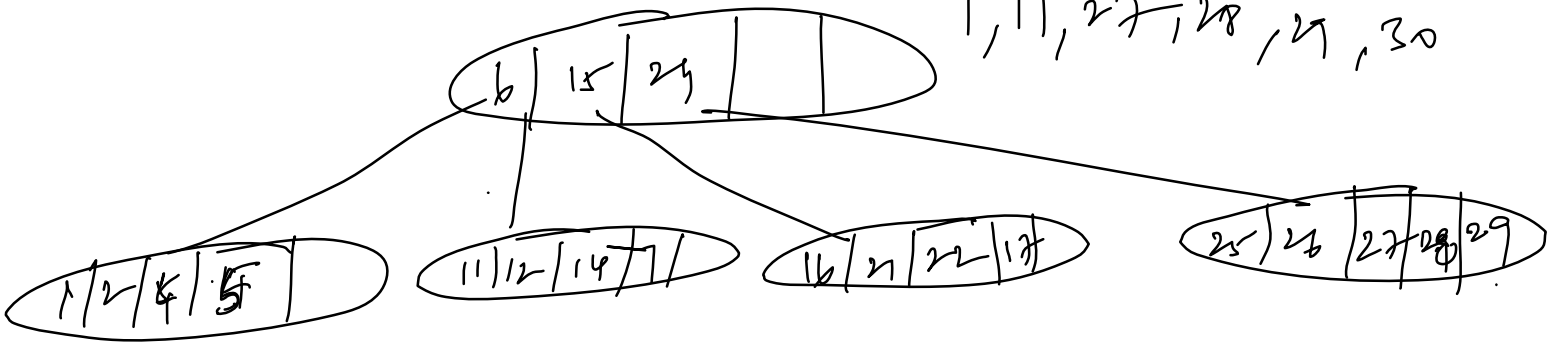




16, 26, 4, 14, 24

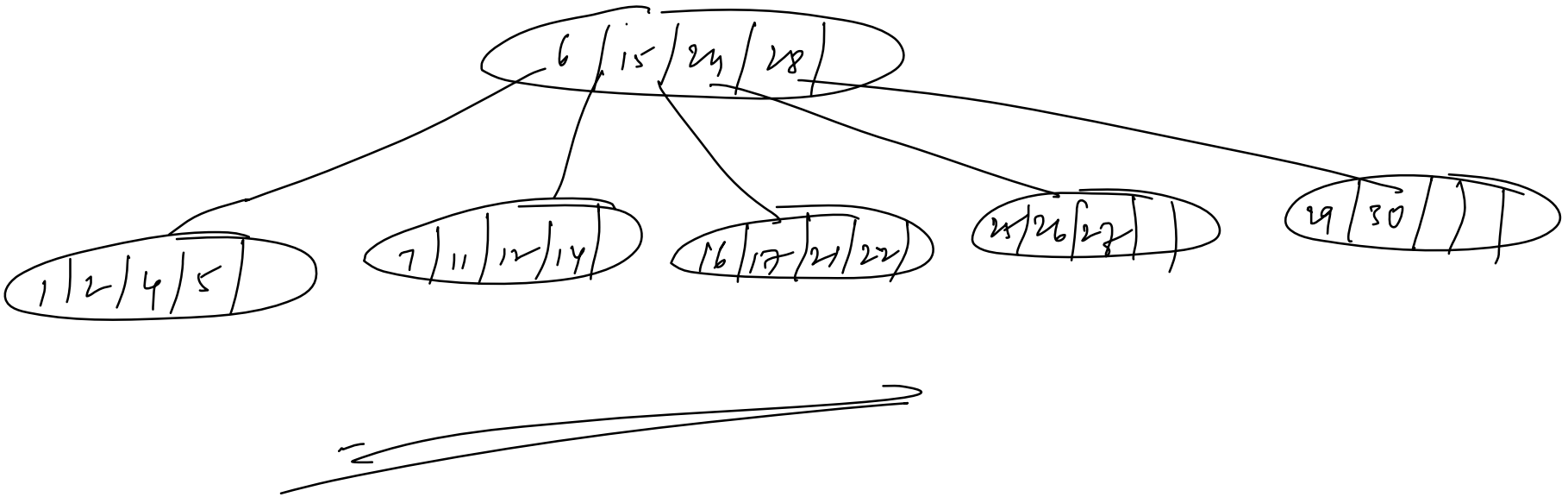
1 2 5 6 11 12

16, 21, 22, 24, 25, 26



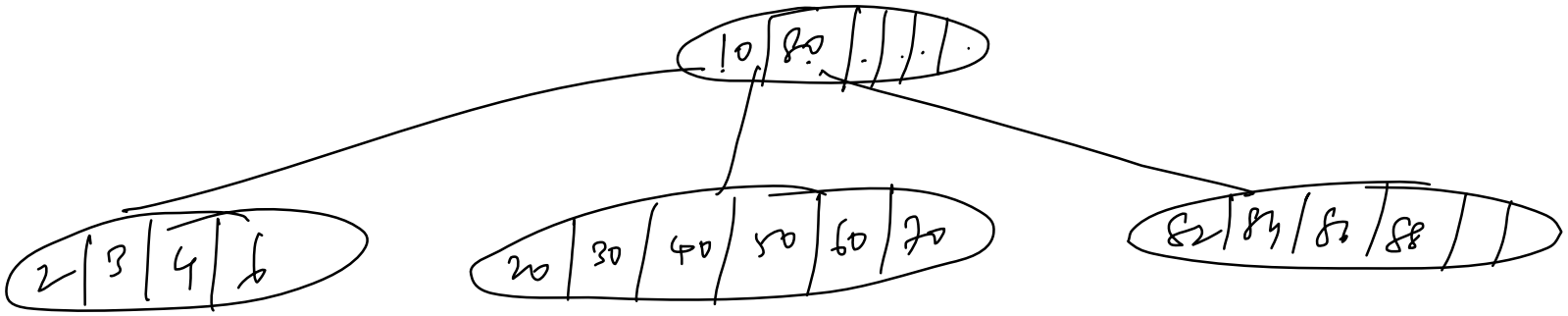
7, 17, 27, 28, 29, 30

25 26 27 28 29 30

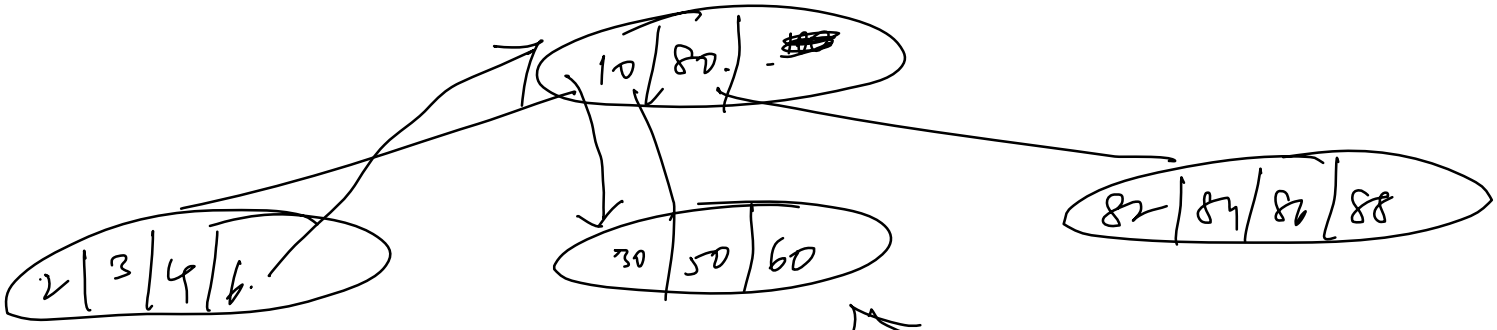

6 15 24 28
1 2 4 5
7 11 12 14
16 17 21 22
25 26 27
29 30

Deletion order = 7 maxptrs = 7, max keys = 6

minptrs = $\lceil \frac{7}{2} \rceil = 4$ min keys = 4 − 1 = 3



delete 40, 20, 70

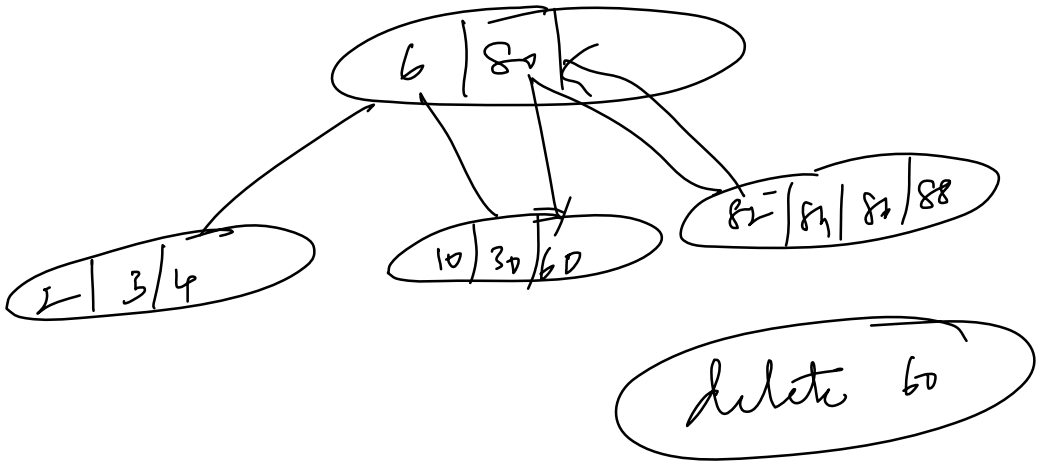


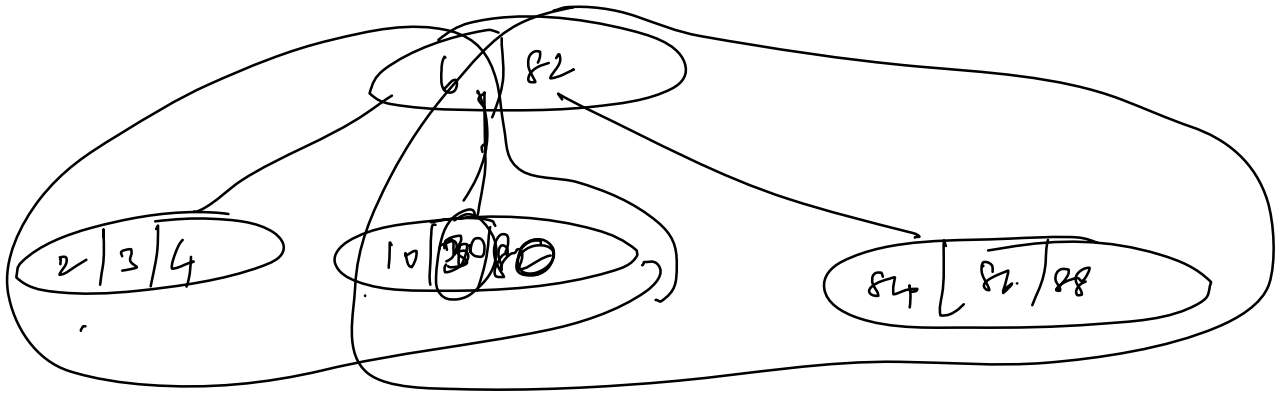
delete 50

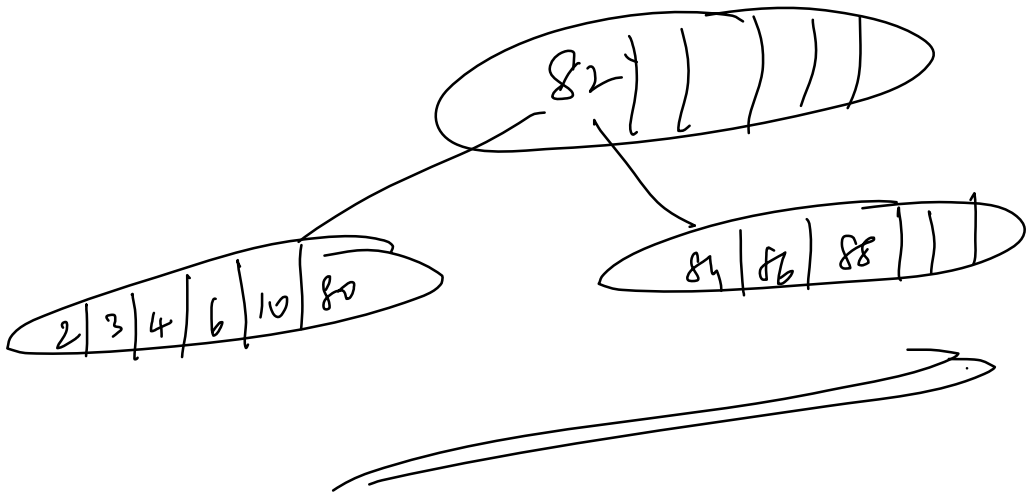
min no of keys property is violated.

get the max value from its left node to root & value into the node

get (or) min value from its right node and into root value from root to node

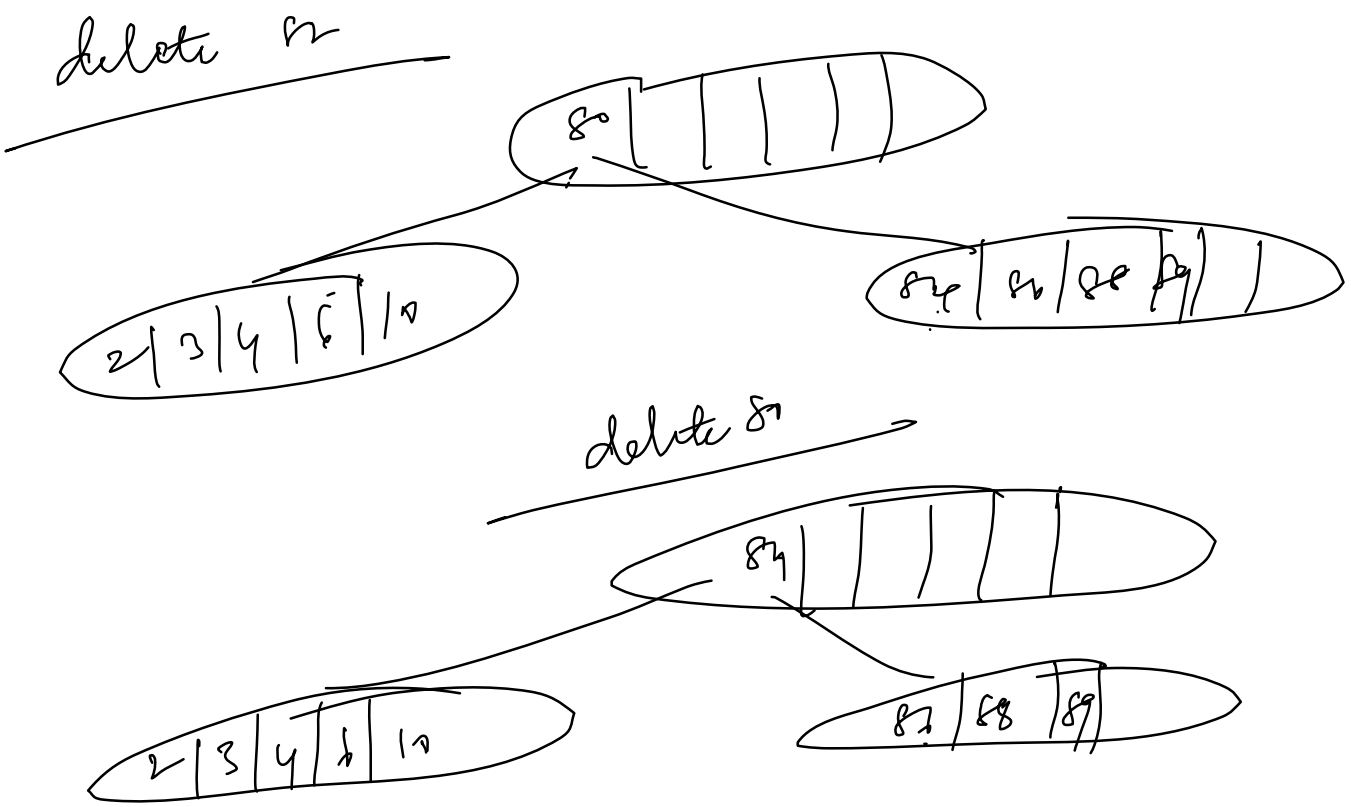






delete 30

if left & right do not have more than the required values we will merge the node with either left or right


delete 50
50
2 3 4 6 10
54 58 59
delete 54
54
2 3 4 6 10
57 58 59

Hashing :- It is a mechanism which is used to represent the data or elements of a dictionary.

Dictionary :- has elements in the form of key, value pairs

Using hashing we store the elements of dictionary in a hash table

Hashing mechanism is generally a modulus function (%) or division function which is applied on the key

$h(key) = key \% D$

where D is the size of the hash table

hash function when applied on key gives the location in the hash table where (key, value) pair is to be stored

The advantage of this hashing mechanism is any operation (insertion, deletion, search) will take O(1) unit of time.

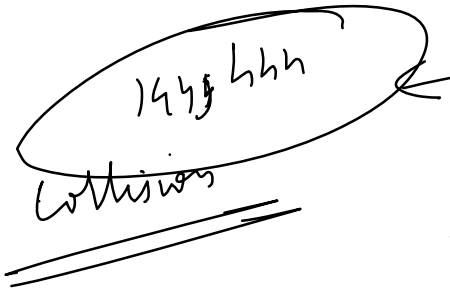
Static Hashing :-

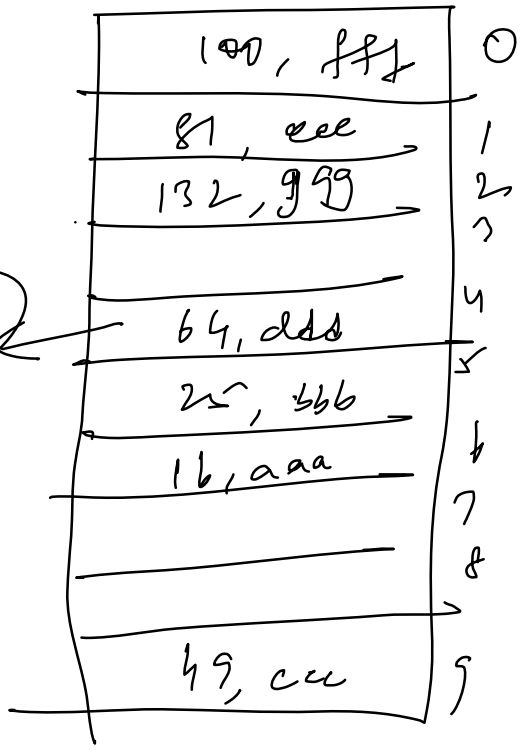
Eg :-

size of hash table = 10

h(key) = key % 10

Key	value
16	aaa
25	bbb
49	ccc
64	ddd
81	eee
100	fff
132	ggg
144	hhh





The problem with static hashing is

a) collisions :- getting a location which is already filled with a value on applying the hash function to the key.

b) overflow :- There are no empty locations in the hash table to place values

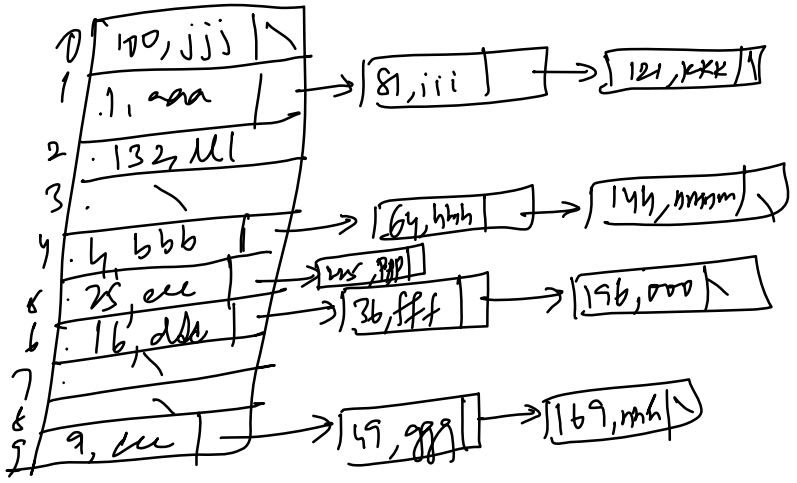
To solve the problems of collisions & overflows we have 2 mechanisms

1) Separate Chaining (solves both collisions & overflows)

2) Open Addressing (solves collisions)

Separate Chaining :- We create an array of linked lists where every location of the hash table is a linked list.

(1, aaa), (4, bbb), (9, ccc), (16, ddd), (25, eee), (36, fff), (49, ggg), (64, hhh), (81, iii), (100, jjj), (121, kkk), (132, lll), (144, mmm), (169, nnn), (196, ooo), (225, ppp). Size of hash table = 10 $h(key) = key \% 10$



Advantage :- unlimited no. of dictionary entries can be stored.

Disadvantage :- maintenance of linked list

Open Addressing :-

a) Linear Probing
b) Quadratic Probing
c) Double Hashing

} size of hash table is fixed

d) rehashing →

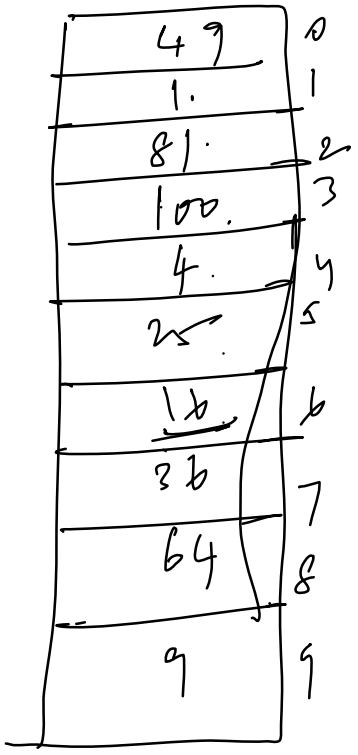
Linear Probing :- If a collision occurs identify the next free location in a circular fashion and store the dictionary entry.

Key — { 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 132, 144, 169, 196 }

Size of hash table = 10

Key % 10

121 → hash table is full



1 % 10 = 1

4 % 10 = 4

9 % 10 = 9

16 % 10 = 6

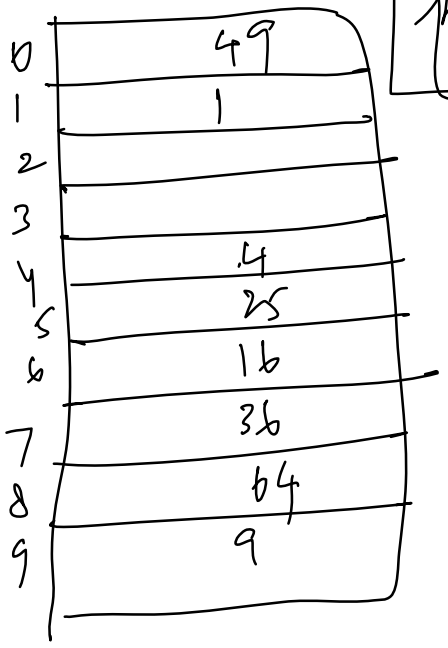
25 % 10 = 5

36 % 10 = 6

49 % 10 = 9

2) Quadratic Probing :- If a collision occurs, the collision is resolved by identify next free location as follows

$$h(h(key) + i^2), \quad i > 0$$



keys = {1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 182, 144}

size = 10

key % 10

36 % 10 = 6 → collision

$h(6+1^2) = h(7) = 7 \% 10 = 7$

49 % 10 = 9 → collision

$h(9+1^2) = h(10) = 10 \% 10 = 0$

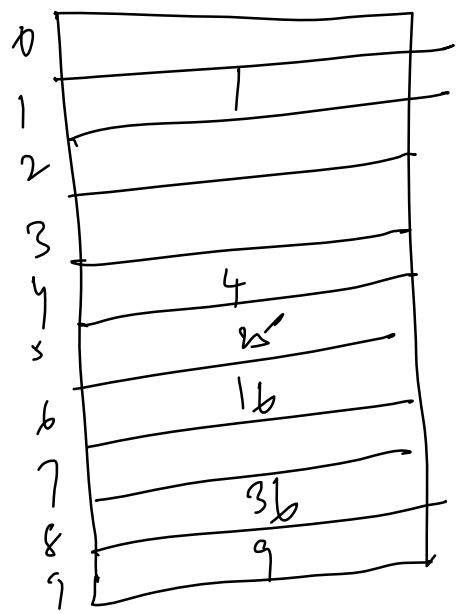
64 % 10 = 4 → collision

$h(4+1^2) = 5 \% 10 = 5$ collision

$h(4+2^2) = 8 \% 10 = 8$

3) Double Hashing :- If a collision occurs we will resolve the collision as follows $h(h(key) + h2(key) * i)$, $i > 0$

$h2(key) = q - key \% q$ where q is a prime no. $q = 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, \ldots$



keys = {1, 4, 9, 16, 25, 36, 49, 64, 81, 100, ...}

size = 10

$h\,36 \% 10 = 6 \rightarrow$ collision

$h2(36) = 2 - 36 \% 2 = 2 - 0 = 2$, $q = 2$, $i = 1$

$h(6 + 2 * 1) = h(8) = 8 \% 10 = 8$

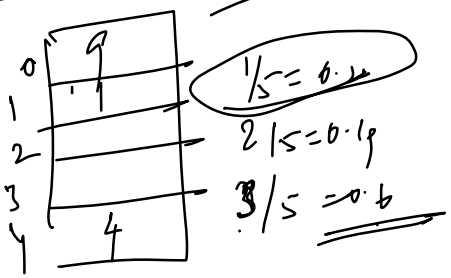
4) rehashing:- It is based on the concept of load factor.. whenever the loadfactor of the hash table is 0.5 the hash table size is doubled and the values are rehashed with new hash function

If a collision occurs either linear/quadratic / double hashing mechanisms are used.

loadfactor = no: of dictionary entries / Size of hash table

Eg:- {1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 132, 144, 169, ...}

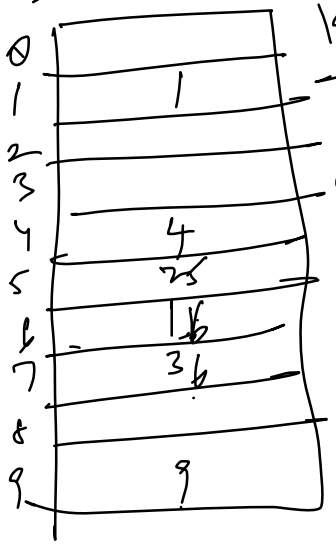
Size = 5 → key % 5



1/5 = 0.2
2/5 = 0.4
3/5 = 0.6

10 % 5 = 0
4 % 5 = 4
9 % 5 = 4
16 % 5 = 1
25 % 5 = 0
36 % 5 = 1

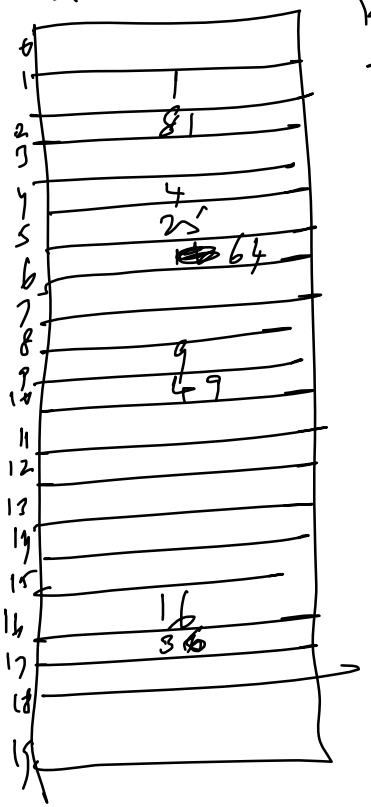
Size = 10 → key % 10



0/10 = 0
0.6 > 0

11 % 10 = 1
4 % 10 = 4
9 % 10 = 9
16 % 10 = 6
25 % 10 = 5
36 % 10 = 6

Size = 20 → key % 20



Pattern Matching :- It is the process of identifying the first occurance position of a pattern string (P) in an original string (T)

T → 0 1 2 3 4 5 6 7 8 9 10 11 12
naresh i Technologies

P → tech

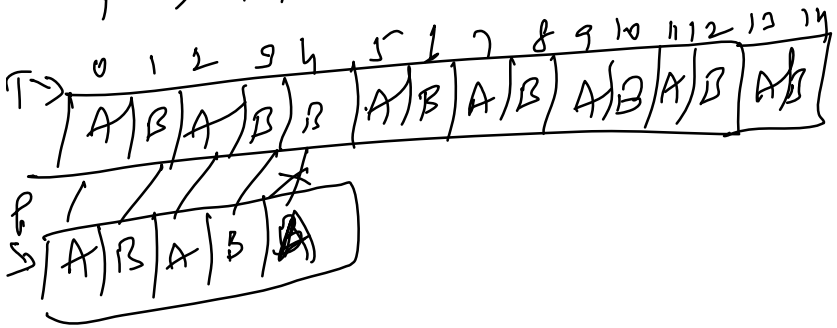
3 Mechanisms

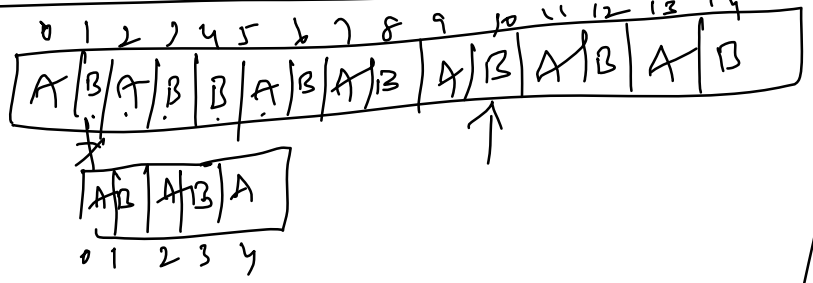
1) Brute Force algorithm
2) Boyer Moore algorithm
3) KMP algorithm

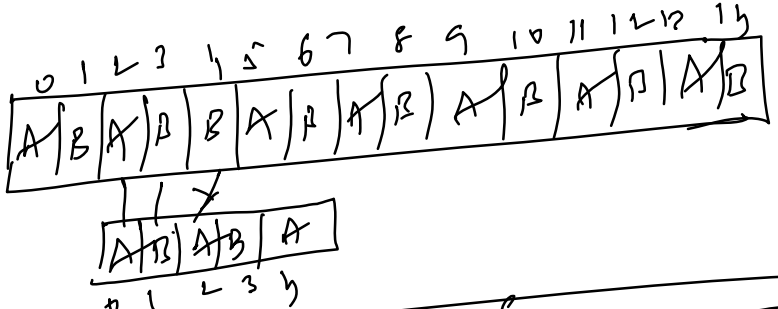
Brute force Algorithm

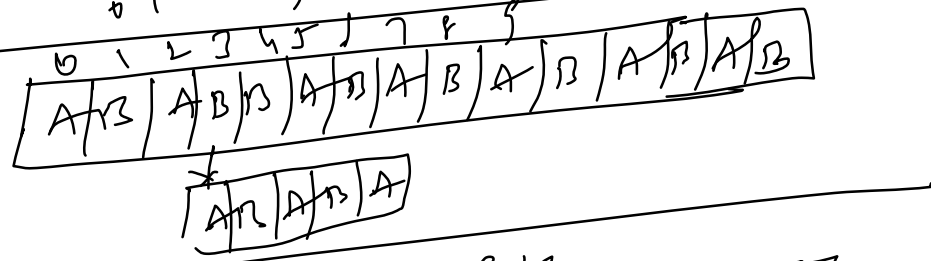
T → ABABBABABABABAB

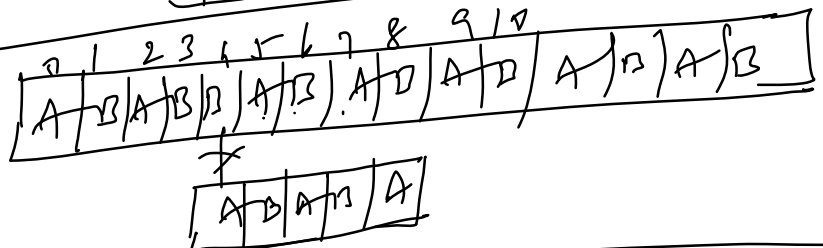
P → ABABA

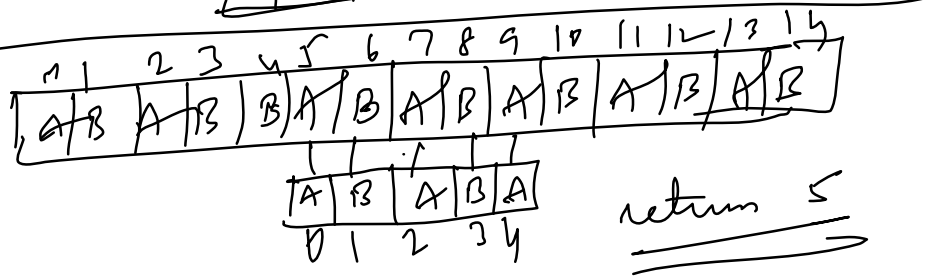












return 5

```
Algorithm BruteForce(T, P)
{
  //T is the original string of length n, P is pattern string of length m
  for i := 0 to n-m step 1 do
  {
    j := 0;
    while( j < m and T[i+j] = P[j]) do
    {
      j := j+1;
    }
    if(j = m) then
      return i;
  }
  return -1;
}
```

Boyer-Moore Algorithm — 2 heuristics

1) looking glass heuristic :- we compare the pattern P with original T from the last position of pattern string.

2) Character jump heuristic :- Here we compute last(c) where c is the character of the string T and last(c) is the last occurance position of c in pattern P

In the process of comparison if a mismatch occurs at jth position of pattern string we compute

1+last(c) where c is the character of orginal string because of which mismatch occurs.

if 1+last(c) <= j shift the position of orginal string by j - last(c) units

if 1+last(c) > j shift the position of orginal string by 1 unit

Eg:: T → ABACAABABACAABACABAABB

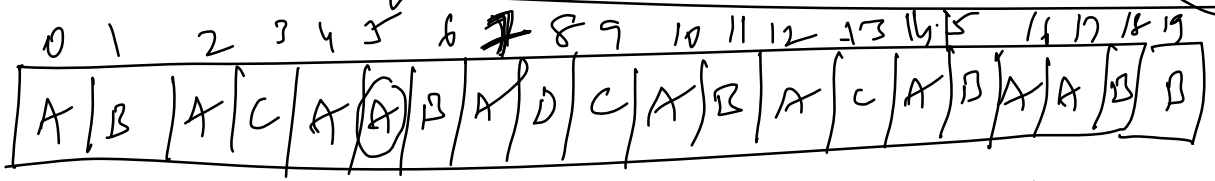
P → ABACAB

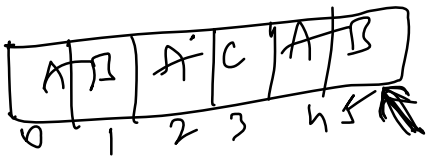
Pattern	A	B	A	C	A	B
Index	0	1	2	3	4	5

char(c)	A	B	C	D
last(c)	4	5	3	-1

i = 5 → T

j = 5 → F

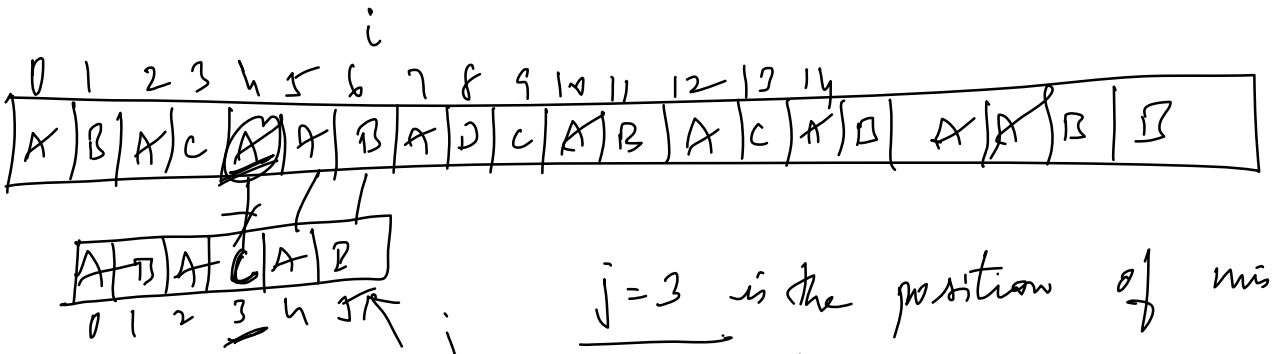




j = 5 is the position of mismatch

1 + last(A) = 1 + 4 = 5 <= 5 ⇒ 1 + last(c) <= j

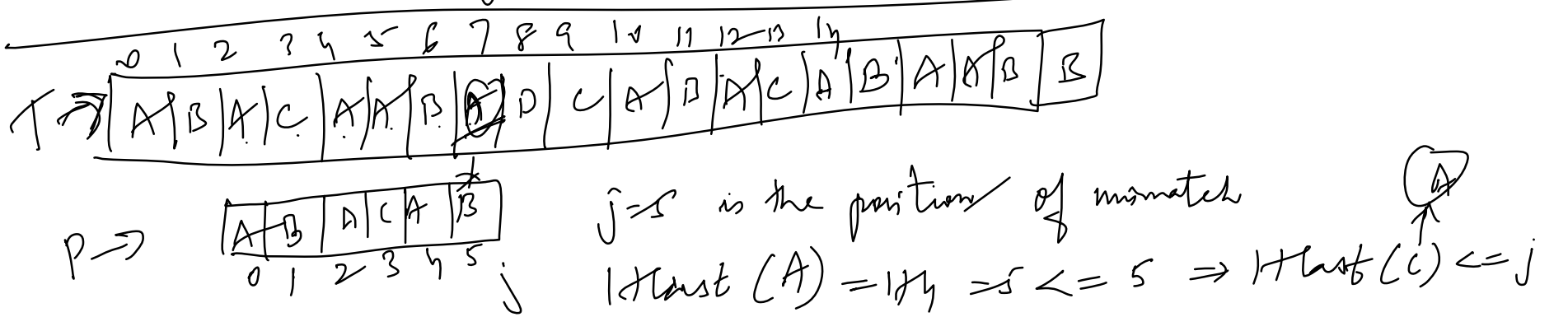
shift i by j − last(A) = 5 − 4 = 1 unit



$j = 3$ is the position of mismatch

$1 + last(A) = 1 + 4 = 5 > 3 \Rightarrow 1 + last(C) > j$

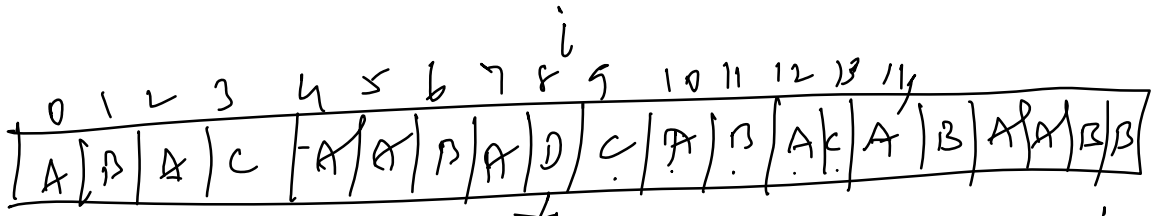
shift by 1 unit



$j = 5$ is the position of mismatch

$1 + last(A) = 1 + 4 = 5 <= 5 \Rightarrow 1 + last(C) <= j$

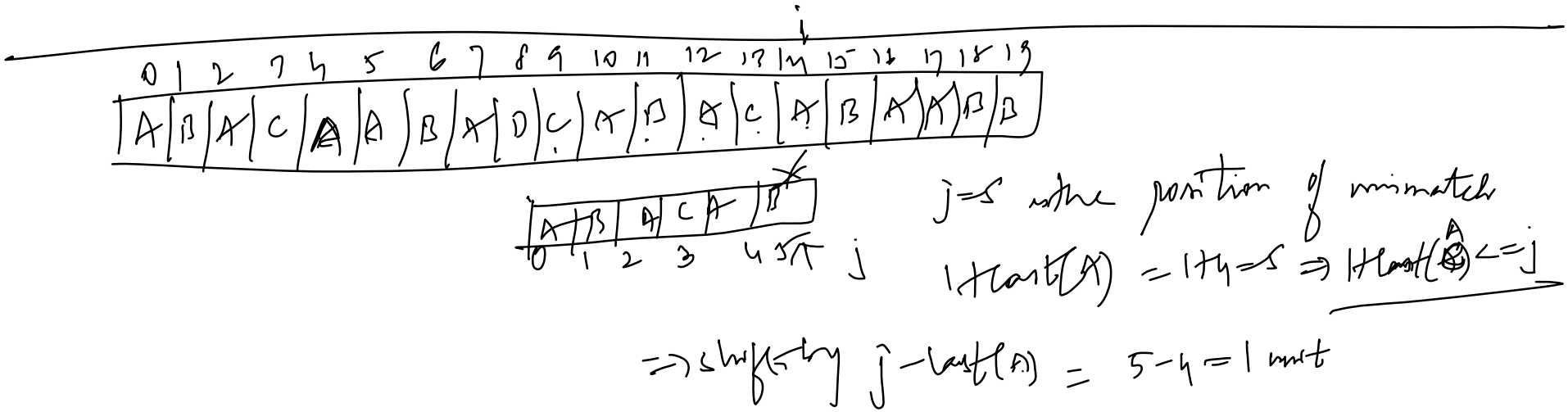
shift by $j - last(A) \Rightarrow 5 - 4 = 1$ unit



j=5 is the position of mismatch

1+last(D) = 1+(-1) = 0 ⇒ 1+last(C) <= j

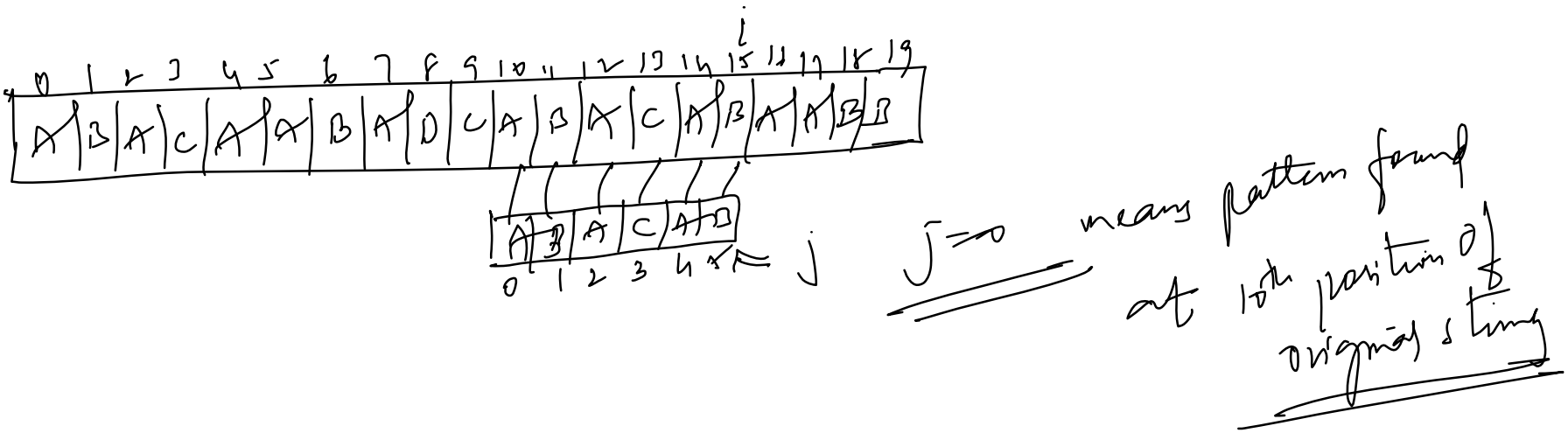
shift by j - last(D) = 5 - (-1) = 6 units



j=5 is the position of mismatch

1+last(A) = 1+4 = 5 ⇒ 1+last(A) <= j

⇒ shifting by j - last(A) = 5-4 = 1 unit


i
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
A B A C A A B A D C A B A C A B A A B B
A B A C A D
0 1 2 3 4 5
j
j == 0 means pattern found at 10th position of original string

```
Algorithm BoyerMoore(T, P)
{
  // T is original string of length n, P is pattern string of length m.
     compute the function last();
     i := m-1;
     j := m-1;
     while(i < n)
      {
        if( P[j] = T(i] ) then
          {
```

```
if (j=0) then
    return i;
else
    i := i-1;
    j := j-1;
}
else
{
i := i + m - min(j, 1 + last(T[i]));
j := m-1;
}
}
return -1;
}
```

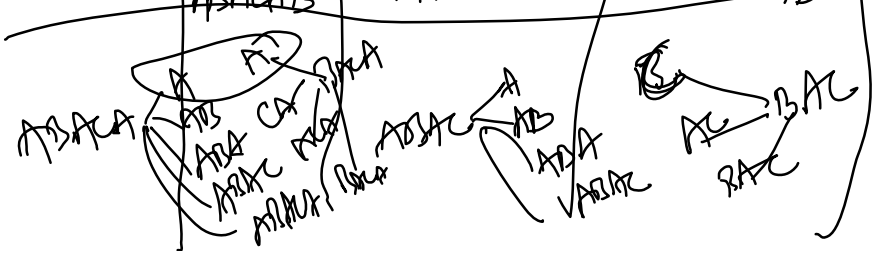
KMP Algorithm:-

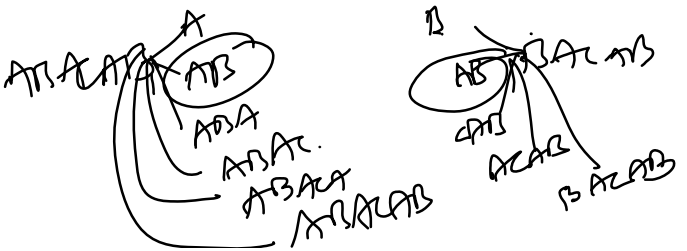
Knuth - Morris - Pratt Algorithm

Failure function :- Π_f is the length of the longest prefix of $P[0..j]$ which is also the suffix of $P[1..j]$ where P is pattern string.

Eg:- P → ABACAB (0 1 2 3 4 5)

j	Pattern P	P[0..j]	P[1..j]	e (string)	f(j) (length)
0	ABACAB	A	–	null	0
1	ABACAB	AB	B	null	0
2	ABACAB	ABA	BA	A	1
3	ABACAB	ABAC	BAC	null	0
4	ABACAB	ABACA	BACA	A	1
5	ABACAB	ABACAB	BACAB	AB	2





Pattern	A	B	A	C	A	B
Index	0	1	2	3	4	5
f(j)	0	0	1	0	1	2

In the process of comparision (i for T, j for P)

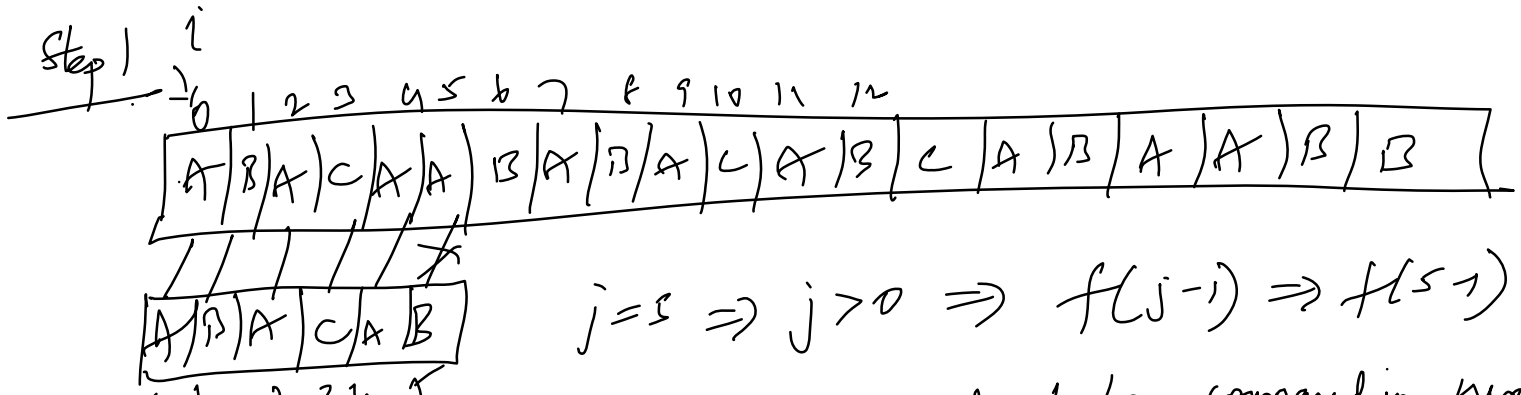
If a mismatch occurs at j^{th} position and if $j \Rightarrow 0$ then increment i by 1 and start comparision.

If mismatch occurs at $j > 0$ → calculate $f(j-1)$ which gives no. of characters that need not be compared in next iteration, i stays from the index where mismatch occured, and pattern starts from $i - f(j-1)$ j starts from $f(j-1)$ position

Eg:- T → ABACAABABACAIBCABAABB

P → ABACAIB

Step 1



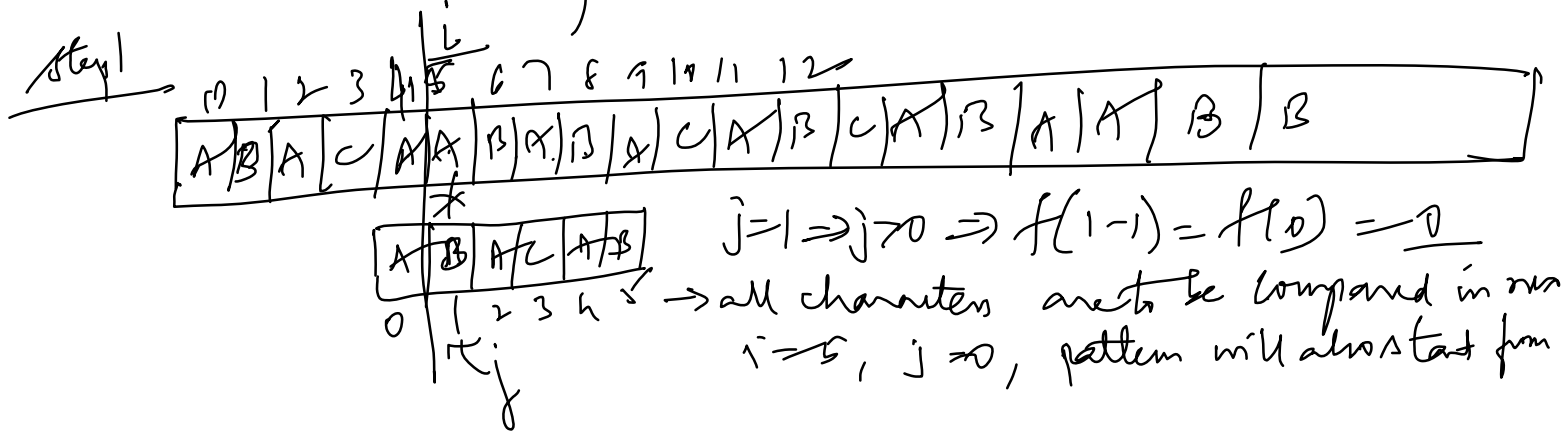
$j=5 \Rightarrow j>0 \Rightarrow f(j-1) \Rightarrow f(5-1)=f(4)=1$

$\Rightarrow$ 1 character need not be compared in next iteration

i will start from 5, j will start from 1

pattern will start from $i-f(j-1) = 5-1 = 4$ pos wrt the original T

Step 1

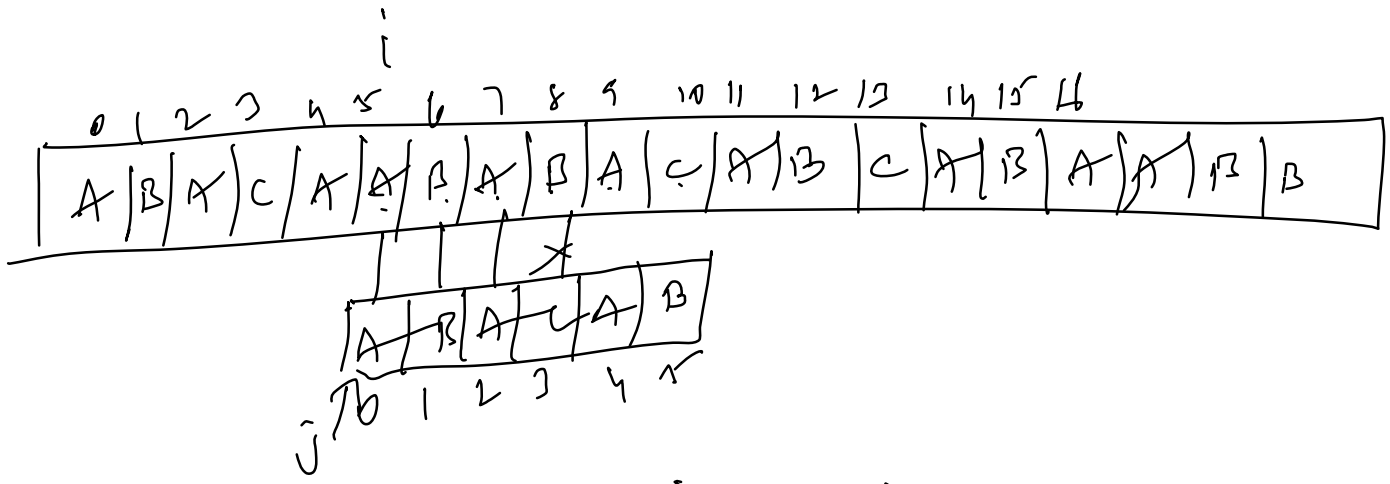


$j=1 \Rightarrow j>0 \Rightarrow f(1-1)=f(0)=0$

$\rightarrow$ all characters are to be compared in next iteration

$i=5$, $j=0$, pattern will also start from $i-f(j-1)=5-0=5$

Step 3



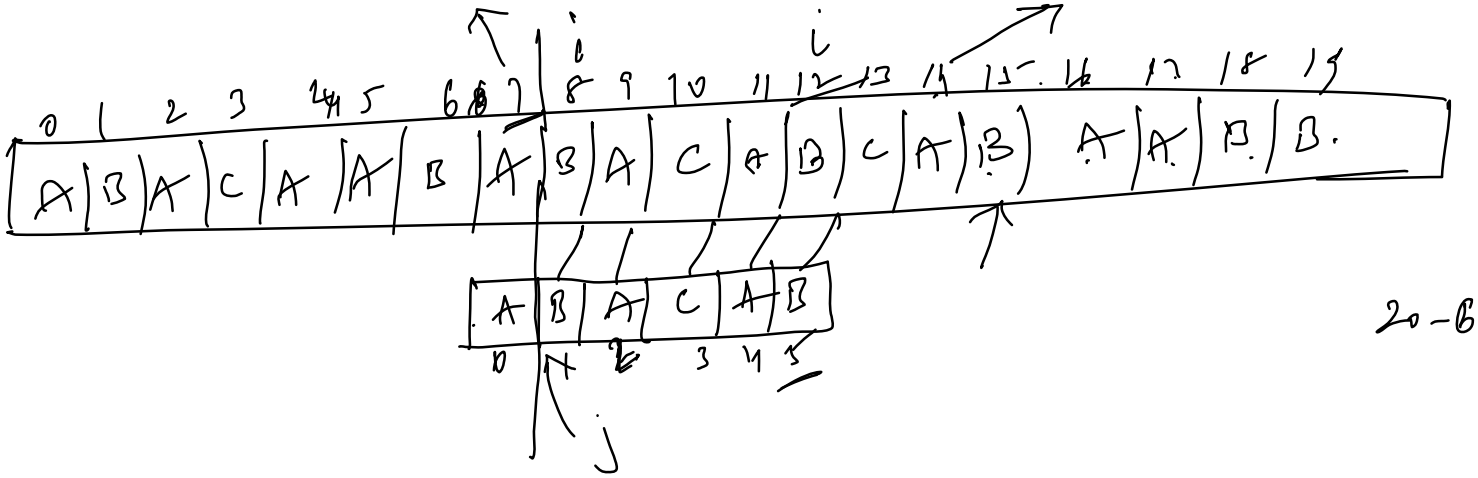
$j = 3 \Rightarrow j > 0 \Rightarrow f(3-1) = f(2) = 1 \Rightarrow$

character need not be compared in next iteration

$i = 8$, pattern wrt T will start from $i - f(j-1) = 8 - 1 = 7$

j will start from $f(j-1) = 1$

Step 4



20 - 6 = 14

return i - m + 1 = 12 - 6 + 1 = 7

12 - 14 + 1 = 7

```
Algorithm KmpMatch(T, P)
{
  // T is the original string of length n, P is pattern string of
  // length m.
      f := KmpFailureFunction(P);
      i := 0;
      j := 0;
  while(i < n-m)
  {
      if( P[j] = T(i])
      {
```

```
if (j = m-1) then
    return i-m+1;
i := i+1;
j := j+1;
}
else if (j > 0)
{
    j := f(j-1);
}
```

```
else
{
    i := i+1;
}
}
return -1;
}
```

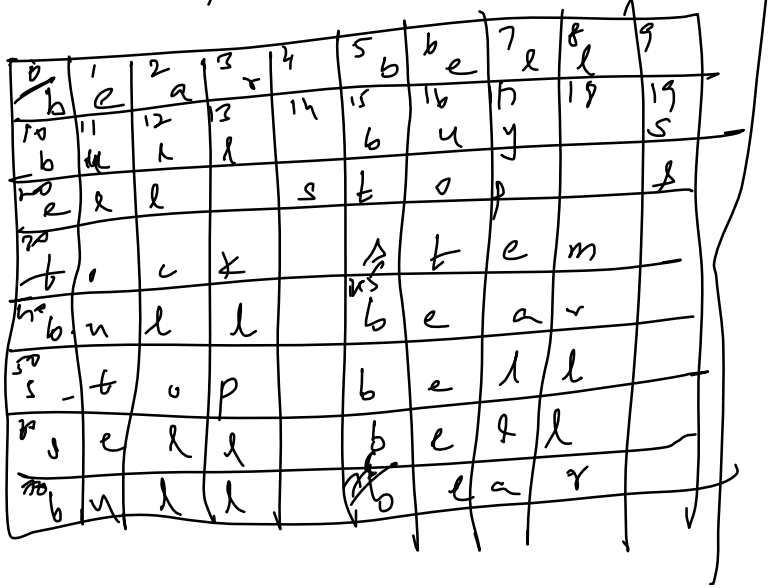
Tries

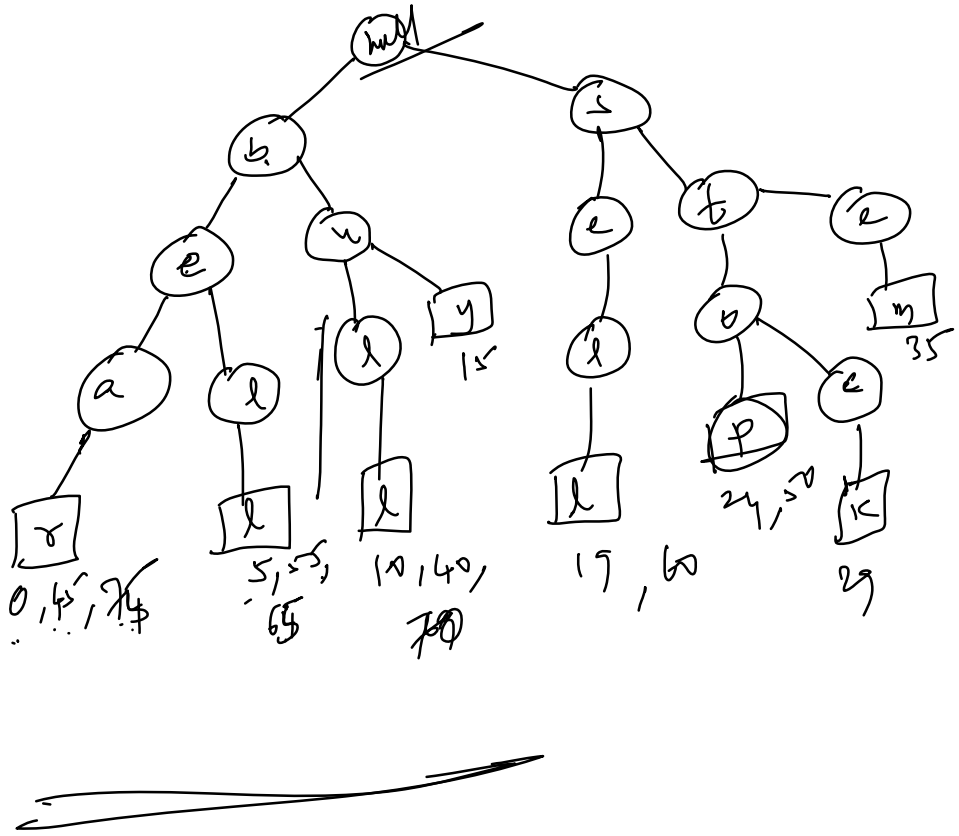
A tree based data structure or a prefix data structure which stores a collection of strings with their starting indices.
Very useful in pattern matching if a pattern occurs more than once in a given original string.

1) Standard Tries
2) Compressed Tries
3) Suffix Tries

Standard Trie

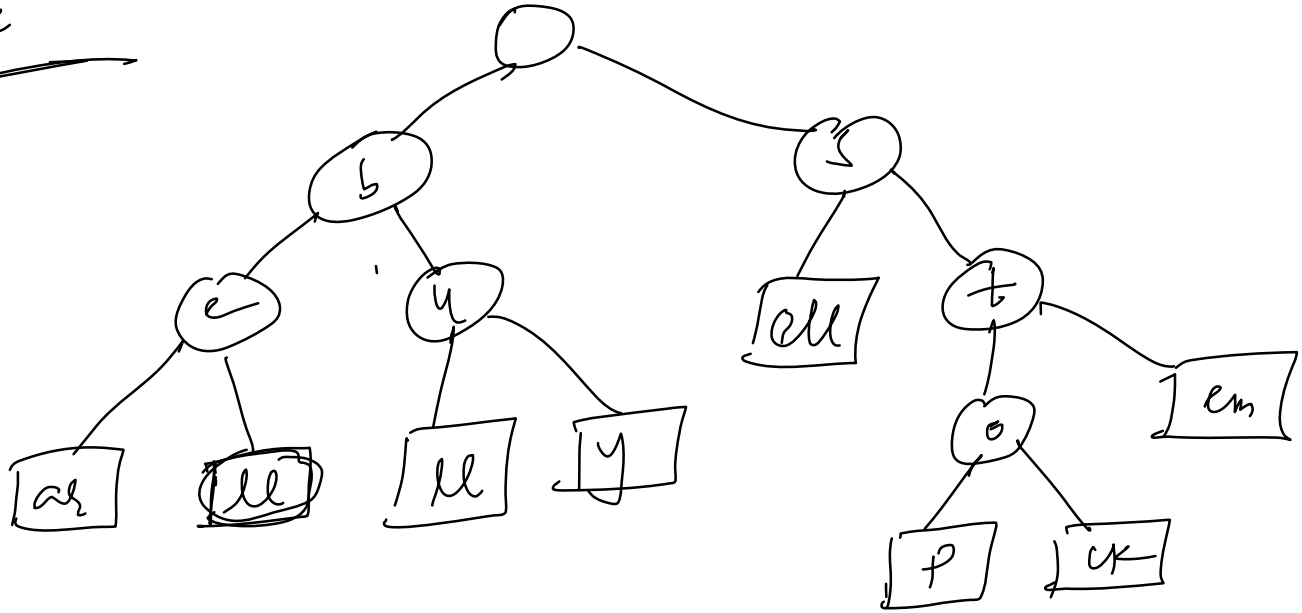
{bear, bell, bull, buy, sell, stop, stock, stem, bull, bear, stop, bell, sell, bell, bull, bear}





Compressed Trie :- It is a trie where every node of the trie must have atleast 2 child nodes. If any node of the trie has a single child, the node and its child node are combined into a single node.

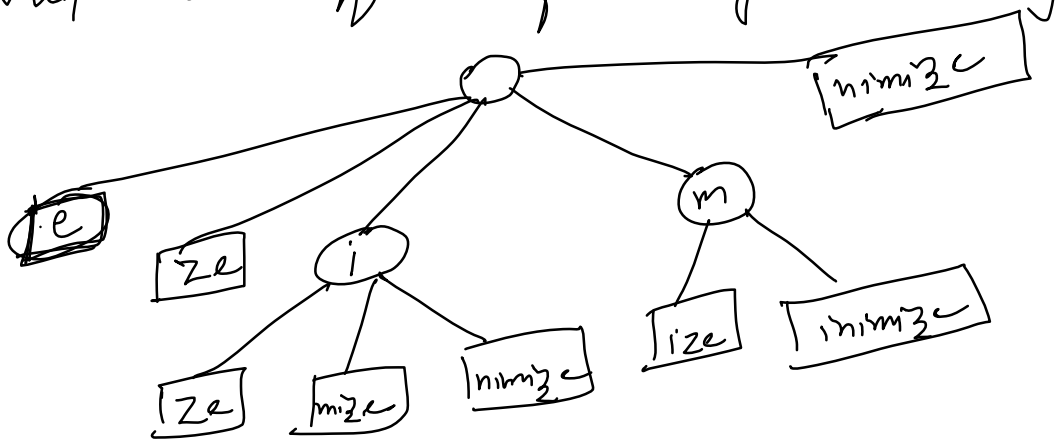
Compressed Trie



Suffix Trie :- It is a compressed tree where the set of strings are generated as suffixes for a given string

→ minimize

{e, ze, ize, mize, imize, nimize, inimize, minimize}



Algorithms :- An algorithm is a sequence of instructions which when followed accomplishes a particular task.

Performance of an Algorithm

1) Space Complexity

2) Time Complexity

Space Complexity is the amount of computer main memory required by the algorithm or program to run to its completion.

Memory is required by constants, arguments of functions, local variables, arrays, recursion stack space.

Time Complexity is the amount of computer time required by the algorithm to run to its completion.

We assume every instruction takes 1 unit of time. Hence time complexity is equal to no: of executable instructions in algorithm

Algorithm Techniques

1) Divide & Conquer → Binary Search, Quick, Merge

2) Greedy Method →

3) Dynamic Programming →

Greedy Method:-
&
Dynamic Programming

} Problems involving the computation of min cost / max profit

Greedy Method:- for any problem we might have more than one solution.

All possible solutions (which satisfy the objective function/ Criterion function) are known as feasible solutions.

The solution which maximizes or minimizes the objective function is called optimal solution

Applications

1) Knapsack Problem → Log (n) n objects p_i / w_i
2) Job Sequencing with deadlines →
3) Single Source Shortest Paths → Dijkstras Algorithm
4) Prims Algorithm } Minimum spanning Trees
5) Kruskals Algorithm. }

Dynamic Programming :-

Principle of optimality :- Whatever may be the initial input and whatever may be the ~~soln~~ solution generated because of the initial input, the remaining inputs must be considered in such a way that we always get an optimal solution.

Applications

All Pairs Shortest Paths

Travelling Salesperson Problem

Optimal Binary Search Tree.

Reliability Design Problem.